Software Modeling & Design

Tourist Agency

May , 2026

MSc. Ari Gjerazi
MSc. Edlira Cani

Alisa Dhima, Henri Shehu, Ermi Xhaferi, Lumturi Bedalli
Amar Kruja, Enian Xhixha, Amanda Stafasani, Era Selmanaj

1. Executive Summary

1.1 Project Overview

This document outlines the functional and non-functional requirements for the Tourist Agency System, an online platform designed to help customers browse, book, and manage tour packages. The system also supports multiple internal roles including booking agents, finance agents, operations agents, administrators, and tourist guides, each with their own set of responsibilities and access levels.

The goal of this project is to replace or digitize manual booking and management workflows with a centralized, web-based application that is reliable, secure, and easy to use for both customers and staff.

2. Product / Service Description

This section provides background context for the Tourist Agency System. It describes the operational environment, the different kinds of users involved, the assumptions behind the design, and the constraints the system must work within.

2.1 Product Context

The Tourist Agency System is a standalone web application. It is not a component of a larger existing platform but will need to integrate with external payment gateways (for processing customer payments) and potentially external email services (for sending booking confirmations and tickets). Internally, the system connects a customer-facing booking interface with a staff-facing management backend, all sharing a single database that tracks packages, bookings, users, and financial records.

2.2 User Characteristics

There are several distinct user groups who will interact with the system:

- Guest Users —> members of the public who visit the platform without logging in. They can browse and search tour packages but cannot book.
- Customers —> registered users who have created an account. They can book tours, manage their profiles, view booking history, make payments, and leave reviews.
- Booking Agents —>internal staff who assist customers by creating or modifying bookings on their behalf. They have access to customer travel history and tour availability.
- Finance Agents —> internal staff who verify payments and review monthly profit and expense reports.
- Operations Agents — >internal staff responsible for creating and updating tour packages, including prices and availability.
- Administrators —>system managers who oversee staff accounts, configure system settings, manage the bus fleet, and view sales reports.
- Tourist Guides —> field staff who confirm client participation and declare collected payments.

2.3 Assumptions

The following assumptions have been made during the requirements gathering phase:

- All users (staff and customers) will access the system through a modern web browser on desktop or mobile devices.
- Customers are assumed to have a basic level of computer literacy and familiarity with online booking platforms.

- A reliable internet connection is expected for all users.
- The system will be hosted on a server with sufficient capacity to handle normal operational load, with room to scale.
- An external payment gateway is assumed to be available and will handle actual financial transactions.

2.4 Constraints and Dependencies

The following constraints apply to the development and operation of this system:

- The system must be compatible with major modern web browsers (Chrome, Firefox, Safari, Edge).
- The system must support both mobile and desktop screen sizes.
- Role-based access control must be implemented, ensuring that each user type can only access the features relevant to their role.
- The payment module depends on integration with a third-party payment gateway. This dependency must be resolved before the booking flow can be fully tested.
- PDF generation for tickets and receipts must be supported natively by the system.
- The database backup mechanism must be automated and must not cause system downtime.

3. Requirements

The following sections describe all functional and non-functional requirements of the Tourist Agency System. Each functional requirement is numbered, given a brief description, and assigned a priority level (1 = must have, 2 = important, 3 = nice to have).

3.1 Functional Requirements

Functional requirements describe what the system shall do, the specific behaviors and operations it must support.

Req #	Requirement	Comments	Priority	Date Rvwd	SWE Reviewed / Approved
BR_TM_01	Guest user shall be able to view, filter, and search tour packages because they need to explore available options	Public access	1	13/3/26	☑
BR_TM_02	Guest user shall be able to create an account and log in because they need to make bookings	Authentication	1	13/3/26	☑
BR_TM_03	Customer shall be able to search and filter tour packages because they want personalized results	Search system	1	13/3/26	☑
BR_TM_04	Customer shall be able to view detailed information about tour packages	Includes price, location	1	13/3/26	☑

Req #	Requirement	Comments	Priority	Date Rvwd	SWE Reviewed / Approved
	because they need to make informed decisions				
BR_TM_05	Customer shall be able to check availability and book tours because they want to reserve trips	Booking process	1	13/3/26	☑
BR_TM_06	Customer shall be able to enter traveler information because booking requires personal details	Names, surname, etc.	1	13/3/26	☑
BR_TM_07	Customer shall be able to make payments because booking must be confirmed	Payment system	1	13/3/26	☑
BR_TM_08	Customer shall be able to submit reviews because they want to share experiences	Feedback system	2	13/3/26	☑
BR_TM_09	Customer shall receive booking confirmation and download tickets/receipts because proof of booking is required	PDF/Email	1	13/3/26	☑
BR_TM_10	Customer shall be able to view booking history because they need to track past trips	User dashboard	2	13/3/26	☑
BR_TM_11	Customer shall be able to cancel bookings because plans may change	Cancellation policy	1	13/3/26	☑
BR_TM_12	Customer shall be able to update personal profile because their information may change	Profile management	2	13/3/26	☑
BR_TM_13	Booking agent shall be able to create bookings for customers because assistance may be needed	Manual booking	1	13/3/26	☑
BR_TM_14	Booking agent shall be able to update or cancel reservations because changes may occur	Reservation control	1	13/3/26	☑
BR_TM_15	Booking agent shall be able to check travel package availability because customers need accurate info	Availability system	1	13/3/26	☑
BR_TM_16	Booking agent shall be able to view customer travel	CRM	2	13/3/26	☑

Req #	Requirement	Comments	Priority	Date Rvwd	SWE Reviewed / Approved
	history because it helps in service personalization				
BR_TM_17	Booking agent shall assist customers with booking modifications because support is required	Customer service	2	13/3/26	☑
BR_TM_18	Booking agent shall be able to reply to reviews because customer engagement is important	Feedback handling	3	13/3/26	☑
BR_TM_19	Finance agent shall be able to verify customer payments because transactions must be validated	Payment verification	1	13/3/26	☑
BR_TM_20	Finance staff shall be able to view monthly profit and expense reports because financial tracking is required	Reports	1	13/3/26	☑
BR_TM_21	Operations agent shall be able to create tour packages because new offers are needed	Package creation	1	13/3/26	☑
BR_TM_22	Operations agent shall be able to update package prices and availability because data changes frequently	Dynamic updates	1	13/3/26	☑
BR_TM_23	Administrator shall be able to create and manage staff accounts because system access must be controlled	User roles	1	13/3/26	☑
BR_TM_24	Administrator shall be able to delete staff accounts because inactive users must be removed	Security	2	13/3/26	☑
BR_TM_25	Administrator shall be able to configure system settings because customization is needed	System control	2	13/3/26	☑
BR_TM_26	Administrator shall be able to view sales reports because performance tracking is required	Analytics	1	13/3/26	☑
BR_TM_27	Administrator shall be able to recruit bus drivers because transportation is required (continuation of27)	HR function	3	13/3/26	☑

Req #	Requirement	Comments	Priority	Date Rvwd	SWE Reviewed / Approved
BR_TM_28	Administrator shall be able to assign buses for service because tours require transportation	Logistics	1	13/3/26	☑
BR_TM_29	Administrator shall be able to register number of buses because fleet management is required	Inventory	2	13/3/26	☑
BR_TM_30	Tourist guide shall confirm clients and collect payments because tour execution requires validation	On-site process	1	13/3/26	☑

SYSTEM BEHAVIOR:

Req #	Requirement	Comments	Priority	Date Rvwd	SWE Reviewed / Approved
BR_SY_01	System shall automatically update inventory after booking because availability must remain accurate	Auto update	1	13/3/26	☑
BR_SY_02	System shall allow users to specify special needs because accessibility is important	Notes field	2	13/3/26	☑
BR_SY_03	System shall generate PDF tickets and receipts because customers need documentation	Inventory	1	13/3/26	☑
BR_SY_04	System shall display popular destinations because it improves user experience	On-site process	3	13/3/26	☑
BR_SY_05	System shall store customer travel history because it supports personalization	Database	2	13/3/26	☑
BR_SY_06	System shall update ratings based on reviews because feedback should affect rankings	Rating system	2	13/3/26	☑

3.2 Non-Functional Requirements

Non-functional requirements define how the system shall perform and the quality attributes it must meet. They do not describe specific features but rather the overall behavior and characteristics of the platform.

3.2.1 Product Requirements

3.2.1.1 Usability Requirements

The system must be straightforward and intuitive for both customers and staff. Key usability requirements include:

- The system shall provide a clean, user-friendly interface that does not require special training for customers to navigate.
- The system shall allow simple, intuitive navigation throughout the booking flow.
- The system shall provide clear and unambiguous booking confirmation messages to the user after each major action.
- The system shall include help documentation or tooltips where needed to guide less experienced users.

3.2.1.2 Performance Requirements

- The system shall load search results within 3 seconds under normal operating conditions.
- The system shall process and confirm a booking within 5 seconds of the customer submitting payment.
- The system shall support multiple simultaneous users without degradation in response time during peak hours.

3.2.1.3 Availability

- The system shall maintain high uptime, targeting at least 99% availability on an annual basis.
- The system shall be capable of recovering automatically or with minimal manual intervention after an unexpected failure.
- Scheduled maintenance windows shall be communicated to users in advance and kept as brief as possible.

3.2.1.4 Security

- The system shall require secure user authentication (e.g. username and password, with support for password reset).
- The system shall store all user passwords in encrypted form — plaintext storage is not permitted.
- The system shall enforce role-based access control, ensuring users can only access the parts of the system appropriate to their role.
- The system shall protect customer personal and payment information in line with applicable data protection standards.
- The system shall maintain activity logs for all significant actions (logins, bookings, payment verifications, admin changes).

3.2.2 Organizational Requirements

- The system shall include technical documentation to support ongoing maintenance and future development by the technical team.
- The system shall be designed to allow easy updates and additions of new features without requiring a full rewrite of existing components.
- Development shall follow agreed coding standards and version control practices.

3.2.3 External Requirements

- The system shall work correctly on all major modern web browsers (Chrome, Firefox, Safari, Edge).
- The system shall be fully functional on both desktop and mobile devices.
- The system shall comply with applicable data protection and privacy legislation for the regions in which it operates.
- The system shall be capable of future expansion to support additional tour packages and an increasing customer base.

4. User Scenarios / Use Cases (by Alisa Dhima)

This section presents all use case specifications for the Tourist Agency System. Each table describes a specific scenario, including the actors involved, the main and alternative sequences of actions, relevant non-functional requirements, and postconditions.

UC Name	BR_TM_01 - View and Search Tour Packages
Summary	Guest user can view, filter, and search available tour packages to explore options.
Dependency	
Actors	Primary: Guest User Secondary: System
Preconditions	1. System is online and accessible. 2. Guest user has access to the website/app.
Description of the Main Sequence	Guest user opens the tour packages page. System displays all available tour packages. Guest user applies filters (location, price). System updates displayed results according to filters. Guest user searches for specific tours using keywords. System displays search results matching criteria.
Description of the Alternative Sequence	If no packages match the filters or search, system displays a "No results found" message.
Non-functional requirements	Page load time must be less than 3 seconds. Search/filter functionality must support at least 1000 users searching at the same time.
Postconditions	Guest user can view and explore the filtered or searched tour packages.

UC Name	BR_TM_02 - Create Account and Login
Summary	Guest user can create an account and log in to the system to make bookings.
Dependency	Depends on BR_TM_01, because the user must access the system to create an account.
Actors	Primary: Guest User Secondary: System
Preconditions	1. System is online and accessible. 2. User doesn't already have an account.
Description of the Main Sequence	Guest user navigates to the registration page. Guest user enters required details to register (name, email, password). System validates and stores user details. Guest user logs in using credentials. System verifies credentials and grants access to account.
Description of the Alternative Sequence	If the email is already registered, system displays an error message. If login fails, system asks to re-enter credentials or password reset.
Non-functional requirements	Password must be stored securely (encrypted). Registration response time ≤ 2 seconds.
Postconditions	Guest user has a valid account and can log in.

UC Name	BR_TM_03 - Search and Filter Tour Packages
Summary	Customer searches and filters packages for personalized results.
Dependency	Depends on BR_TM_02, because in order to process a booking the user has to have an account and be logged in on it.
Actors	Primary: Customer Secondary: System
Preconditions	1. Customer is logged in. 2. Tour package exists.
Description of the Main Sequence	Customer opens search page. Customer enters filters or keywords. System displays matching packages. Customer reviews the results.
Description of the Alternative Sequence	If there are no matches the system displays a "No results found" message.
Non-functional requirements	Loading results must not take more than 2 seconds. Handling 1000 concurrent queries at the same time.
Postconditions	Customer sees personalized tour packages results.

UC Name	BR_TM_04 - View Tour Details
Summary	Customer views detailed information about a tour package.
Dependency	Depends on BR_TM_03, because in order to view a package's details, the customer first needs to search for it and select it.
Actors	Primary: Customer Secondary: System
Preconditions	1. Customer is logged in. 2. Tour package exists.
Description of the Main Sequence	Customer selects a package. System shows details (price, location, itinerary).
Description of the Alternative Sequence	If the package is removed from the website the system shows a "Package not available" message.
Non-functional requirements	Loading the details page <= 2 seconds.
Postconditions	Customer has full package information.

UC Name	BR_TM_05 - Book Tour Package
Summary	Customer checks availability and books tours.
Dependency	Depends on BR_TM_03 and BR_TM_04, because in order to book a tour package the customer needs to search for it, view its details and select it.
Actors	Primary: Customer Secondary: System
Preconditions	1. Customer is logged in. 2. Tour package is available.
Description of the Main Sequence	Customer selects package and views availability. Enter traveler info. Customer confirms booking. Customer makes payment. System verifies payment and confirms.
Description of the Alternative Sequence	If the tour is not available, the system suggests other alternatives. If the payment process fails the system prompts the user to retry.
Non-functional requirements	Booking confirmation must not take more than 2 minutes. PCI DSS compliance.
Postconditions	Booking confirmed and payment processed.

UC Name	BR_TM_06 - Enter Traveler Information
Summary	Customer provides personal details for booking or the details of the person he is booking it for.
Dependency	Depends on BR_TM_05, customer must proceed with a booking and then enter the traveler details.
Actors	Primary: Customer Secondary: System
Preconditions	1. Customer is logged in. 2. Booking in process.
Description of the Main Sequence	Customer enters names, surnames, contact info. System validates required fields. System saves the data.
Description of the Alternative Sequence	If there are missing or invalid fields the system shows an error.
Non-functional requirements	Form validation <= 1 second. Secured data entry.
Postconditions	Traveler information stored for booking.

UC Name	BR_TM_07 - Make Payment
Summary	Customer completes payment to confirm booking.
Dependency	Depends on BR_TM_05 and BR_TM_06.
Actors	Primary: Customer Secondary: System
Preconditions	1. Booking is selected. 2. Traveler information is entered.
Description of the Main Sequence	Customer selects payment method. Customer enters the payment information. System processes payment. Payment is confirmed by system. System updates booking status of the tour package.
Description of the Alternative Sequence	If the current payment method fails, the system asks to retry or change method.
Non-functional requirements	Response must take no longer than 5 seconds. PCI DSS compliant.
Postconditions	Booking is confirmed and payment recorded.

UC Name	BR_TM_08 - Submit Review
Summary	Customer submits review for tours experienced.
Dependency	Depends on BR_TM_05.
Actors	Primary: Customer Secondary: System
Preconditions	Customer must have completed a tour.
Description of the Main Sequence	Customer navigates to review page. Enter rating and comments for completed tours. Customer submits review. System stores the review. System updates rating.
Description of the Alternative Sequence	If there is an invalid input, the system shows an error.
Non-functional requirements	Review submission <= 2 seconds.
Postconditions	A new review is stored and the tour rating is updated.

UC Name	BR_TM_09 - Receive Booking Confirmation
Summary	Customer receives booking confirmation and PDF tickets.
Dependency	Depends on BR_TM_05 and BR_TM_07, the customer should have made a booking and the payment for it in order to receive a confirmation.
Actors	Primary: Customer Secondary: System
Preconditions	Booking must be done and paid for.
Description of the Main Sequence	After payment is confirmed, system generates PDF ticket/receipt. Send via email or dashboard to customer. System stores copies of the PDFs.
Description of the Alternative Sequence	If the email delivery of the PDF fails, the system prompts a download link.
Non-functional requirements	PDF must be accessible on all devices. Confirmation should not take more than 2 minutes.
Postconditions	Customer has proof of booking.

UC Name	BR_TM_10 - View Booking History
Summary	Customer views past bookings for reference.
Dependency	BR_TM_05, the customer should have made a booking previously.
Actors	Primary: Customer Secondary: System
Preconditions	1. Customer is logged in. 2. Past bookings exist.
Description of the Main Sequence	Customer navigates to his profile. Clicks on booking history. System displays the booking history. Customer reviews details of his past bookings.
Description of the Alternative Sequence	If there are no bookings a "No history" message is displayed.
Non-functional requirements	The dashboard should load in 2 seconds at most.
Postconditions	Customer sees all previous bookings.

UC Name	BR_TM_11 - Cancel Booking
Summary	Customer cancels a booking according to the cancellation policy.
Dependency	Depends on BR_TM_05.
Actors	Primary: Customer Secondary: System
Preconditions	1. Customer is logged in. 2. Booking exists.
Description of the Main Sequence	Customer clicks on his active bookings. Customer selects booking to cancel. System checks cancellation eligibility. Customer confirms cancellation. System updates booking status.
Description of the Alternative Sequence	If the booking is not eligible the system will show a message.
Non-functional requirements	Cancellation confirmation <= 2 seconds.
Postconditions	Booking is canceled and the system is updated.

UC Name	BR_TM_12 - Update Personal Profile
Summary	Customer updates personal information in their profile.
Dependency	Depends on BR_TM_02.
Actors	Primary: Customer Secondary: System
Preconditions	Customer is logged in.
Description of the Main Sequence	Customer navigates to his profile page. Update information (name, contact, etc.). Click on Save Changes button. System validates the changes. System updates database.
Description of the Alternative Sequence	If the user enters invalid data, the system shows an error.
Non-functional requirements	Profile update <= 2 seconds. Data security is provided.
Postconditions	Customer profile is updated.

UC Name	BR_TM_13 - Create Booking for Customer
Summary	Booking agent manually creates bookings for customers.
Dependency	Depends on BR_TM_05.
Actors	Primary: Booking Agent Secondary: System
Preconditions	1. Customer details available. 2. Tour package exists.
Description of the Main Sequence	Booking agent enters customer details. Selects tour package and dates. System checks availability. Booking confirmed. System updates records.
Description of the Alternative Sequence	If package is unavailable at the moment, the system suggests alternates.
Non-functional requirements	Booking process <= 2 minutes.
Postconditions	Booking created in the system.

UC Name	BR_TM_14 - Update or Cancel Reservations
Summary	Booking agent updates or cancels existing reservations for a customer.
Dependency	Depends on BR_TM_13.
Actors	Primary: Booking Agent Secondary: System
Preconditions	Booking exists.
Description of the Main Sequence	The booking agent selects the active booking. Update details or cancel the booking. System verifies changes. Booking status updated.
Description of the Alternative Sequence	If invalid updates are tried to be made, the system will show an error.
Non-functional requirements	Response <= 2 minutes. Secure handling.
Postconditions	Booking updated or canceled.

UC Name	BR_TM_15 - Check Travel Package Availability
Summary	Booking agent verifies availability for customer tours.
Dependency	
Actors	Primary: Booking Agent Secondary: System
Preconditions	Tour packages exist.
Description of the Main Sequence	Booking agent searches for packages. System displays current availability.
Description of the Alternative Sequence	If a certain tour package is not available the system suggests other alternatives.
Non-functional requirements	Response <= 2 minutes.
Postconditions	Availability information on the tour packages provided.

UC Name	BR_TM_16 - View Customer Travel History
Summary	Booking agent views customer travel history for personalized service.
Dependency	BR_TM_13.
Actors	Primary: Booking Agent Secondary: System
Preconditions	Customer exists in database (has made previous bookings).
Description of the Main Sequence	Booking agent selects a customer. Clicks on customer's profile. Navigates to booking history. System displays past bookings. Booking agent reviews the past bookings of a customer.
Description of the Alternative Sequence	If the customer has no previous bookings the system shows a message "No history".
Non-functional requirements	Retrieval of information <= 2 seconds.
Postconditions	Customer history is displayed.

UC Name	BR_TM_17 - Assist Customers with Booking Modifications
Summary	Booking agent assists customers to modify existing bookings.
Dependency	BR_TM_13 and BR_TM_14.
Actors	Primary: Booking Agent Secondary: System
Preconditions	1. Booking exists. 2. Customer requests modification.
Description of the Main Sequence	Booking agent receives a modification request. Apply changes in system. Confirm modification with customer. Submit changes.
Description of the Alternative Sequence	If modification is not valid the system shows an error.
Non-functional requirements	Update <= 2 seconds. Secure handling.
Postconditions	Booking is modified successfully for a customer.

UC Name	BR_TM_18 - Reply to Customer Reviews
Summary	Booking agent responds to customer reviews.
Dependency	BR_TM_08.
Actors	Primary: Booking Agent Secondary: System
Preconditions	Reviews on a tour package exist.
Description of the Main Sequence	Booking agent logs into his account. Selects a tour package. Select a review. Enter a response. System posts response to customer dashboard.
Description of the Alternative Sequence	Response invalid or inappropriate — the system shows an error and fails to post the answer.
Non-functional requirements	Response <= 2 seconds. Visible to customer.
Postconditions	Customer receives a reply on his review.

UC Name	BR_TM_19 - Verify Customer Payments
Summary	Finance agent validates customer payments.
Dependency	BR_TM_07.
Actors	Primary: Finance Agent Secondary: System
Preconditions	Payment initiated from customer.
Description of the Main Sequence	Finance agent reviews payment transaction. System validates transaction. Confirm valid payment. Update booking.
Description of the Alternative Sequence	If payment is invalid, then alert the agent.
Non-functional requirements	Verification <= 2 seconds. Secure financial processing.
Postconditions	Payment is in status: verified or flagged.

UC Name	BR_TM_20 - View Monthly Profit and Expense Reports
Summary	Finance staff views reports for financial tracking.
Dependency	
Actors	Primary: Finance Agent Secondary: System
Preconditions	Financial data is available.
Description of the Main Sequence	Finance agent accesses reporting module. Agent selects a month. System generates profit/expense report. Finance agent reviews reports.
Description of the Alternative Sequence	If any data is missing, the system shows a message.
Non-functional requirements	Report <= 3 seconds.
Postconditions	Financial reports displayed for business purposes.

UC Name	BR_TM_21 - Create Tour Packages
Summary	Operations agent creates new tour packages.
Dependency	
Actors	Primary: Operations Agent Secondary: System
Preconditions	System is accessible online.
Description of the Main Sequence	Enter tour details (price, location, itinerary, duration). Operation agent saves package. System validates the data. System makes new packages visible to customers.
Description of the Alternative Sequence	If any invalid data entered, the system shows an error.
Non-functional requirements	Creation <= 2 seconds.
Postconditions	A new package stored in the system.

UC Name	BR_TM_22 - Update Package Prices and Availability
Summary	Operations agent updates package details for dynamic pricing and availability.
Dependency	Depends on BR_TM_21.
Actors	Primary: Operations Agent Secondary: System
Preconditions	Package is created previously and exists.
Description of the Main Sequence	Operations agent selects a package. Update price or availability. Save the updated package. System validates the new information. System updates packages.
Description of the Alternative Sequence	If any invalid update is tried to be made, the system shows an error.
Non-functional requirements	Update should not take more than 2 seconds. Real-time availability.
Postconditions	Tour packages updated.

UC Name	BR_TM_23 - Create and Manage Staff Accounts
Summary	Administrator manages staff accounts for system access control.
Dependency	
Actors	Primary: Administrator Secondary: System
Preconditions	Admin is logged into his account.
Description of the Main Sequence	Go to "Create New Account". Add new staff details. System validates and creates account. Manage access roles and permissions.
Description of the Alternative Sequence	If any invalid data entered, the system shows an error.
Non-functional requirements	Account creation should not take more than 2 seconds. Secure access.
Postconditions	Active and personalized staff accounts.

UC Name	BR_TM_24 - Delete Staff Accounts
Summary	Administrator deletes inactive staff accounts.
Dependency	BR_TM_23.
Actors	Primary: Administrator Secondary: System
Preconditions	1. Admin is logged into his account. 2. Staff account exists.
Description of the Main Sequence	Select staff account. Admin reviews the account. Confirm deletion. System removes account.
Description of the Alternative Sequence	If the account is not found, the system shows an "Account not found" message.
Non-functional requirements	Account deletion should not take more than 2 seconds. Secure removal.
Postconditions	Inactive staff accounts deleted from the system.

UC Name	BR_TM_25 - Configure System Settings
Summary	Administrator configures system preferences.
Dependency	
Actors	Primary: Administrator Secondary: System
Preconditions	Admin is logged into his account.
Description of the Main Sequence	Access settings module. Update configurations (theme, policies, modules). Save the changes. System applies changes.
Description of the Alternative Sequence	If any invalid settings are tried to be applied, an error will show.
Non-functional requirements	Updates on the system should not take more than 2 seconds. Persistent storage.
Postconditions	Configured system in compliance with employee competences.

UC Name	BR_TM_26 - View Sales Reports
Summary	Administrator tracks system performance via sales reports.
Dependency	
Actors	Primary: Administrator Secondary: System
Preconditions	Sales data and reports exist.
Description of the Main Sequence	Administrator accesses analytics module. Select a report type and time slot (month). System generates the reports. Administrator reviews the reports.
Description of the Alternative Sequence	If the selected financial data or reports do not exist display a message "Data not available".
Non-functional requirements	The report should not take longer than 3 seconds to load.
Postconditions	Sales report available to the administrator.

UC Name	BR_TM_27 - Recruit Bus Drivers
Summary	Administrator recruits drivers for tour transportation.
Dependency	
Actors	Primary: Administrator Secondary: System
Preconditions	Recruitment process active.
Description of the Main Sequence	Login as admin. Navigate to buses window. Enter driver details. System stores and validates the information for the driver.
Description of the Alternative Sequence	If any invalid data are entered, an error will show. If duplicated input detected, it will show error.
Non-functional requirements	Data stored on the system should not take more than 2 seconds.
Postconditions	Drivers for tours registered.

UC Name	BR_TM_28 - Assign Buses for Service
Summary	Administrator assigns buses to tour schedules.
Dependency	Depends on BR_TM_27.
Actors	Primary: Administrator Secondary: System
Preconditions	1. Bus fleet exists. 2. Tour is scheduled.
Description of the Main Sequence	Login as admin. Click on "Tours". Select a tour. Assign a bus to a certain tour. System updates schedule.
Description of the Alternative Sequence	If the bus is unavailable at the moment, the system will suggest other options.
Non-functional requirements	Update of the tours with the respective buses shouldn't take longer than 2 seconds.
Postconditions	Buses assigned for different tours.

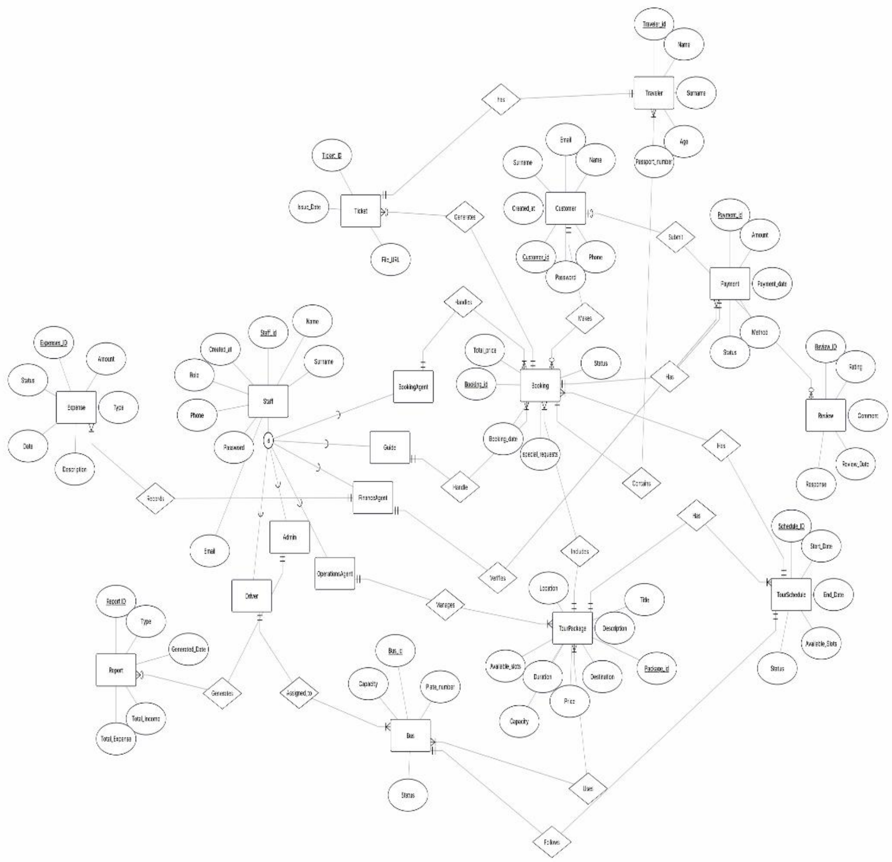
UC Name	BR_TM_29 - Register Number of Buses
Summary	Administrator records fleet information for management.
Dependency	
Actors	Primary: Administrator Secondary: System
Preconditions	System must be accessible online.
Description of the Main Sequence	Login as admin. Navigate to "Buses" window. Enter bus details. System validates and stores.
Description of the Alternative Sequence	If any invalid data are entered, an error will show. If duplicated input detected, it will show error.
Non-functional requirements	Accurate inventory. Process shouldn't take more than 2 seconds.
Postconditions	Always updated bus fleets.

UC Name	BR_TM_30 - Confirm Clients and Collect Payments
Summary	Tourist guide confirms client presence and collects payment on-site.
Dependency	Depends on BR_TM_05 and BR_TM_07.
Actors	Primary: Tourist Guide Secondary: Customer, System
Preconditions	Customer has booked a tour.
Description of the Main Sequence	Confirm client attendance. Collect payment from client. Record transaction in system. System stores the data.
Description of the Alternative Sequence	In case of any payment discrepancy, report the case to the administrator.
Non-functional requirements	Recording of transactions takes less than 1 minute. Secure handling.
Postconditions	Payment recorded and client confirmed.

5. Diagrams

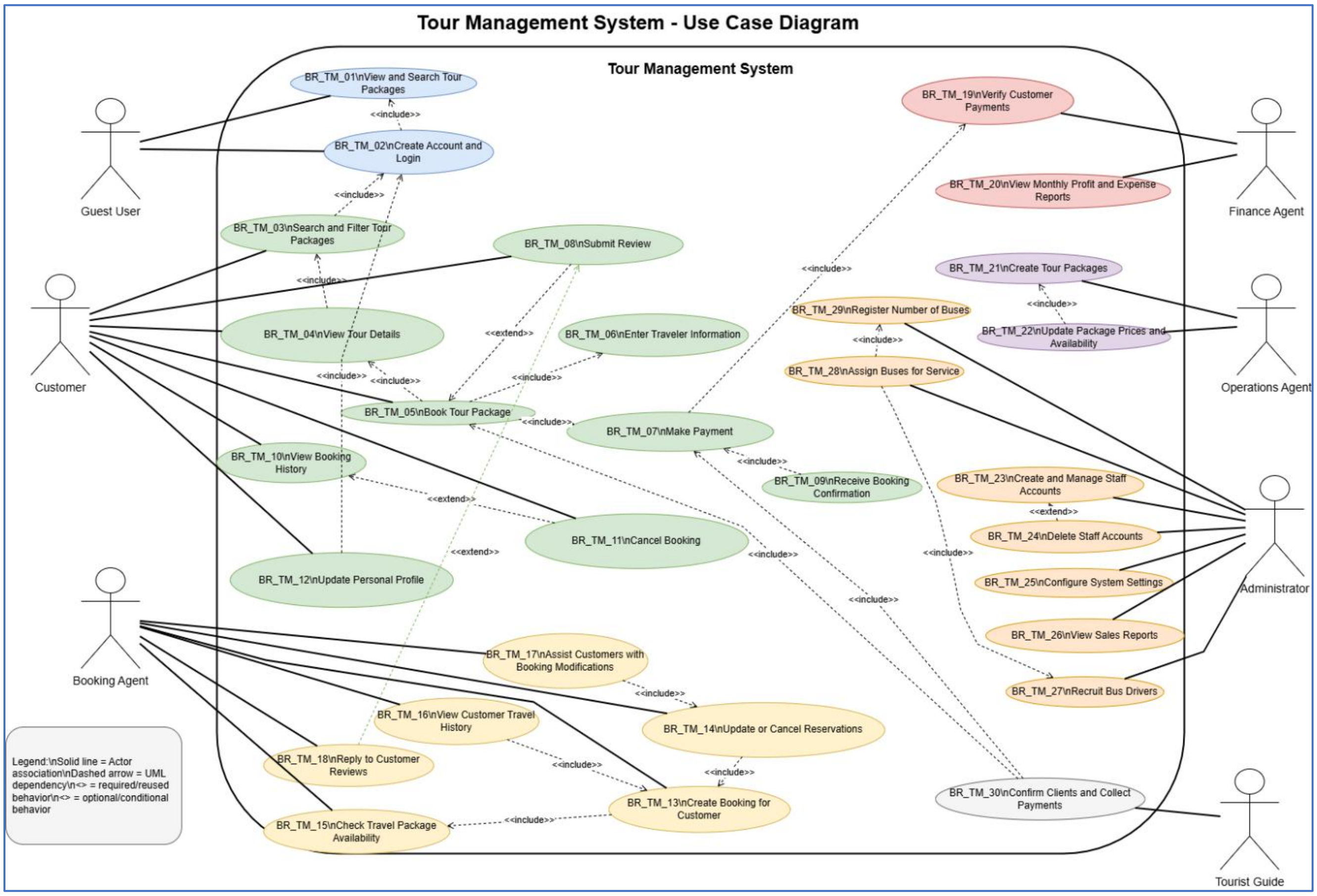
This section is reserved for all technical diagrams produced during the design and analysis phases of the project.

5.1 ER Diagram (by Ermi Xhaferi & Amar Kruja)



Link for image: [Tourist-Agency-Information-System/ERDiagram.png at main · Ermi-Xh/Tourist-Agency-Information-System]

5.2 Use Case Diagram (by Henri Shehu)

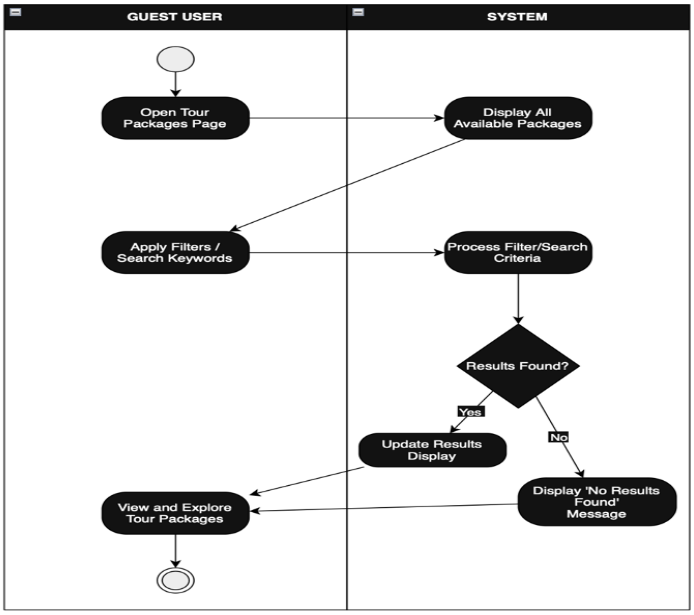


Link for image: [Tourist-Agency-Information-System/UseCaseDiagram.png at main · Ermi-Xh/Tourist-Agency-Information-System]()

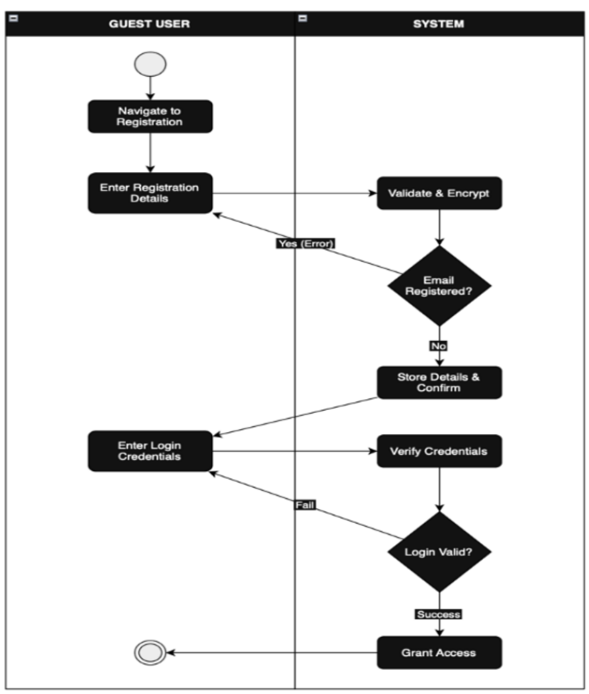
5.3 Activity Diagrams

ERMI XHAFERRI

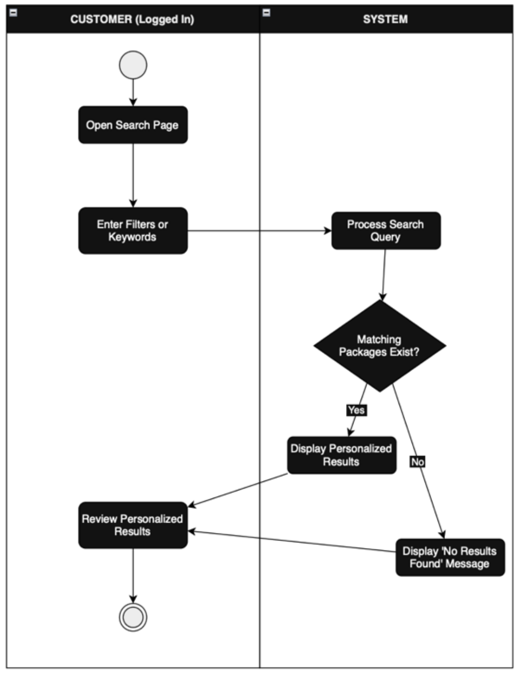
BR_TM_01 - View and Search Tour Packages (Guest User)



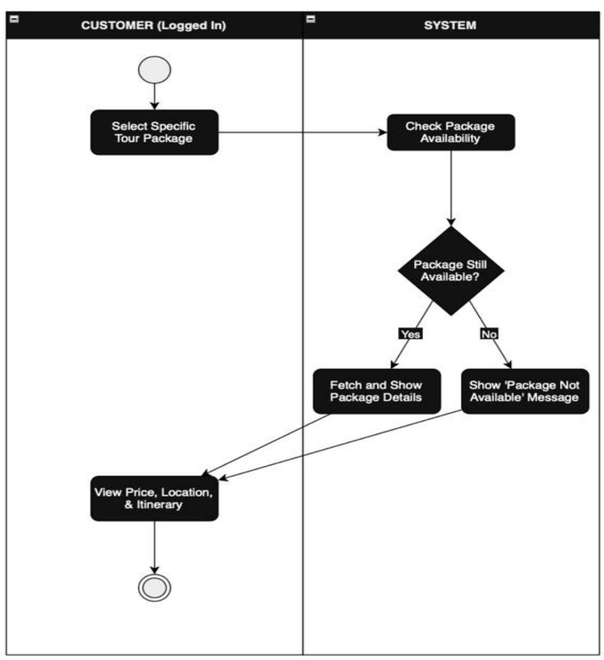
BR_TM_02 - Create Account and Login (Guest User)



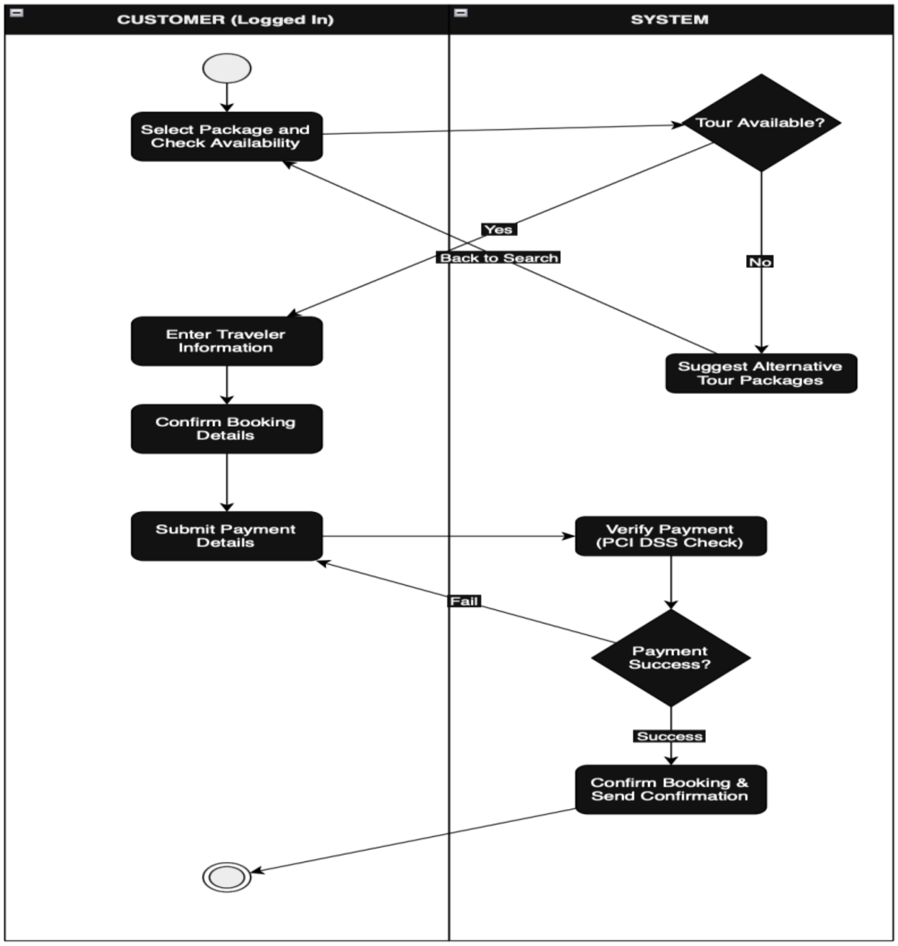
BR_TM_03 - Customer searches and filters tour packages



BR_TM_04 - Customer views detailed information about a tour package

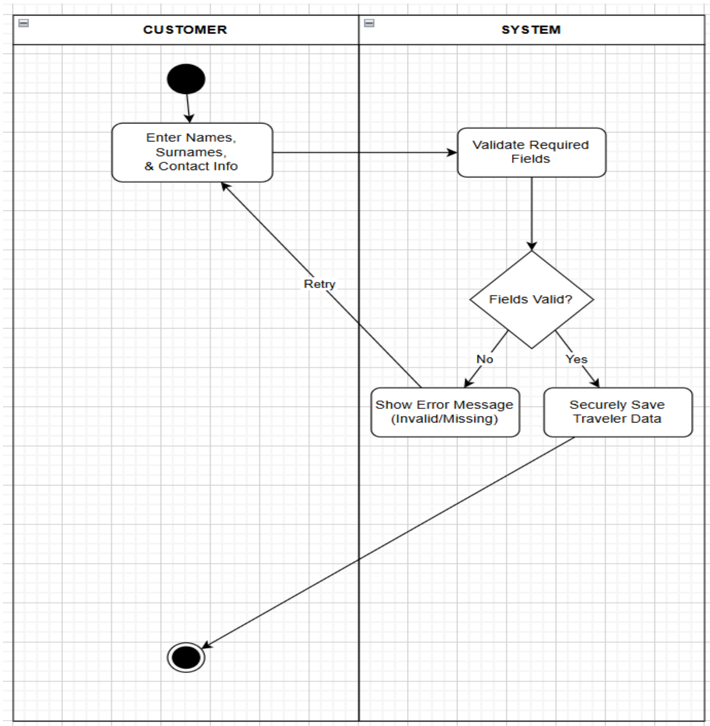


BR_TM_05 – Customer books a tour package

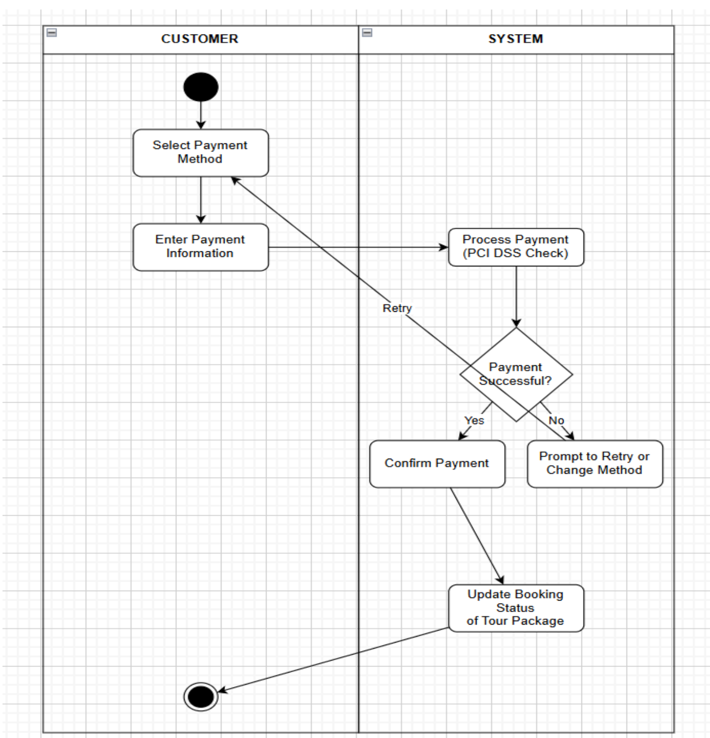


AMAR KRUJA

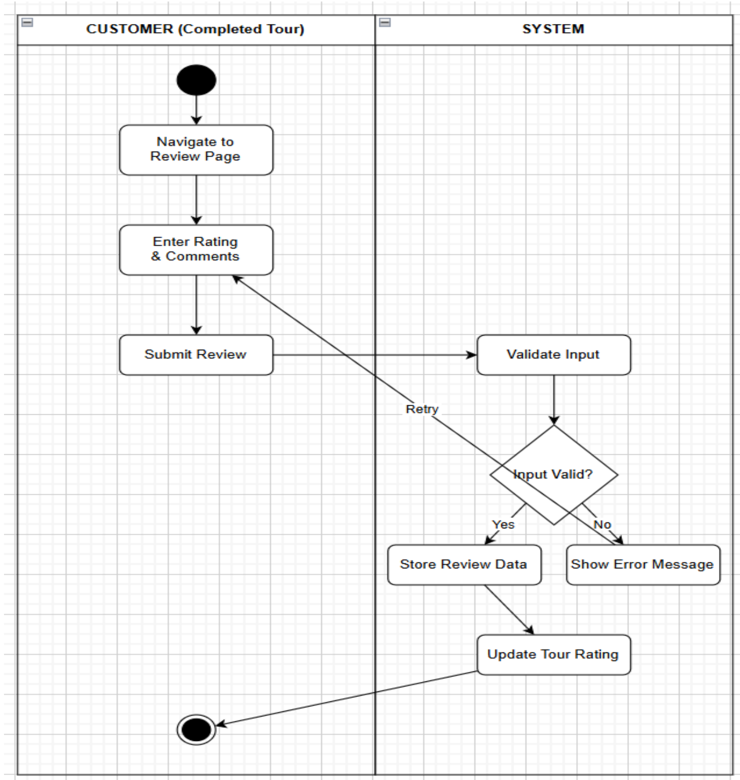
BR_TM_06 - Customer enters traveler information



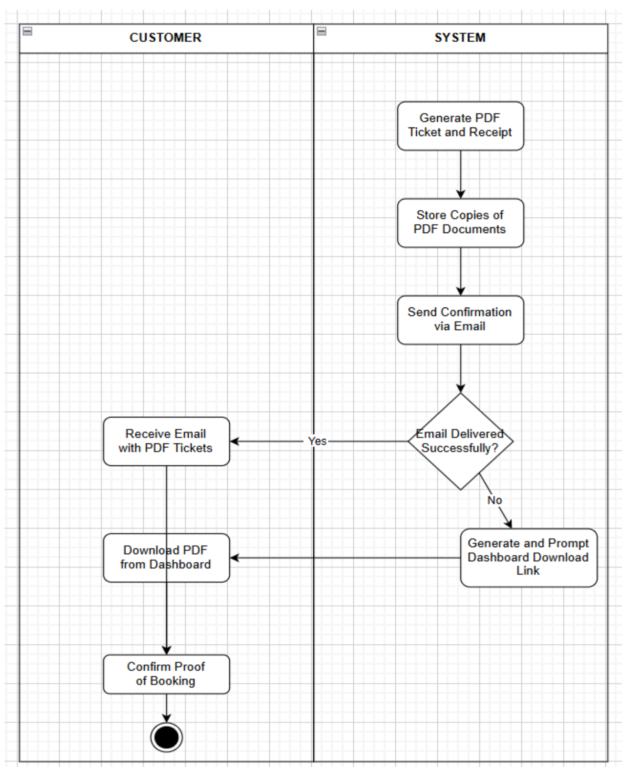
BR_TM_07 - Customer makes payment for Booking



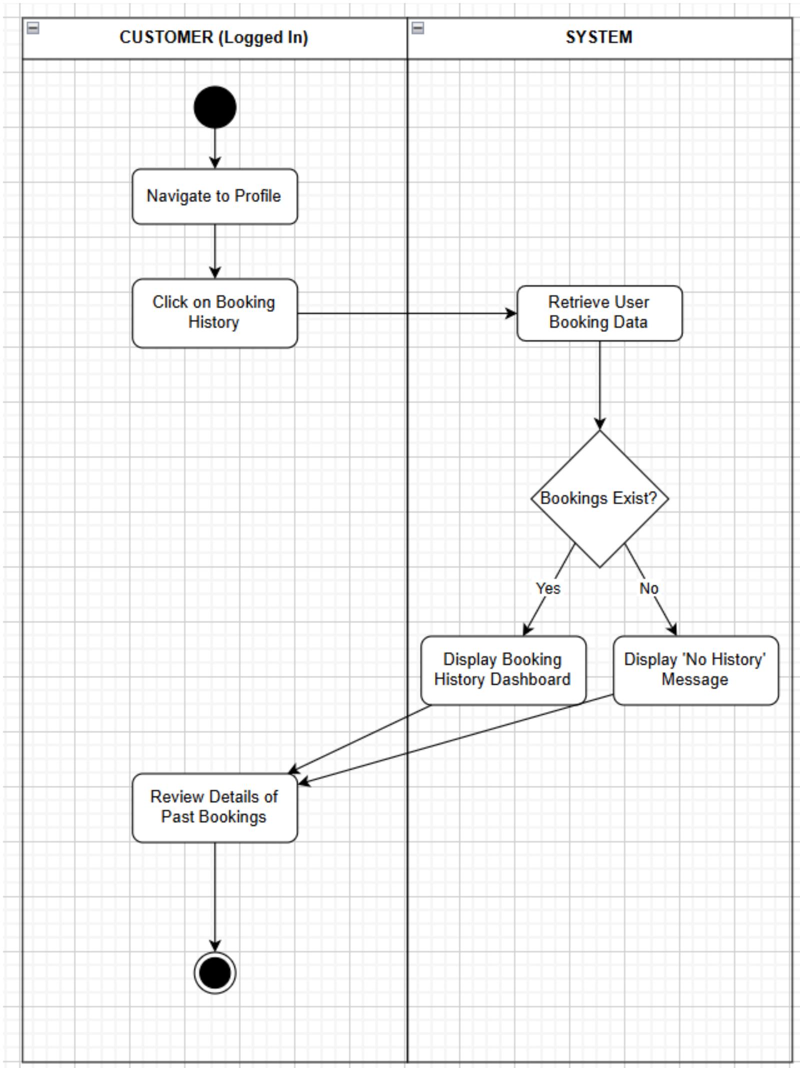
BR_TM_08 - Customer submits a review



BR_TM_09 - Customer receives booking confirmation/pays/downloads tickets

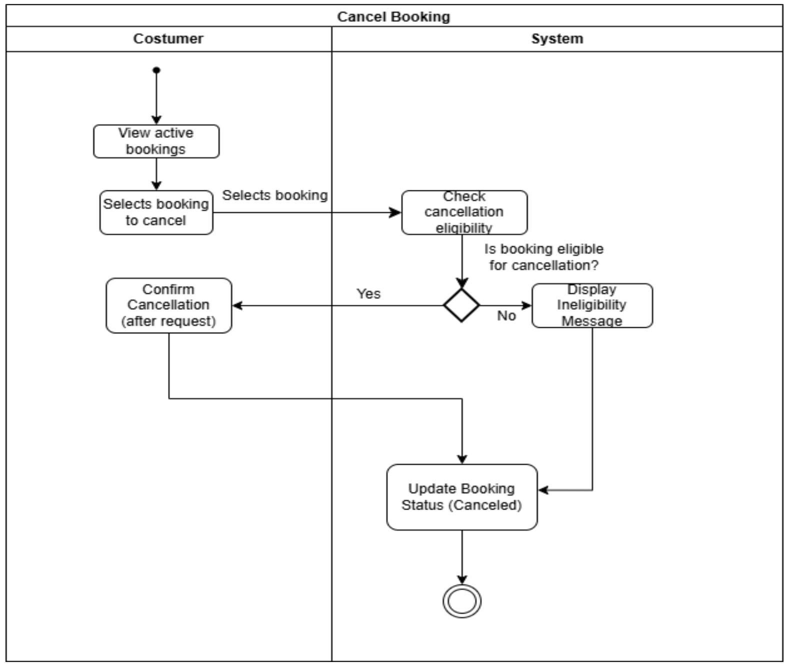


BR_TM_10 - Customer views booking history

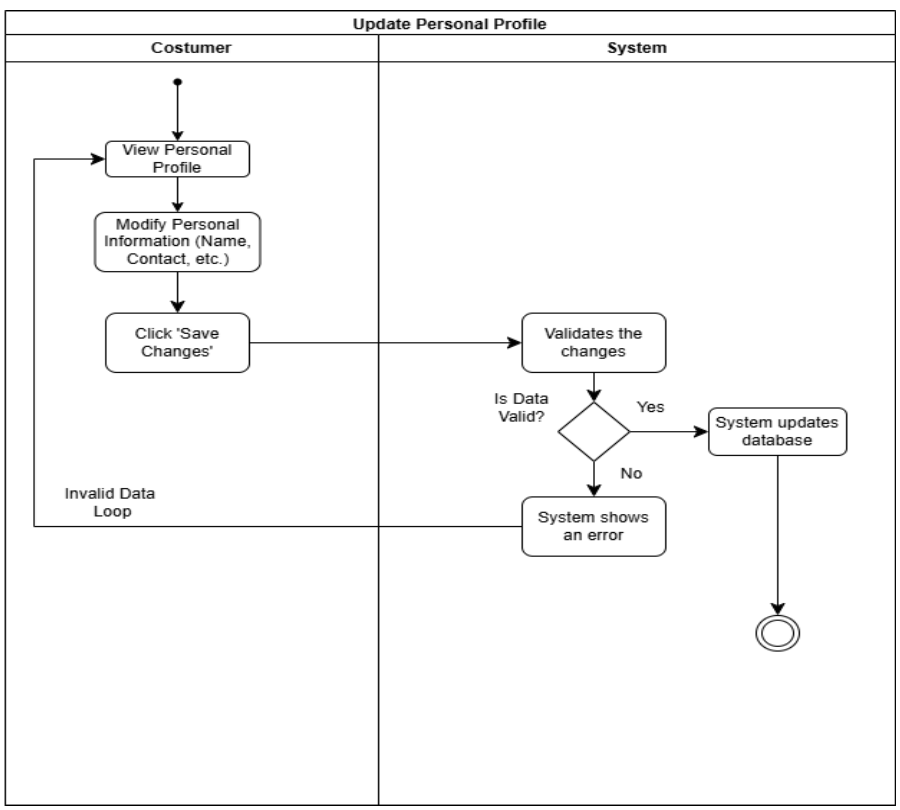


AMANDA STAFASANI

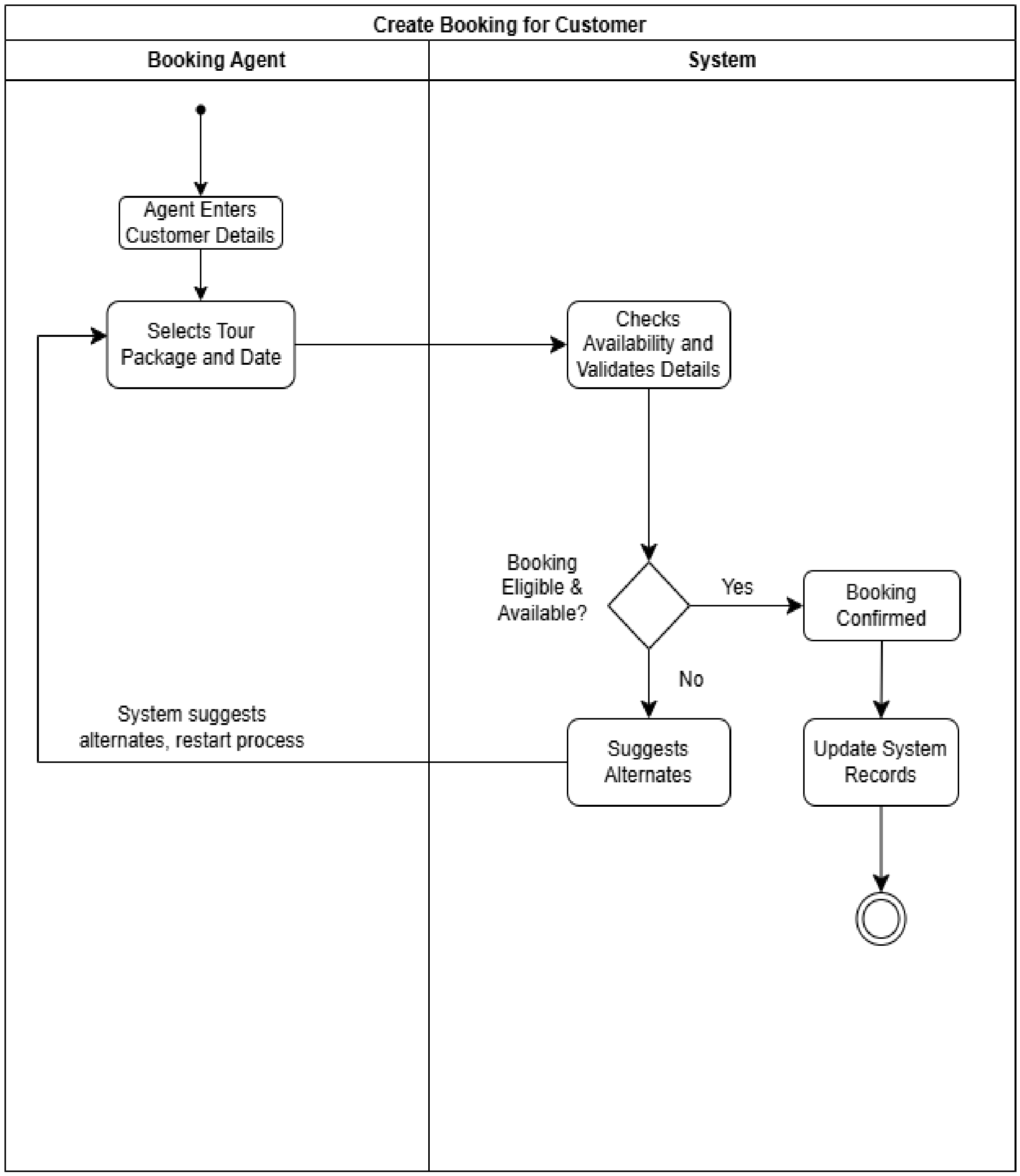
BR_TM_11 - Customer cancels a booking



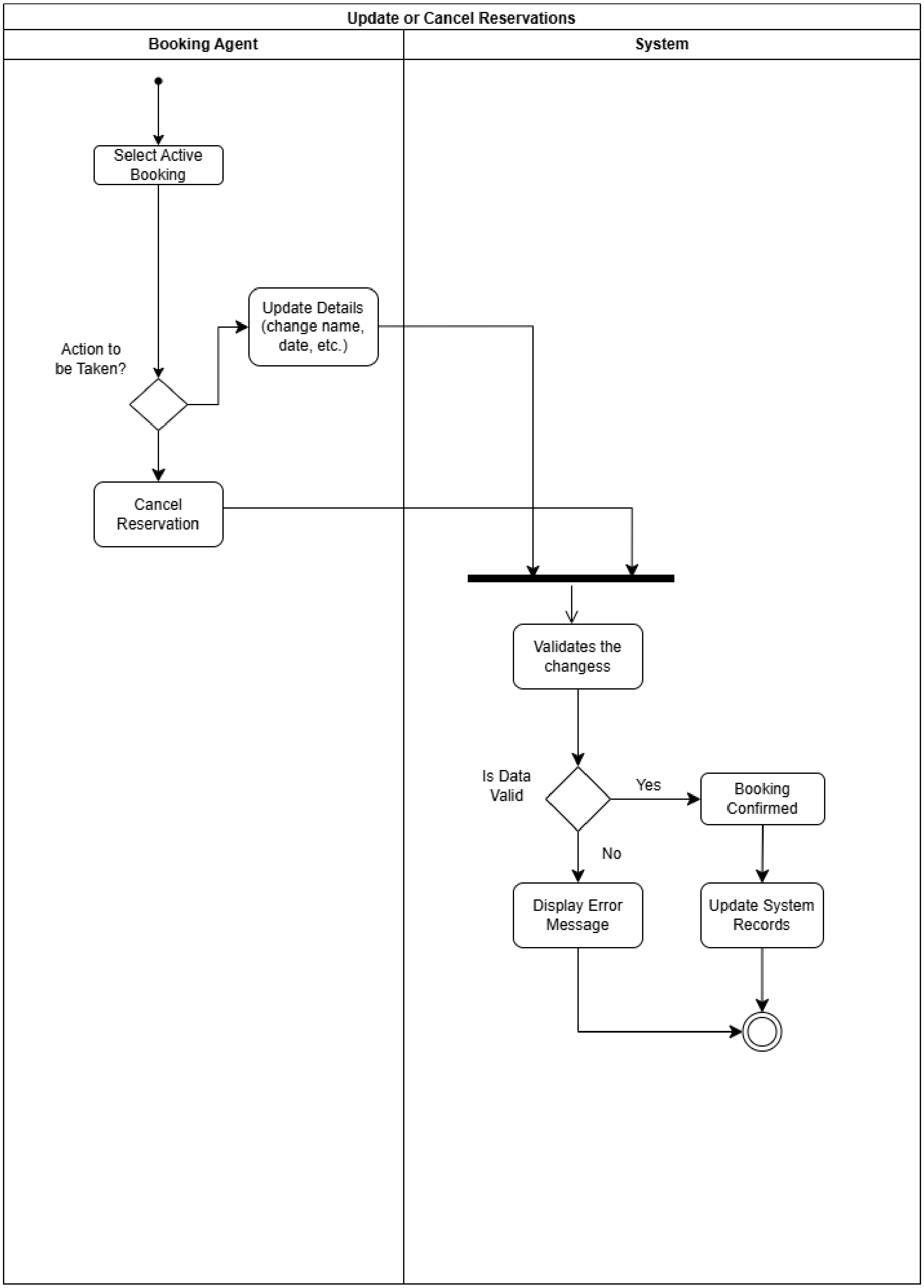
BR_TM_12 - Customer updates personal information in their profile



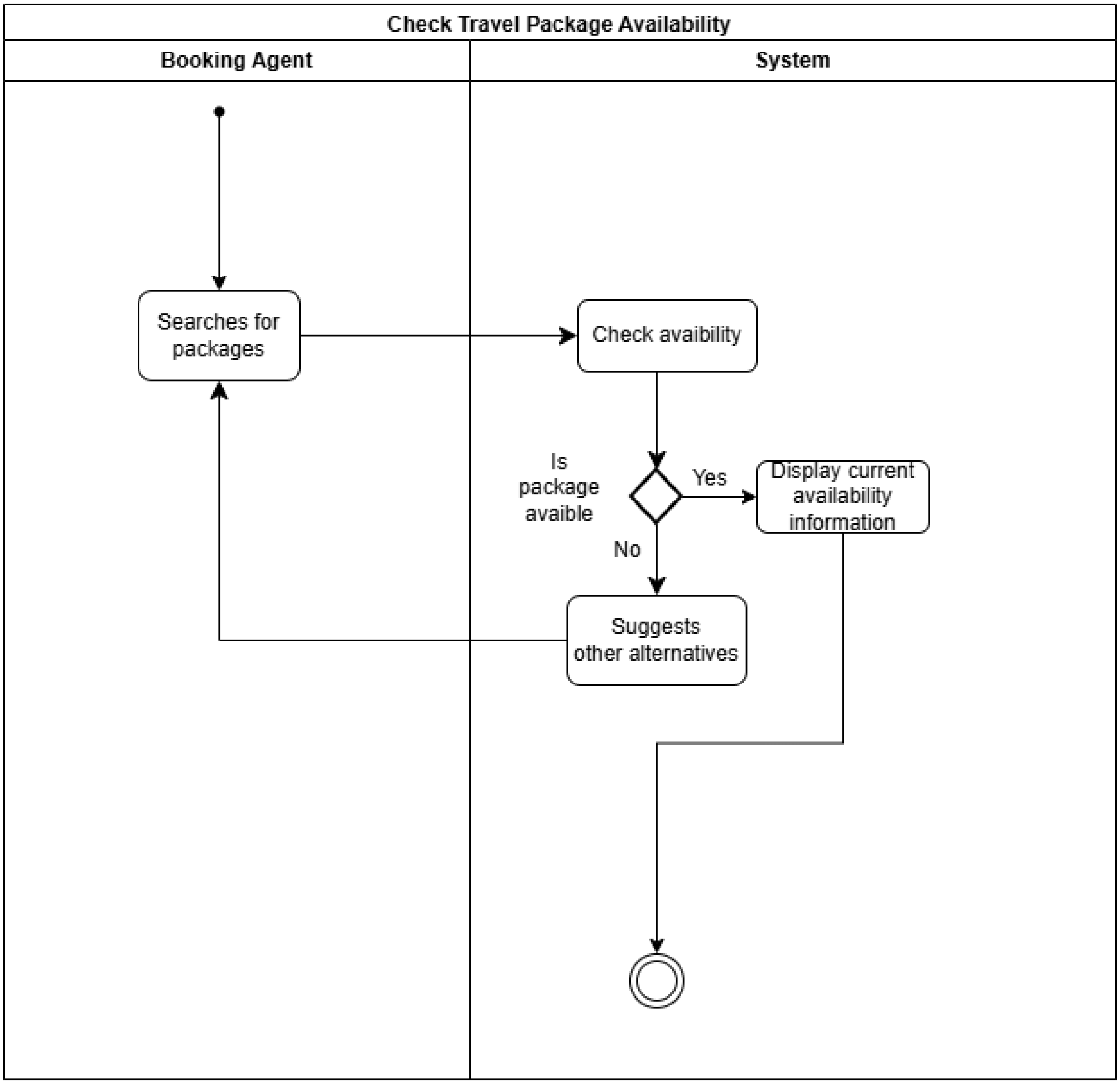
BR_TM_13 - Booking agent creates bookings for customers



BR_TM_14 - Booking agent updates or cancels existing reservations for a customer

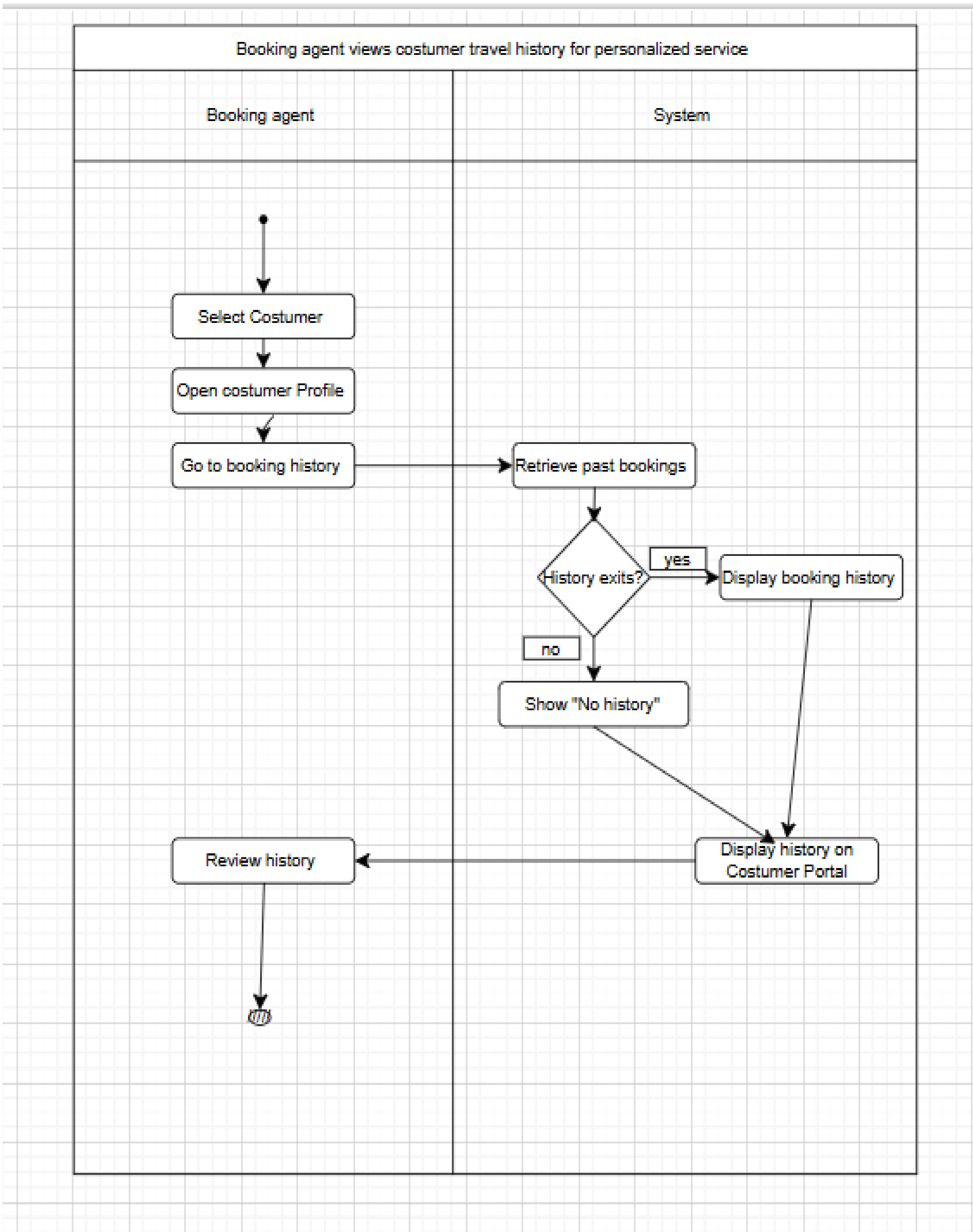


BR_TM_15 - Booking agent checks travel package availability

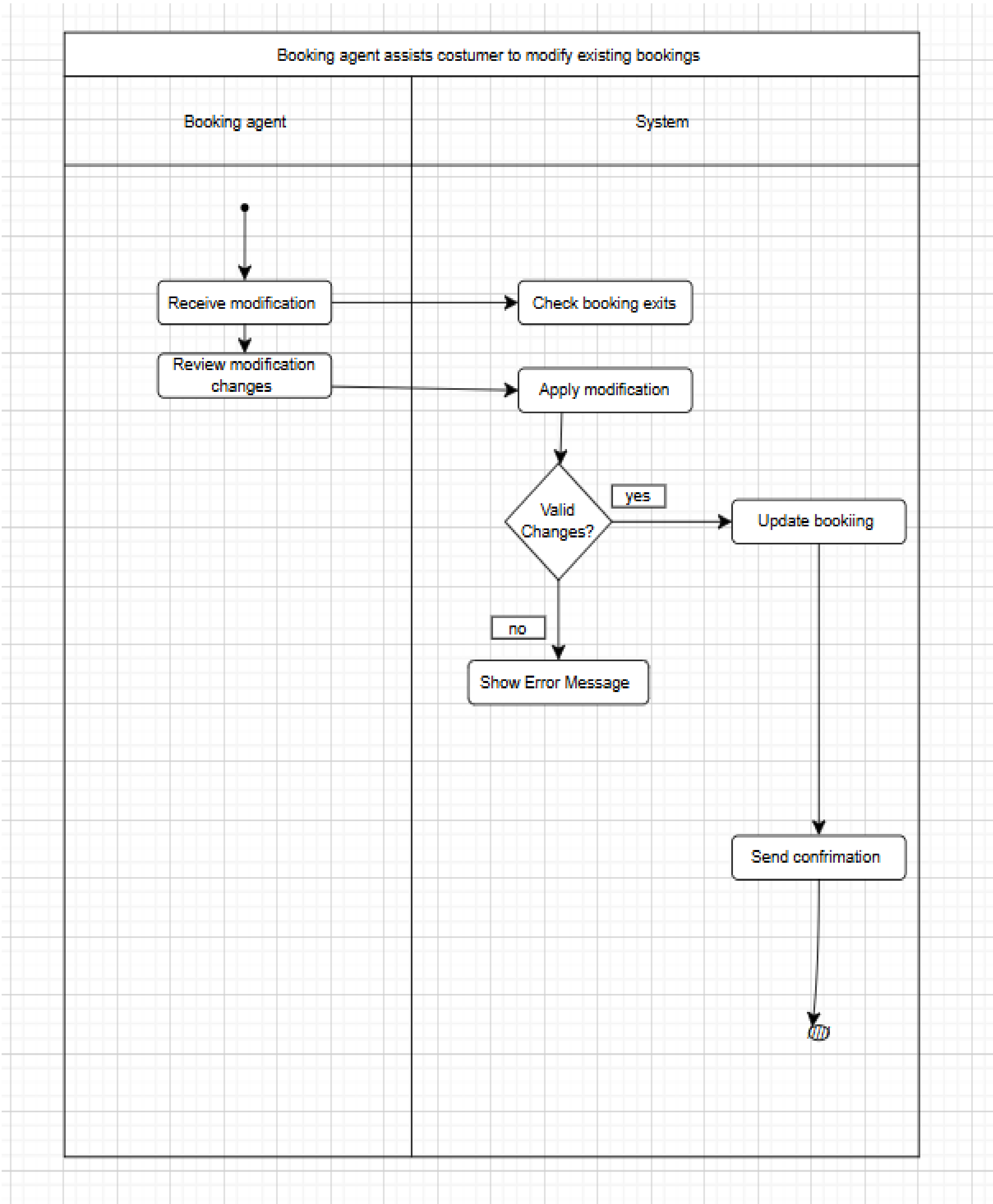


LUMTURI BEDALLI

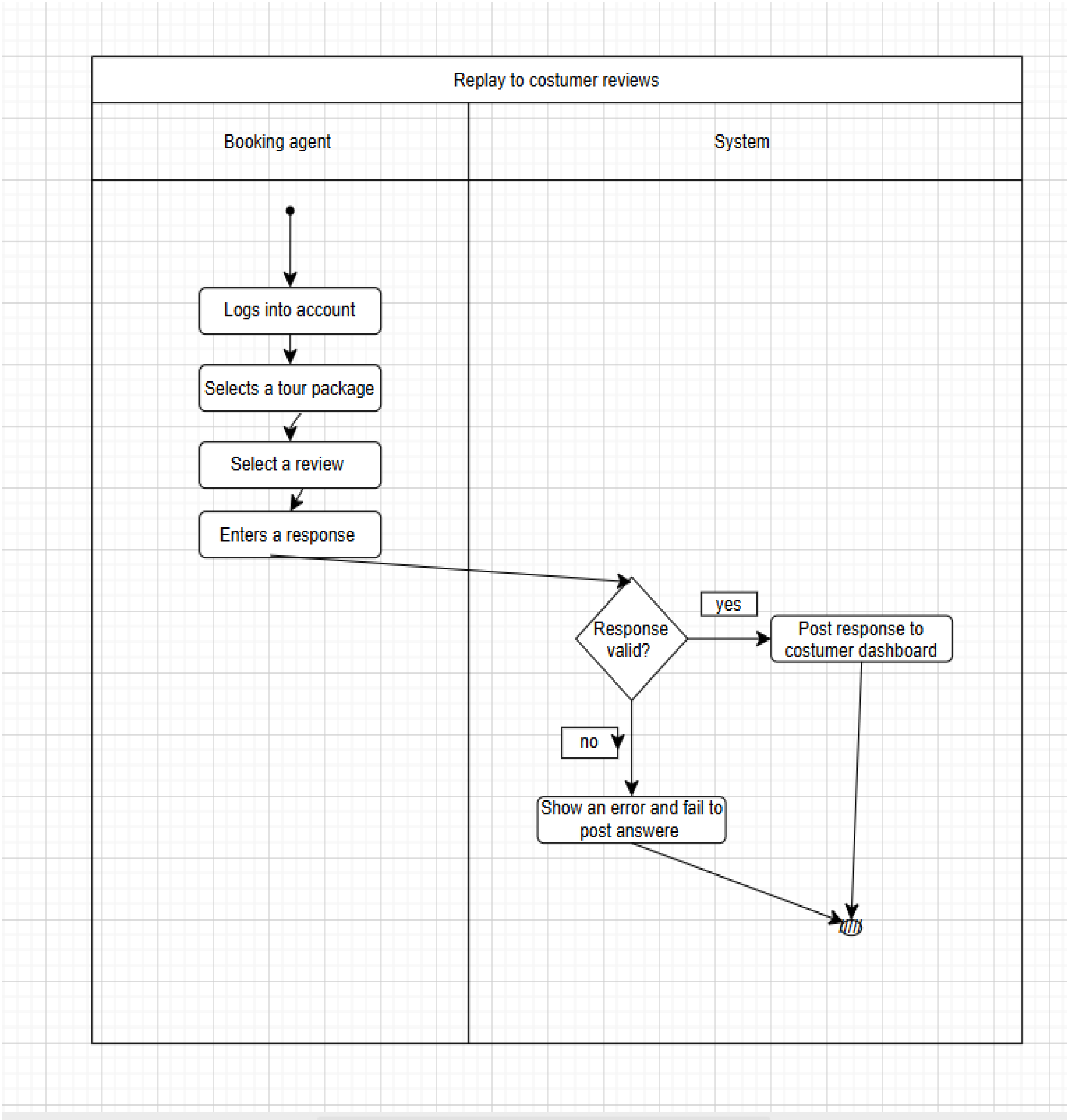
BR_TM_16 – Booking agent views customer travel history



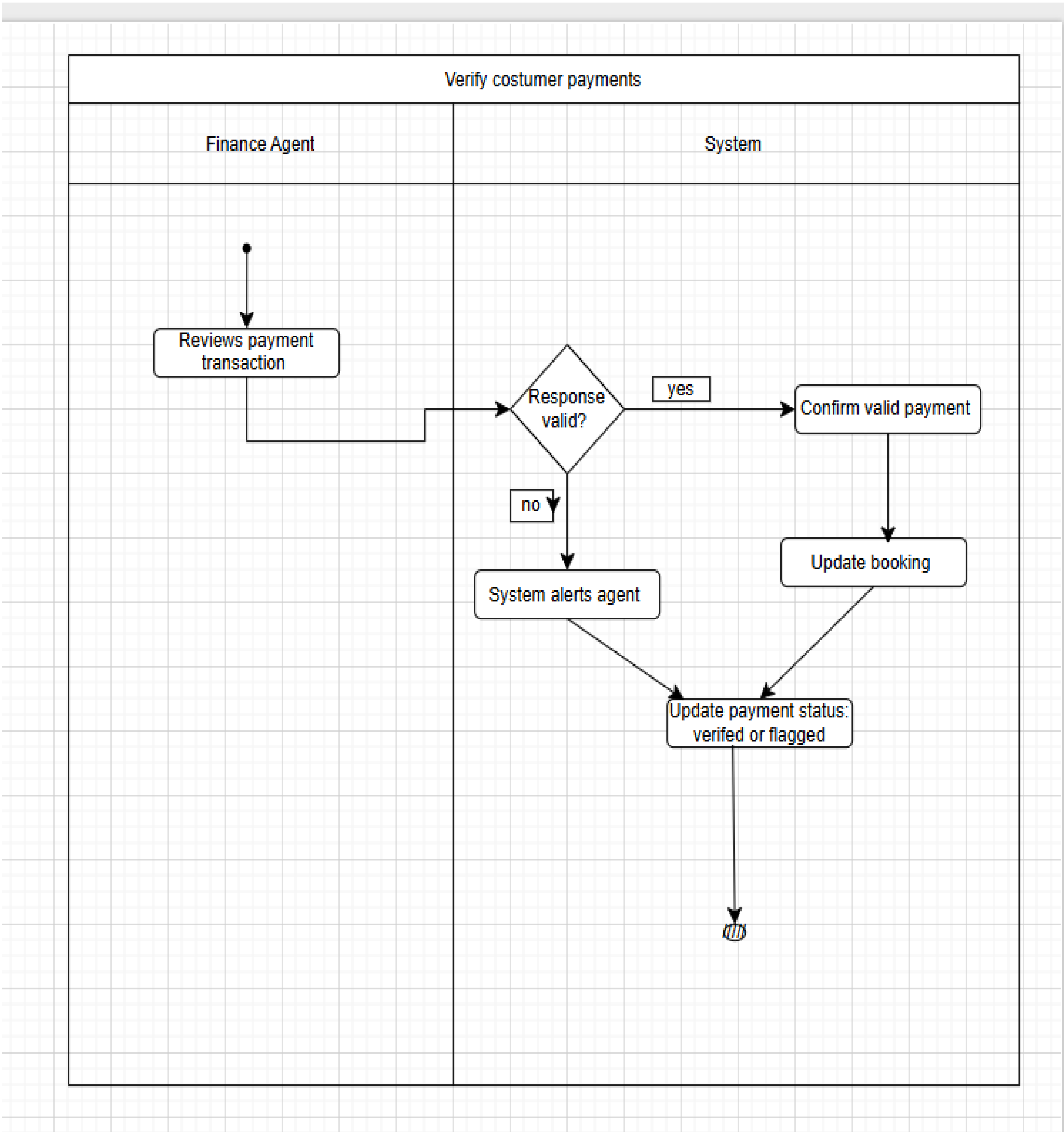
BR_TM_17 – Booking agent assists customer to modify existing bookings



BR_TM_18 – Booking agent replies to customer reviews

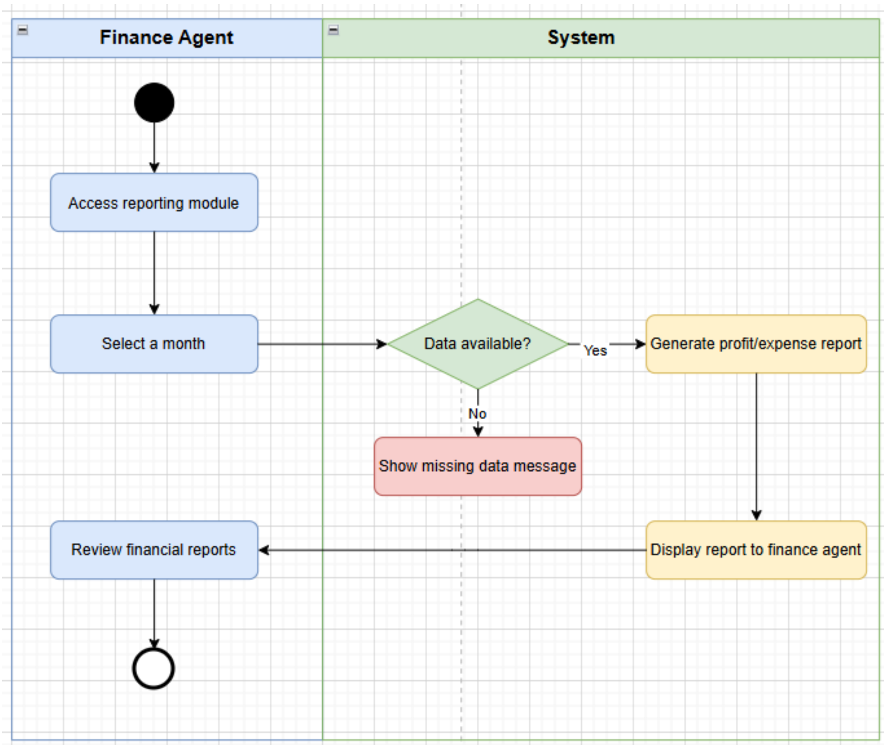


BR_TM_19 – Finance agent verifies customer payments

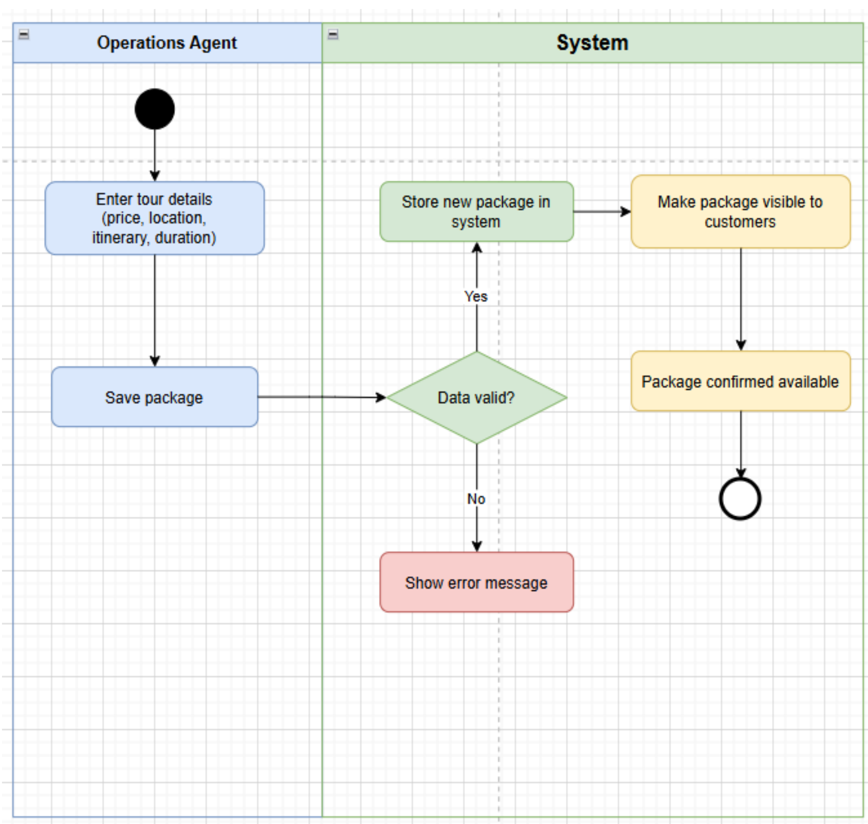


HENRI SHEHU

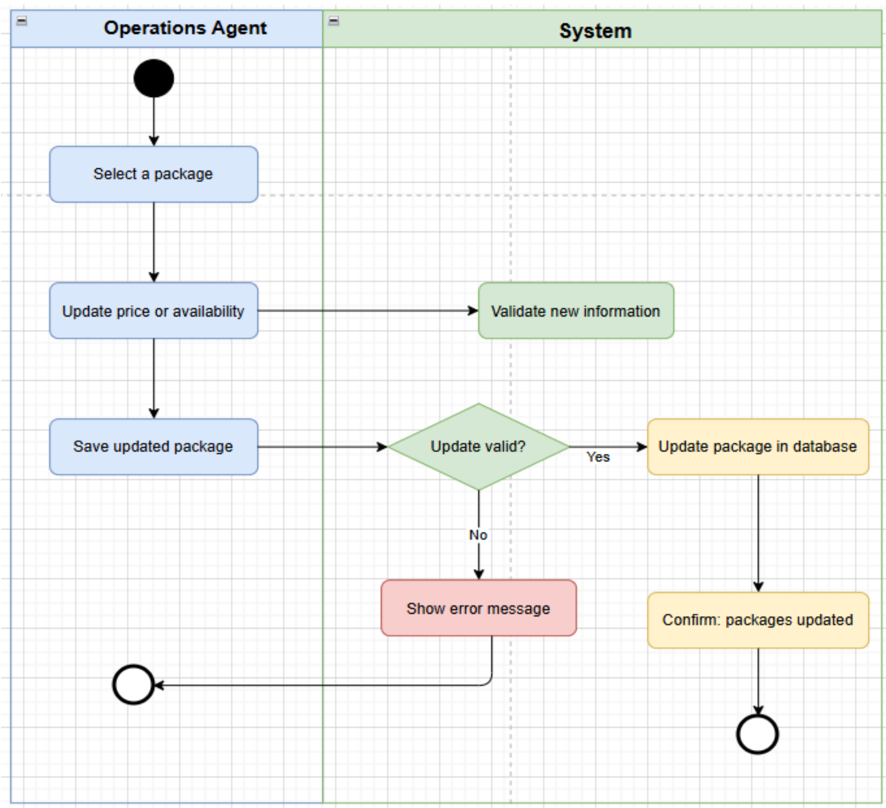
BR_TM_20- Finance agent views monthly profit and expenses report



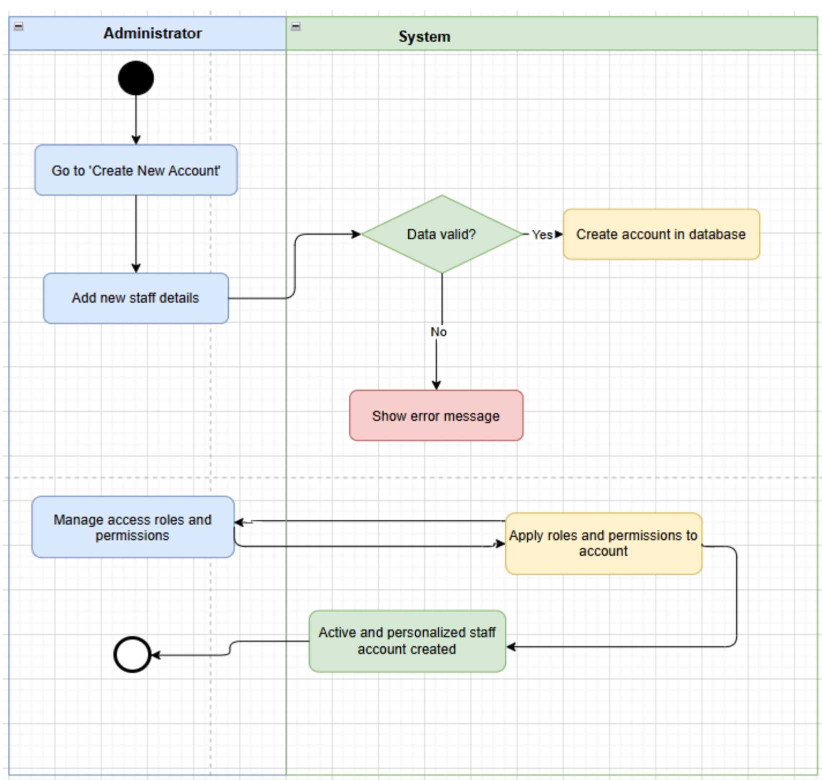
BR_TM_21 – Operations agent creates Tour Packages



BR_TM_22 – Operations agent updates package prices and availability

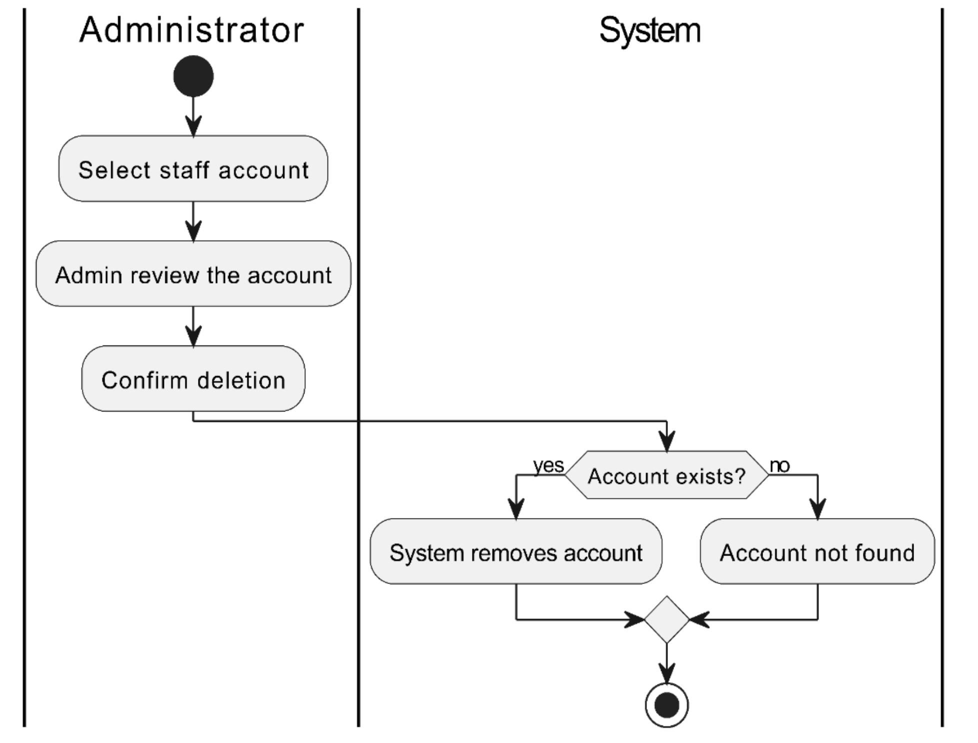


BR_TM_23 – Administrator creates and manages staff accounts

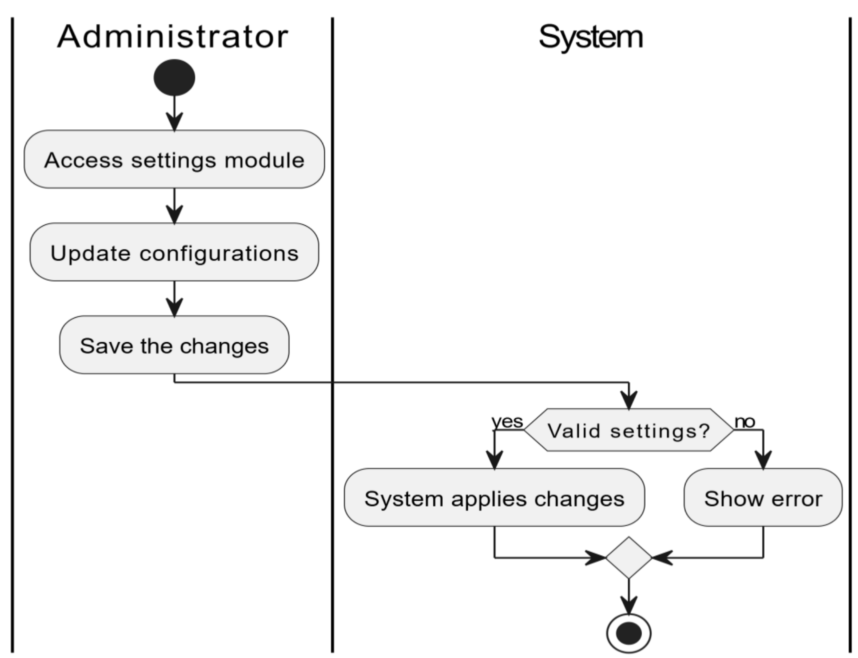


ENIAN XHIXHA

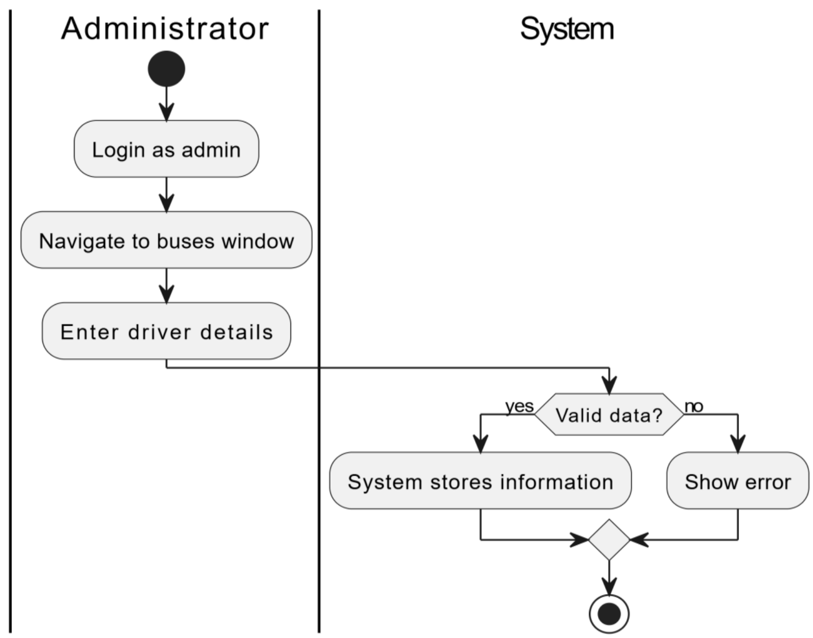
BR_TM_24 - Administrator deletes inactive staff accounts



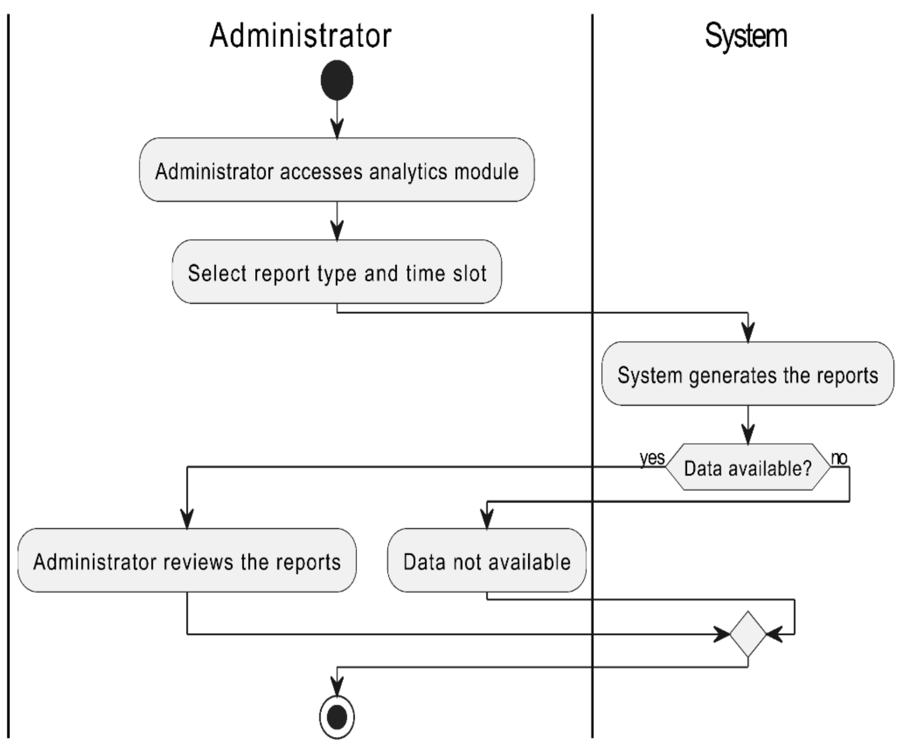
BR_TM_25 – Administrator configurates the system settings



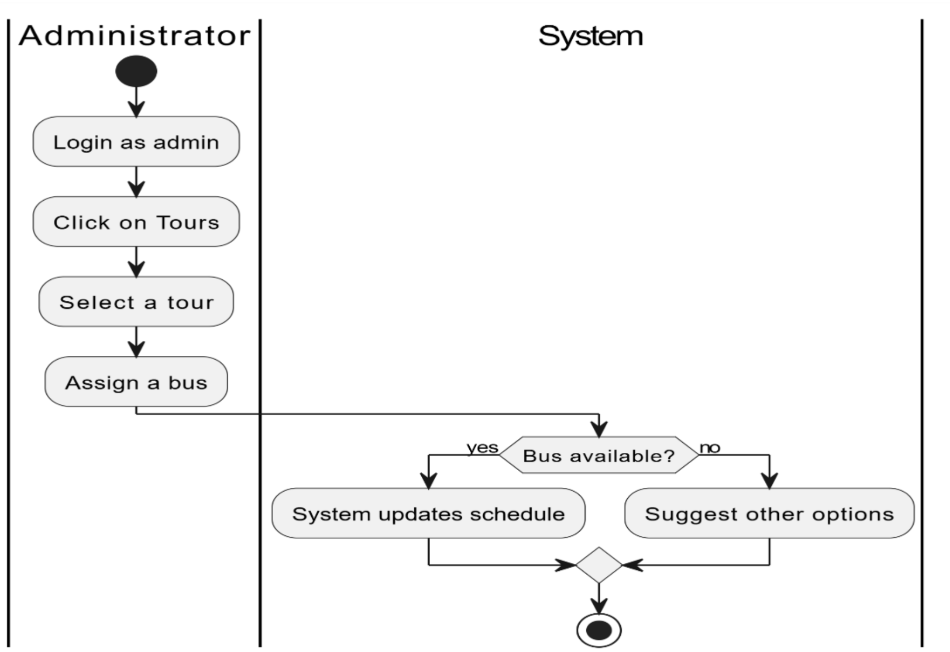
BR_TM_26 - Administrator views sales reports



BR_TM_27 – Administrator Recruits Bus Drivers

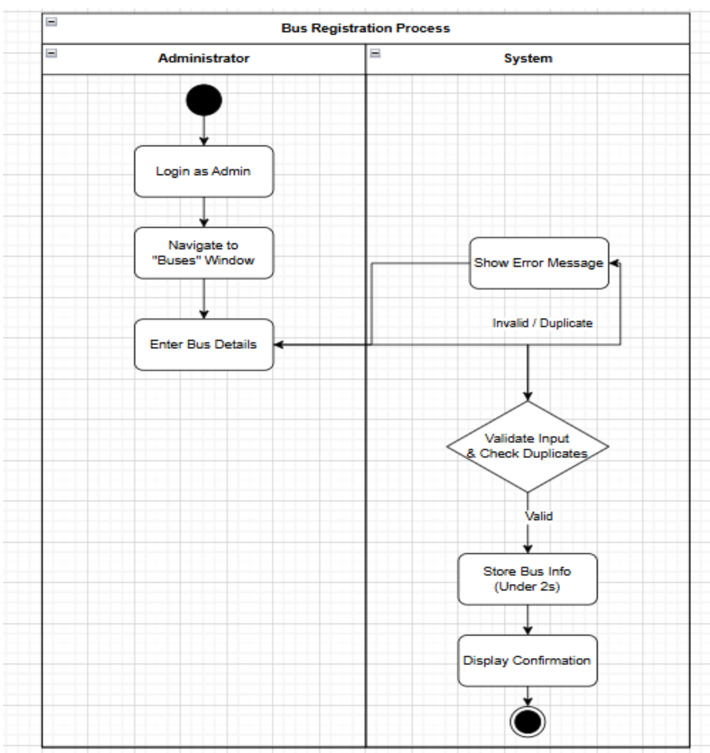


BR_TM_28 – Administrator Assigns Buses for Service

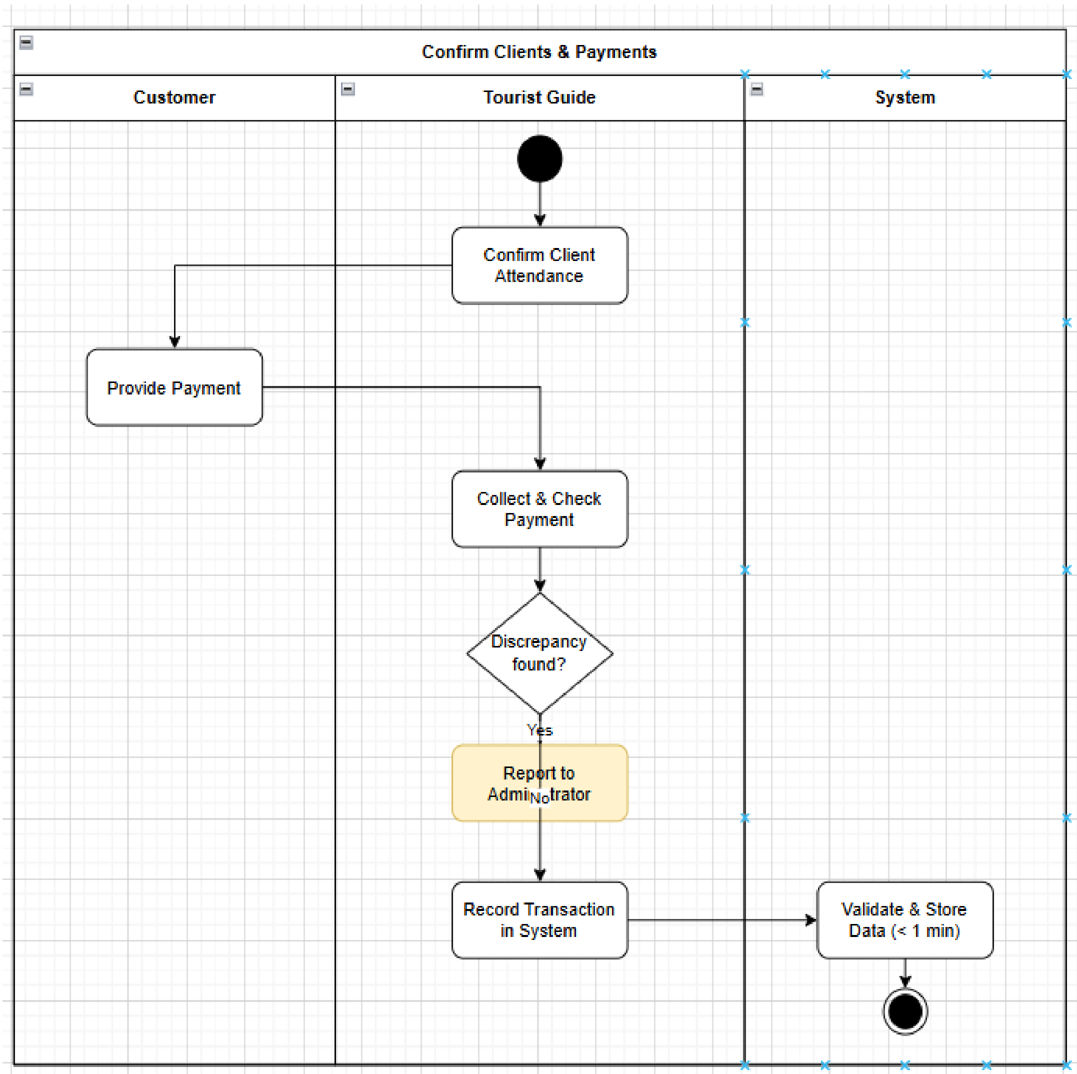


ERA SELMANAJ

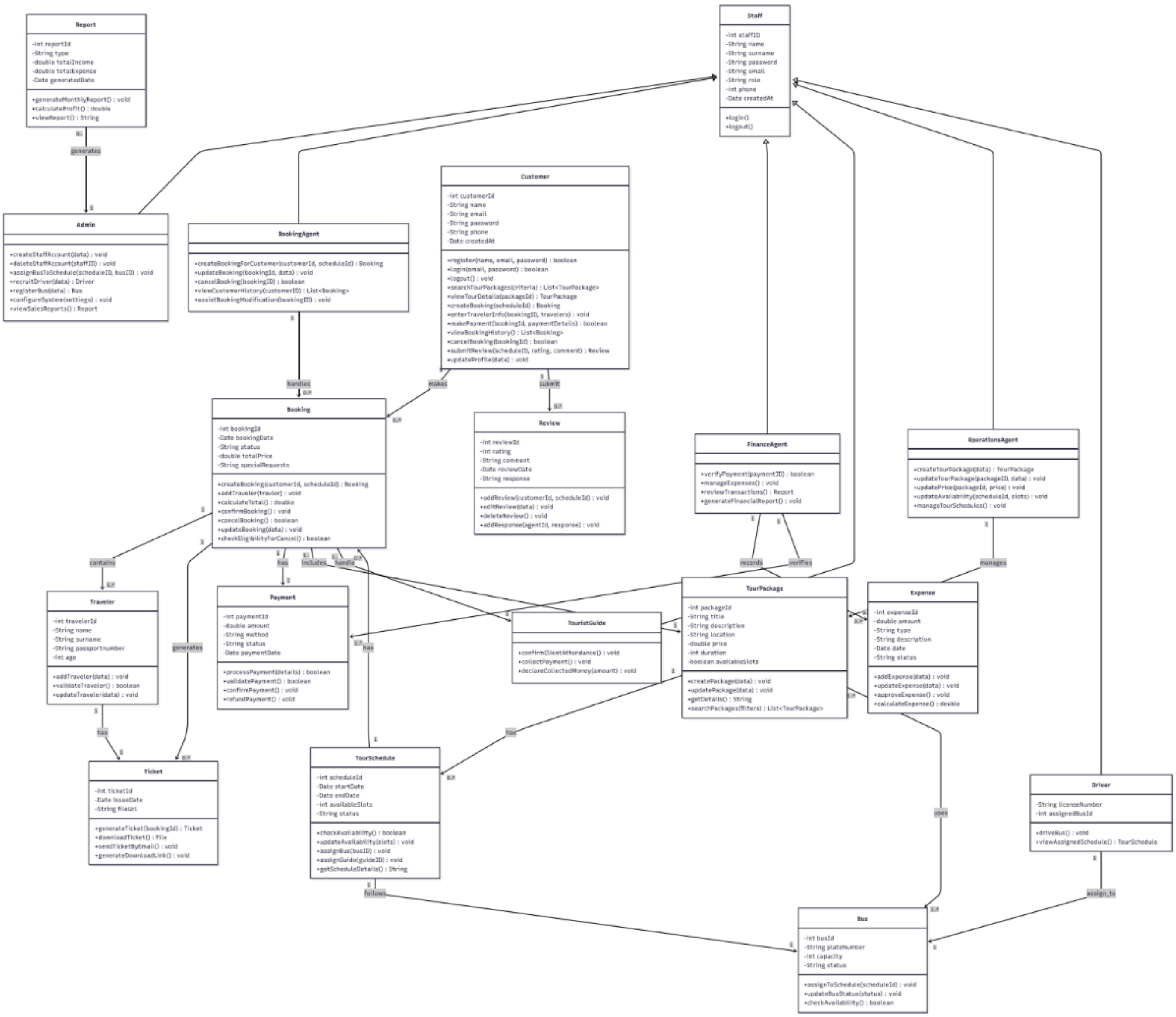
BR_TM_29 – Administrator registers number of buses



BR_TM_30 – Tourist Guide Confirms Clients and Collect Payments



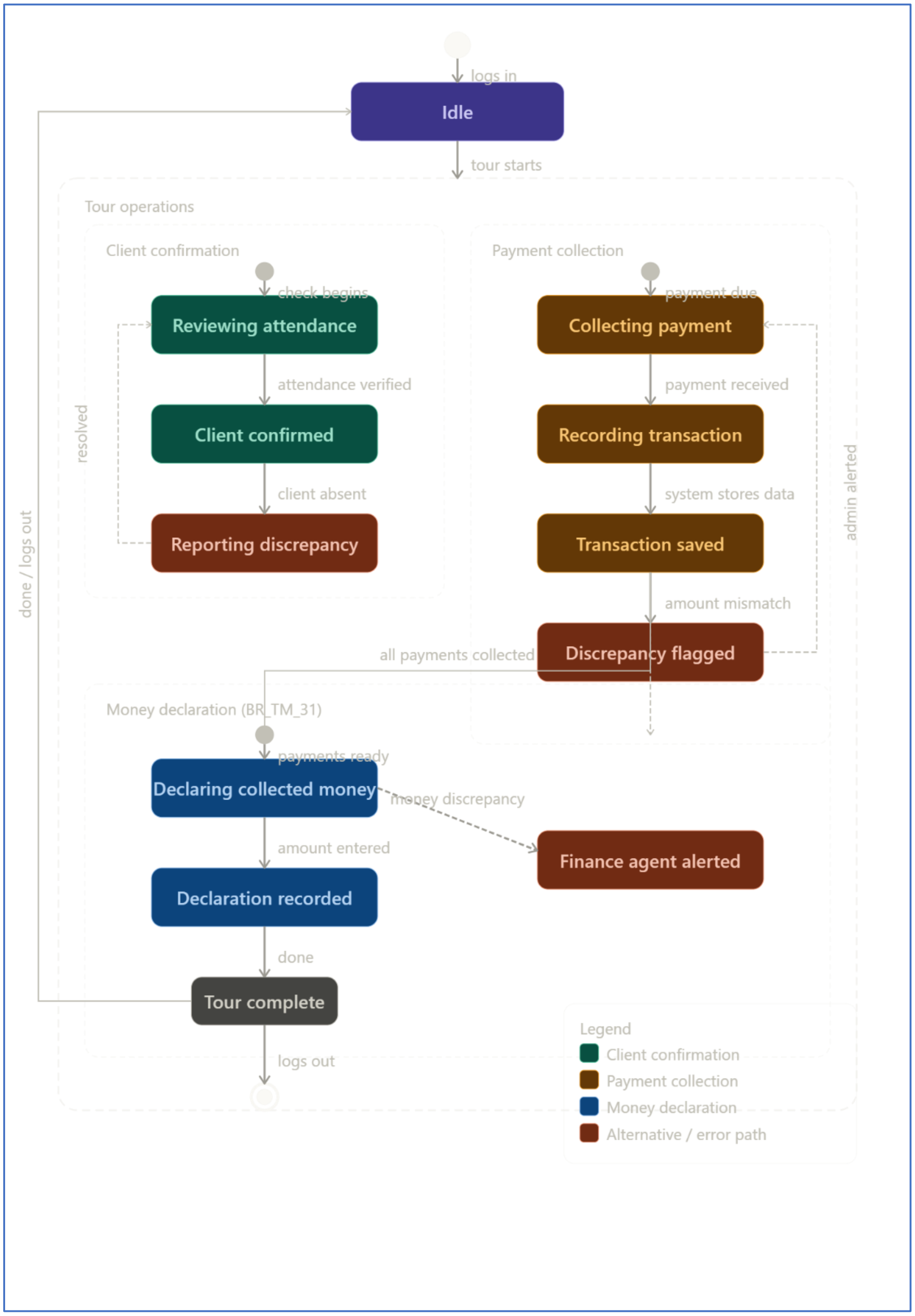
5.4 Class Diagram (by Ermi Xhaferi & Amar Kruja)



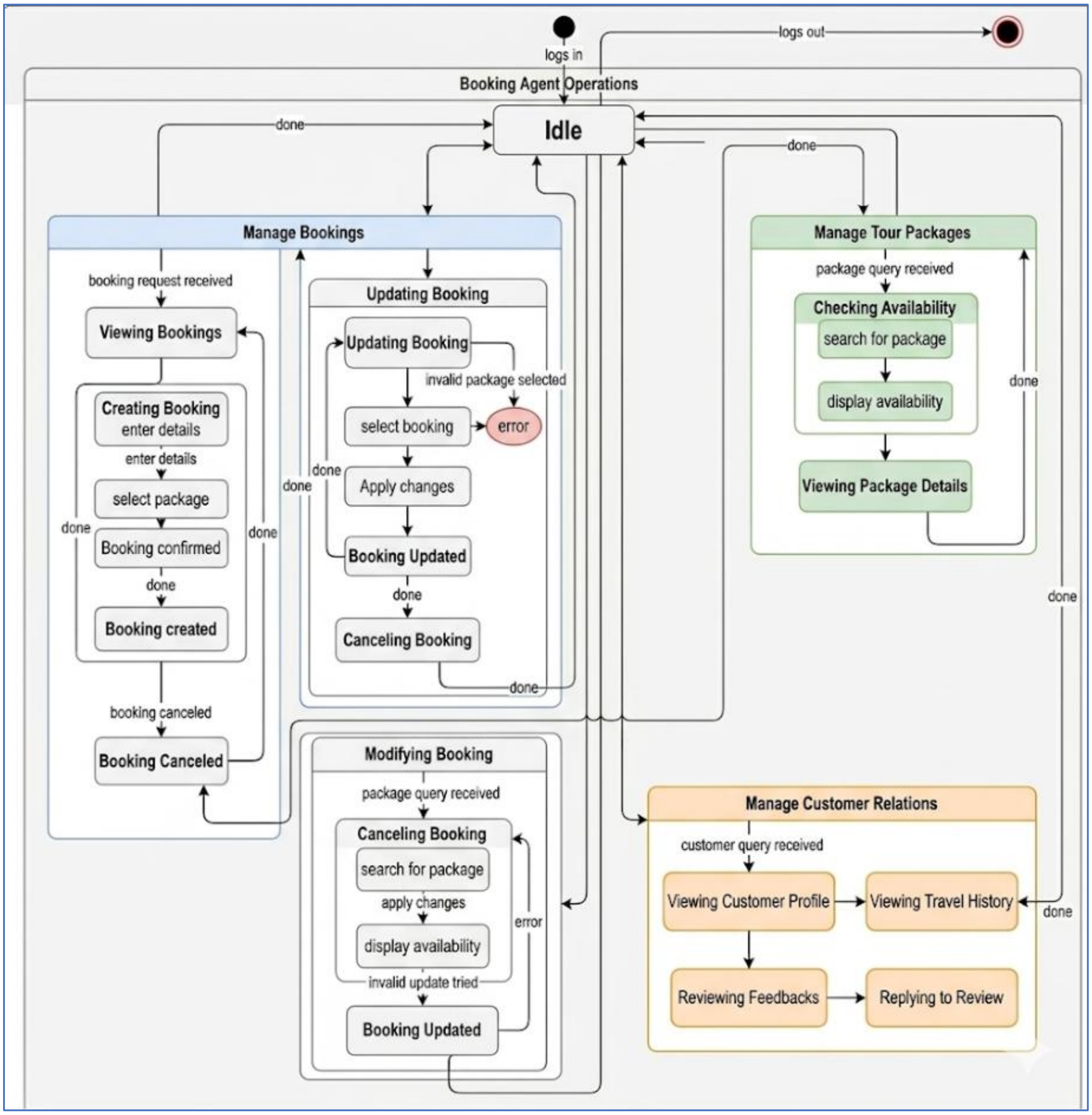
Link for image: [Tourist-Agency-Information-System/ClassDiagram.pdf at main · Ermi-Xh/Tourist-Agency-Information-System]

5.5 State Diagrams

STATE DIAGRAM FOR USER ROLE: TOURIST GUIDE → AMANDA STAFASANI

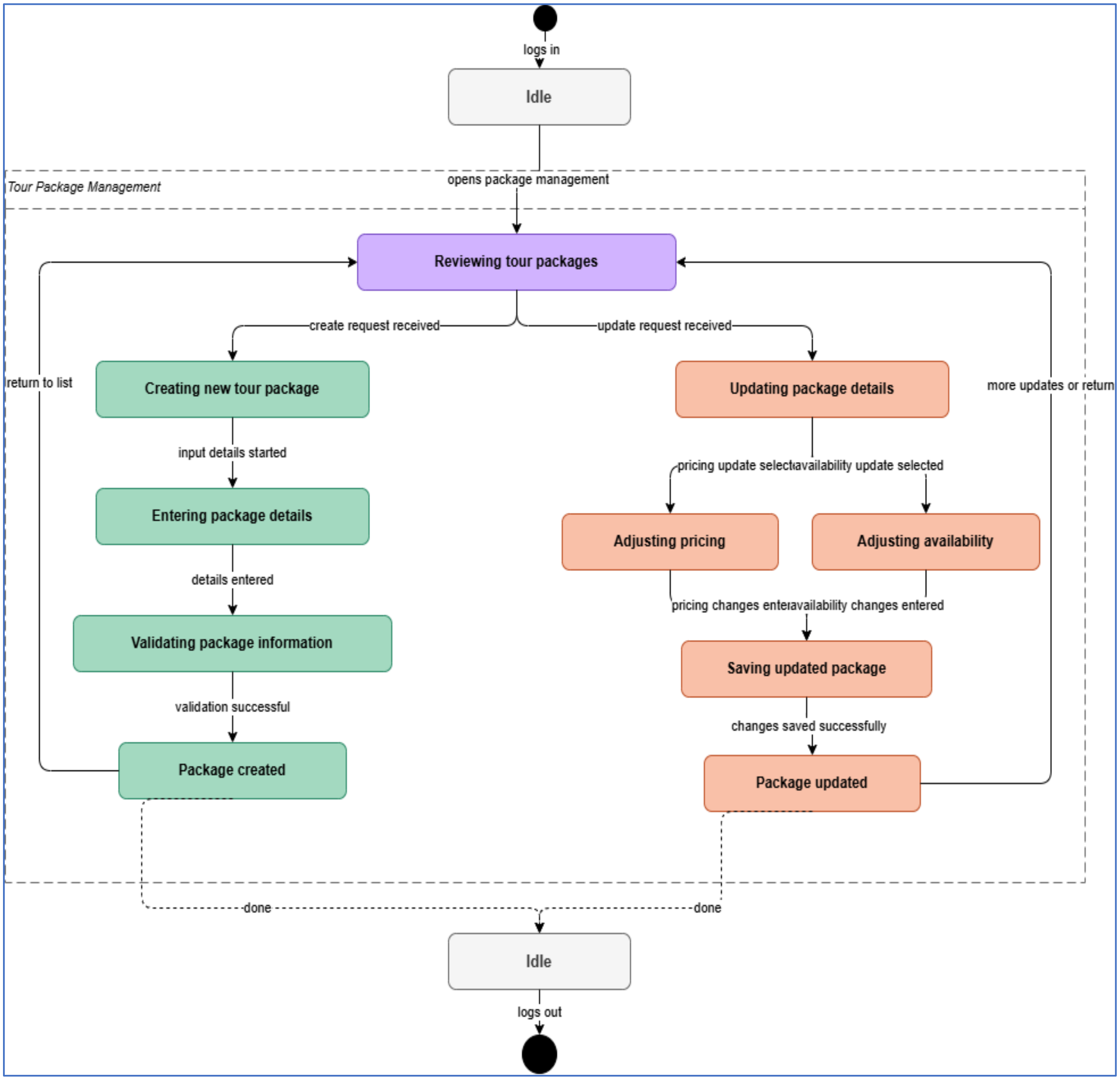


STATE DIAGRAM FOR USER ROLE: BOOKING AGENT → ERA SELMANAJ

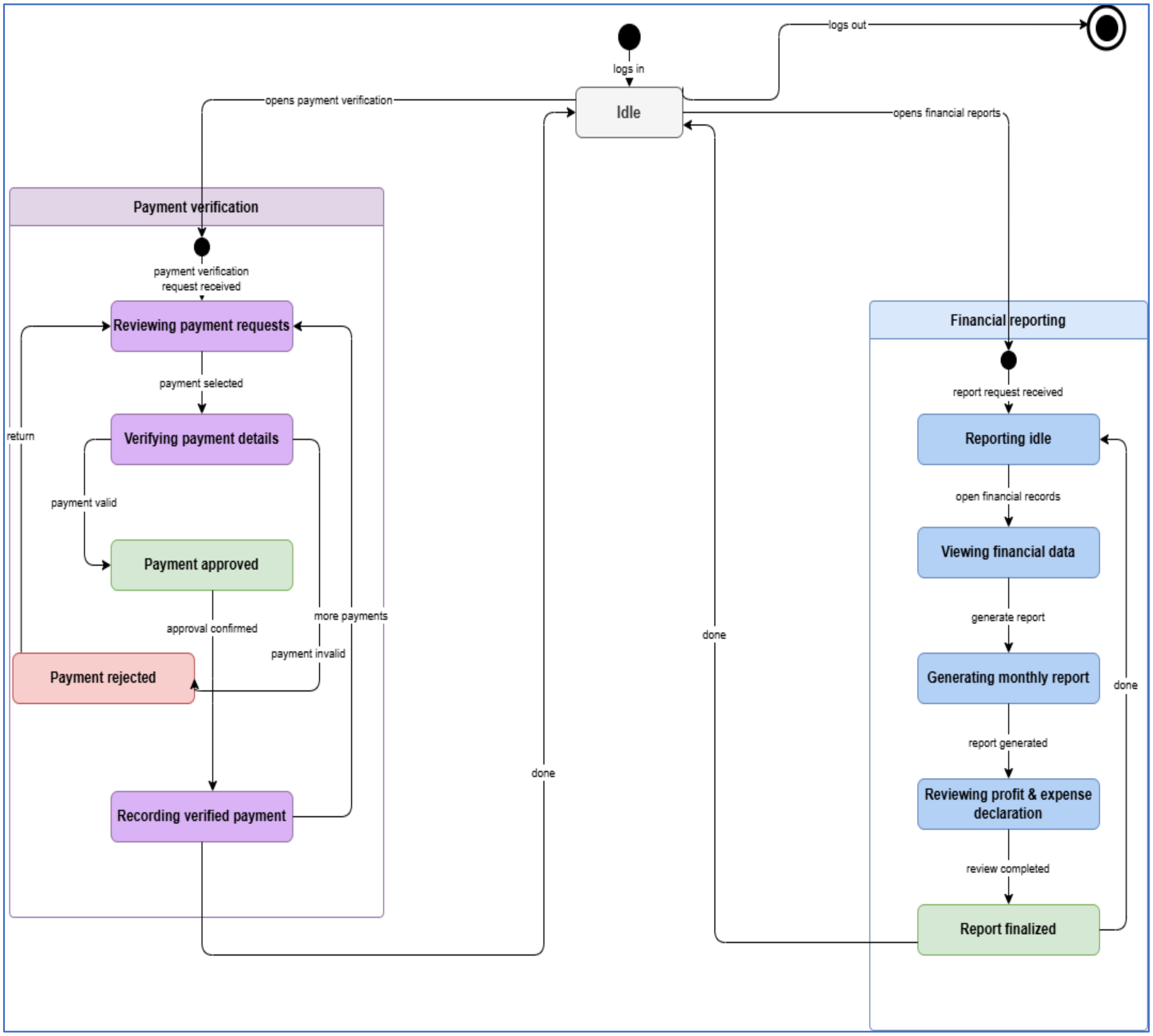


STATE DIAGRAM FOR USER ROLE: CUSTOMER → ALISA DHIMA

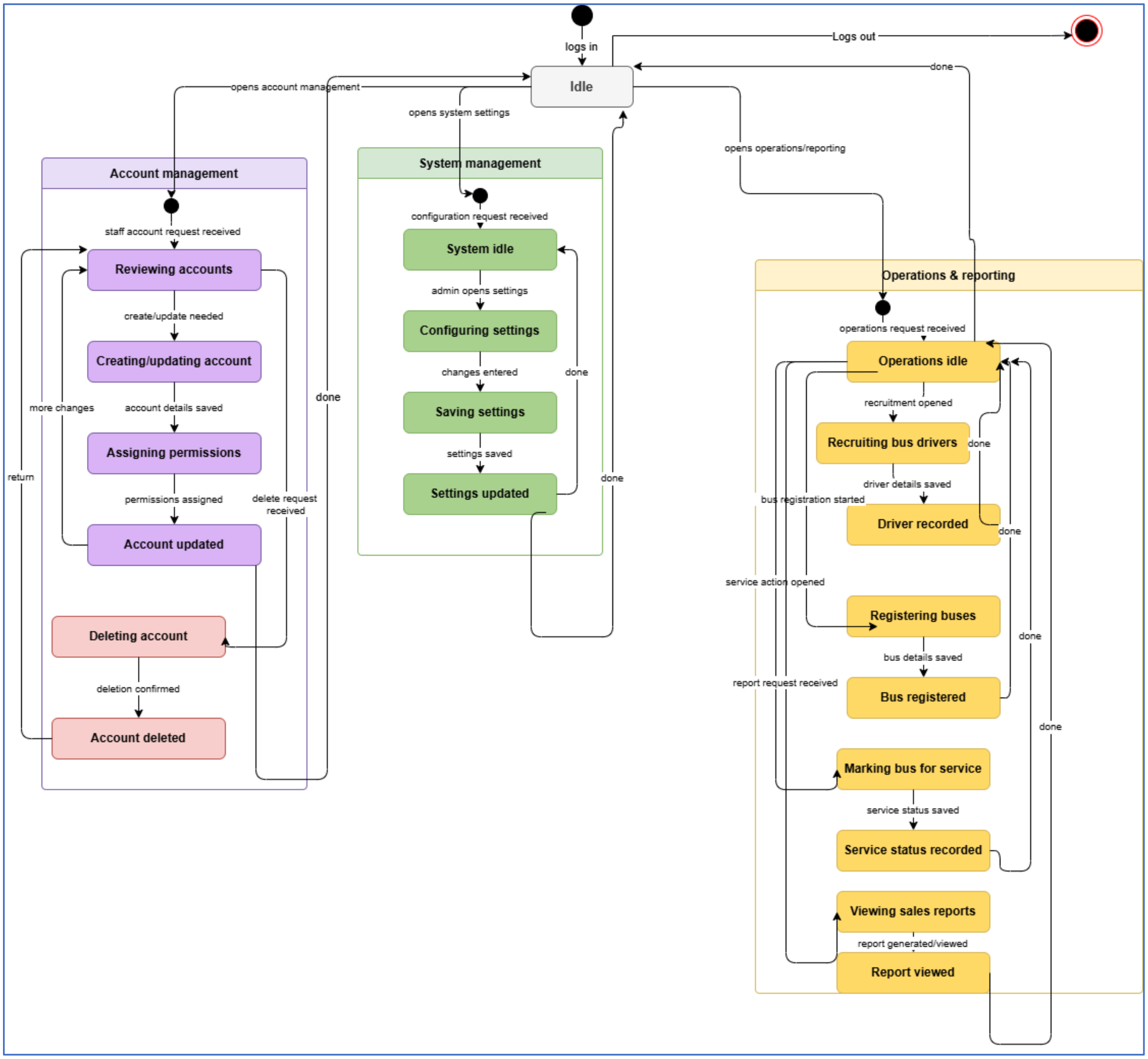
STATE DIAGRAM FOR USER ROLE: OPERATIONS AGENT → ALISA DHIMA



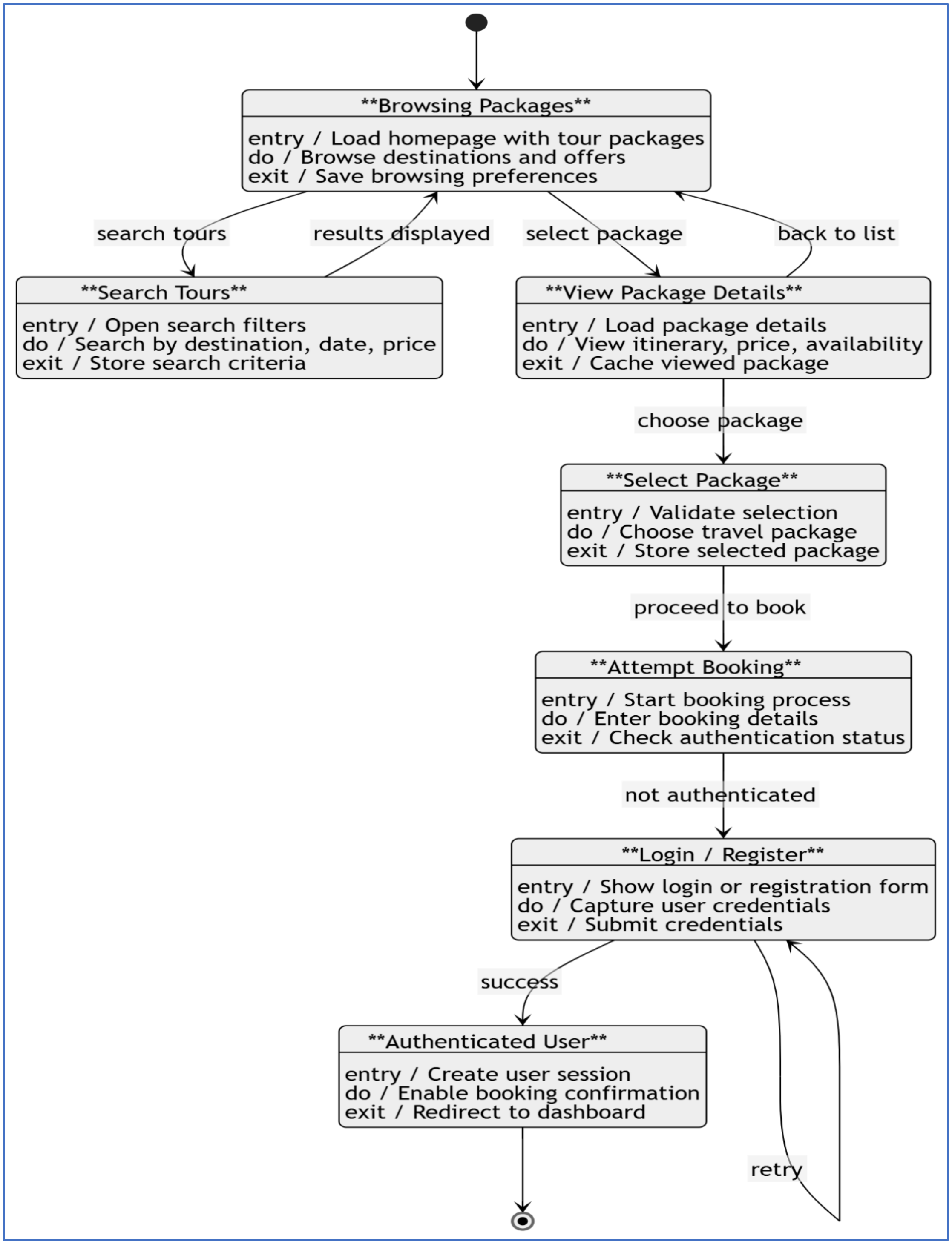
STATE DIAGRAM FOR USER ROLE: FINANCE AGENT → HENRI SHEHU



STATE DIAGRAM FOR USER ROLE: ADMINISTRATOR → HENRI SHEHU



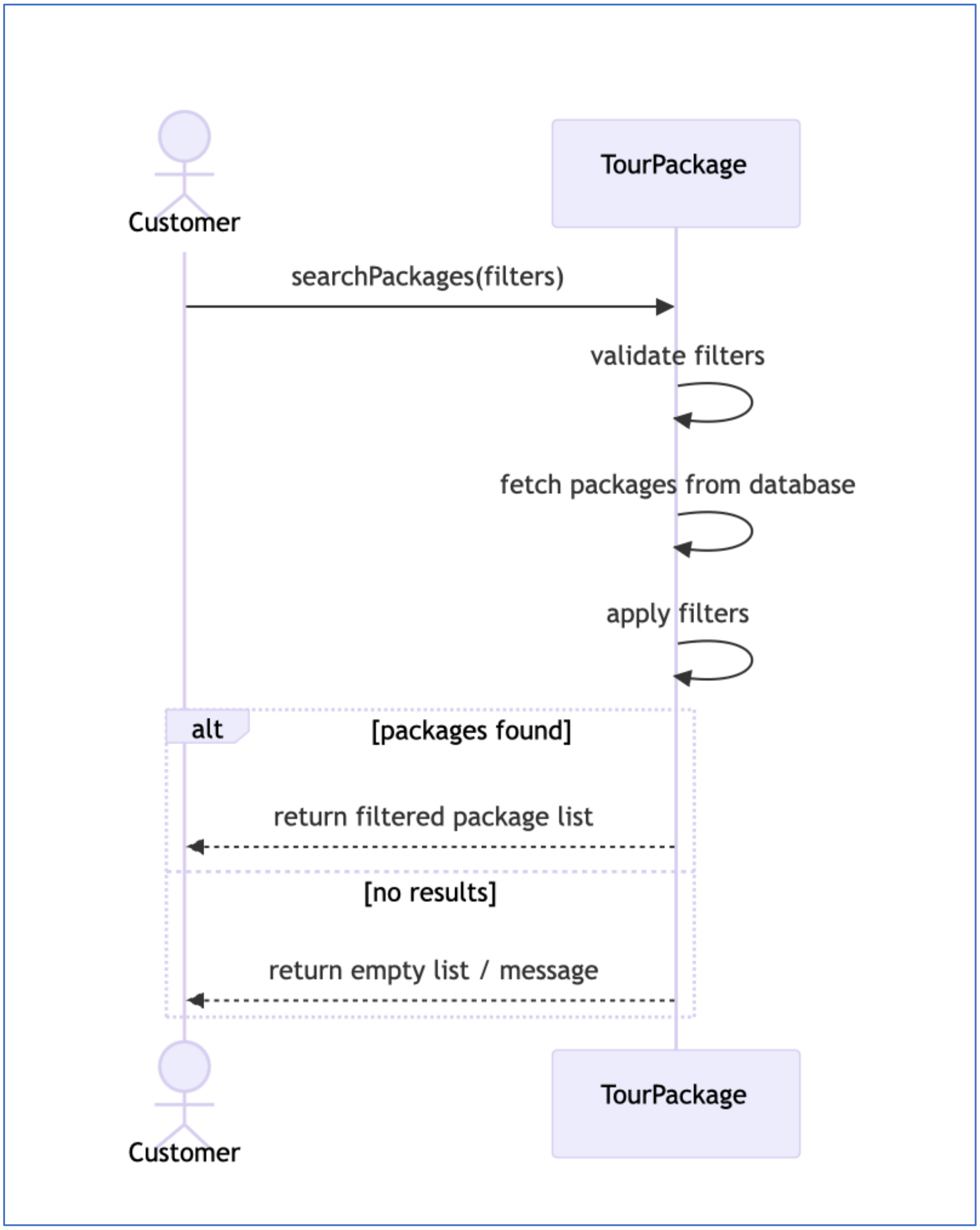
STATE DIAGRAM FOR USER ROLE: GUEST USER →LUMTURI BEDALLI



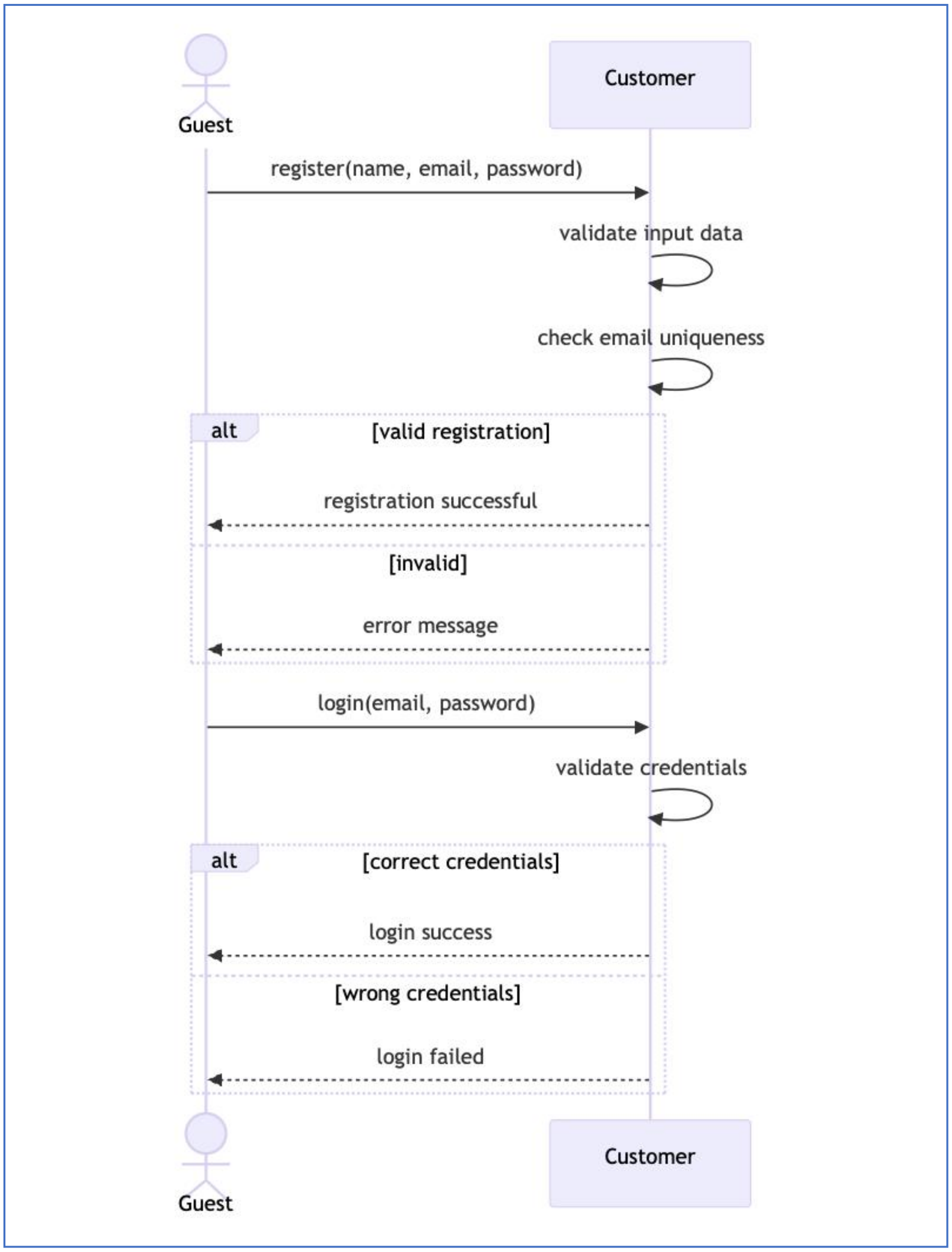
5.6 Sequence Diagrams

Amar & Ermi:

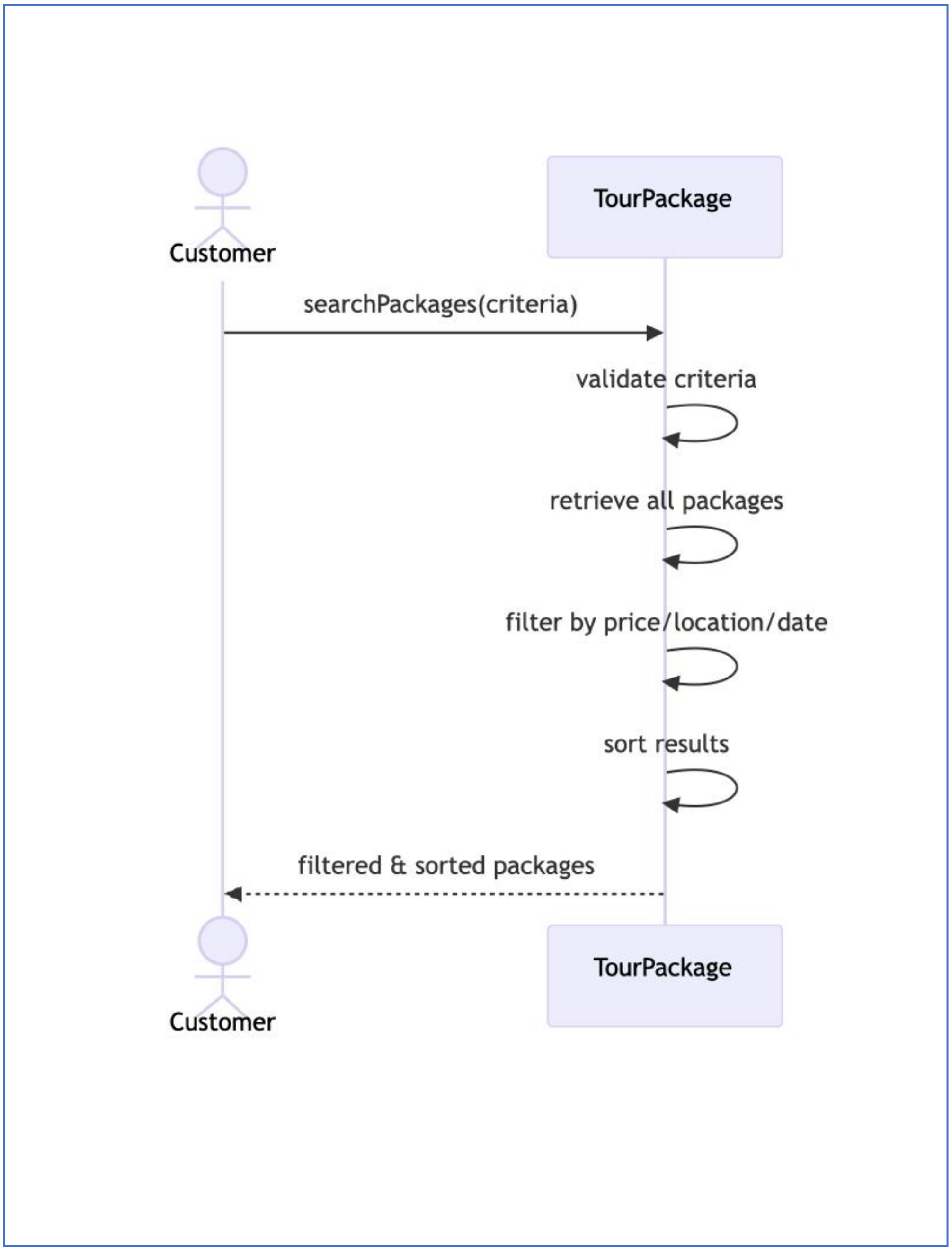
BR_TM_01 - View and Search Tour Packages



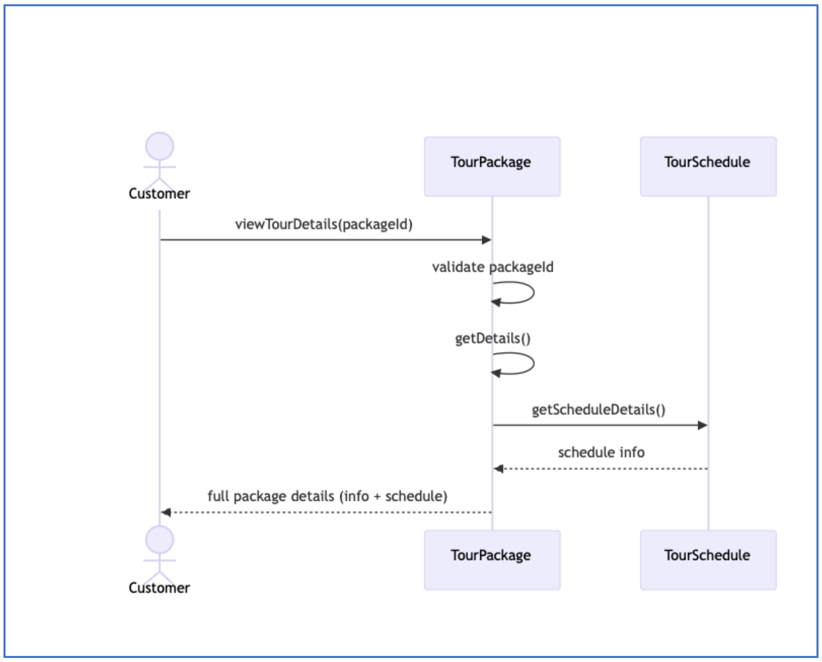
BR_TM_02 - Create Account and Login



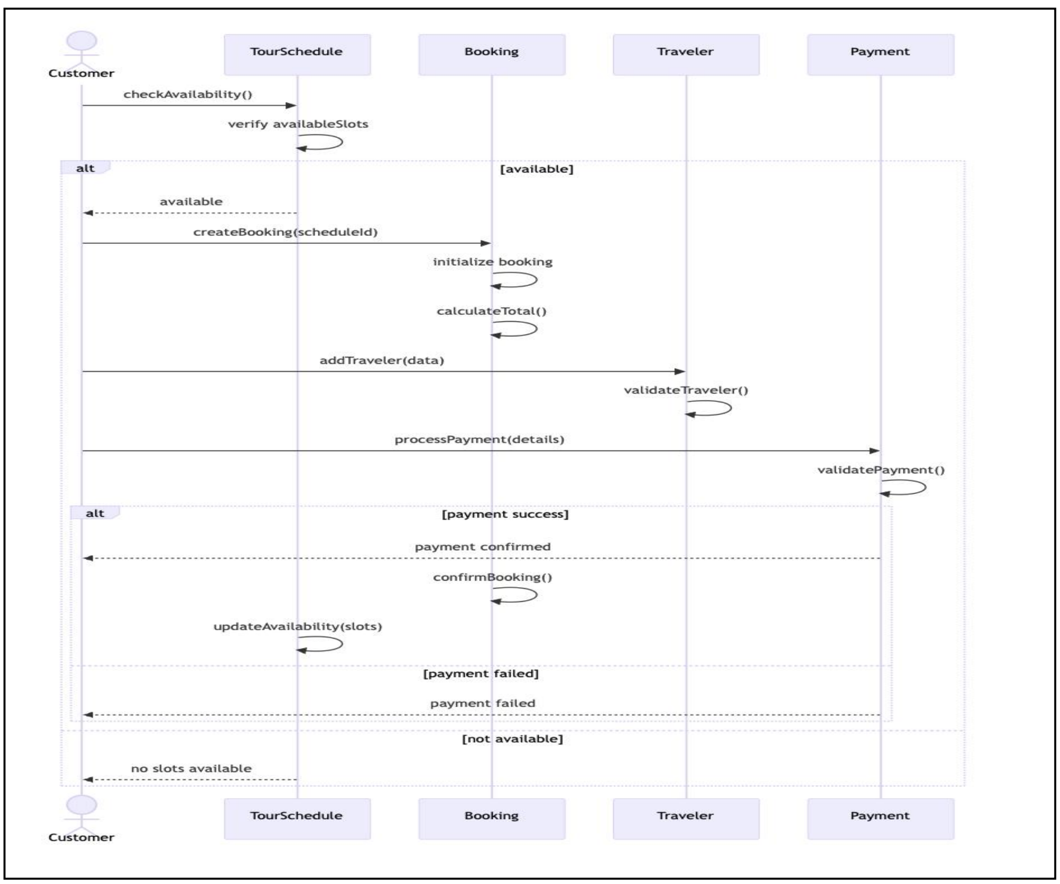
BR_TM_03 - Search and Filter Tour Packages



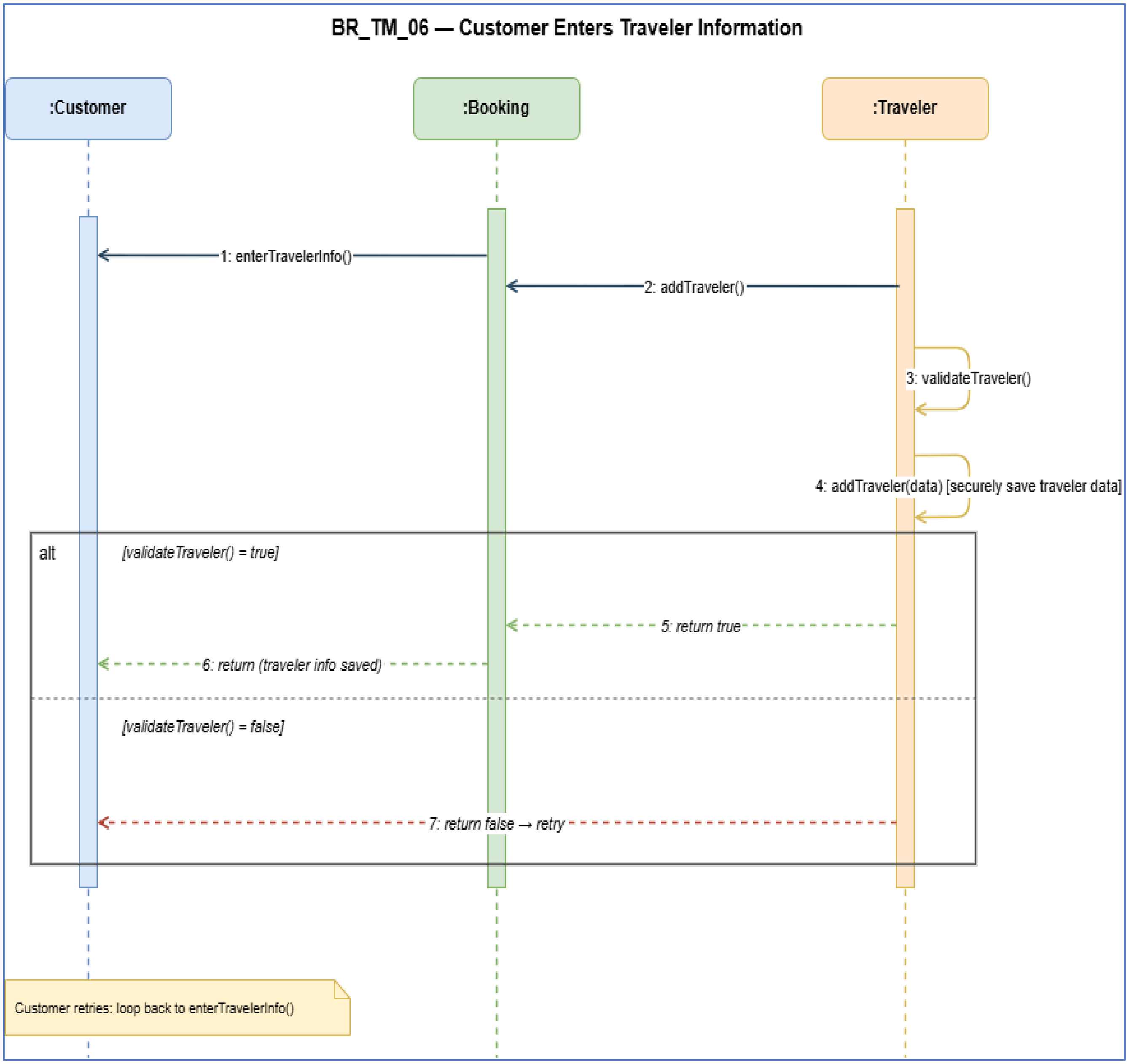
BR_TM_04 - View Tour Details

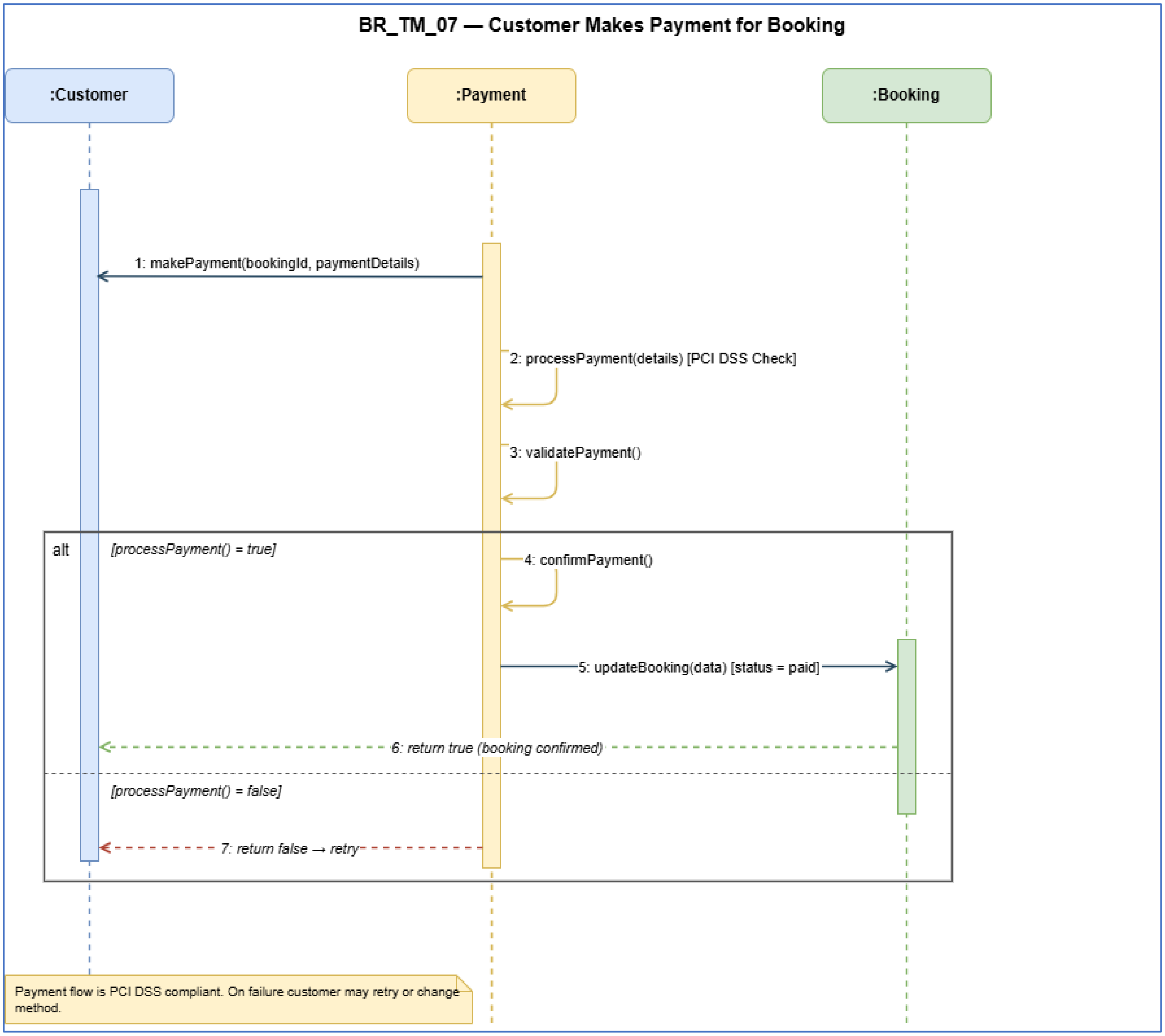


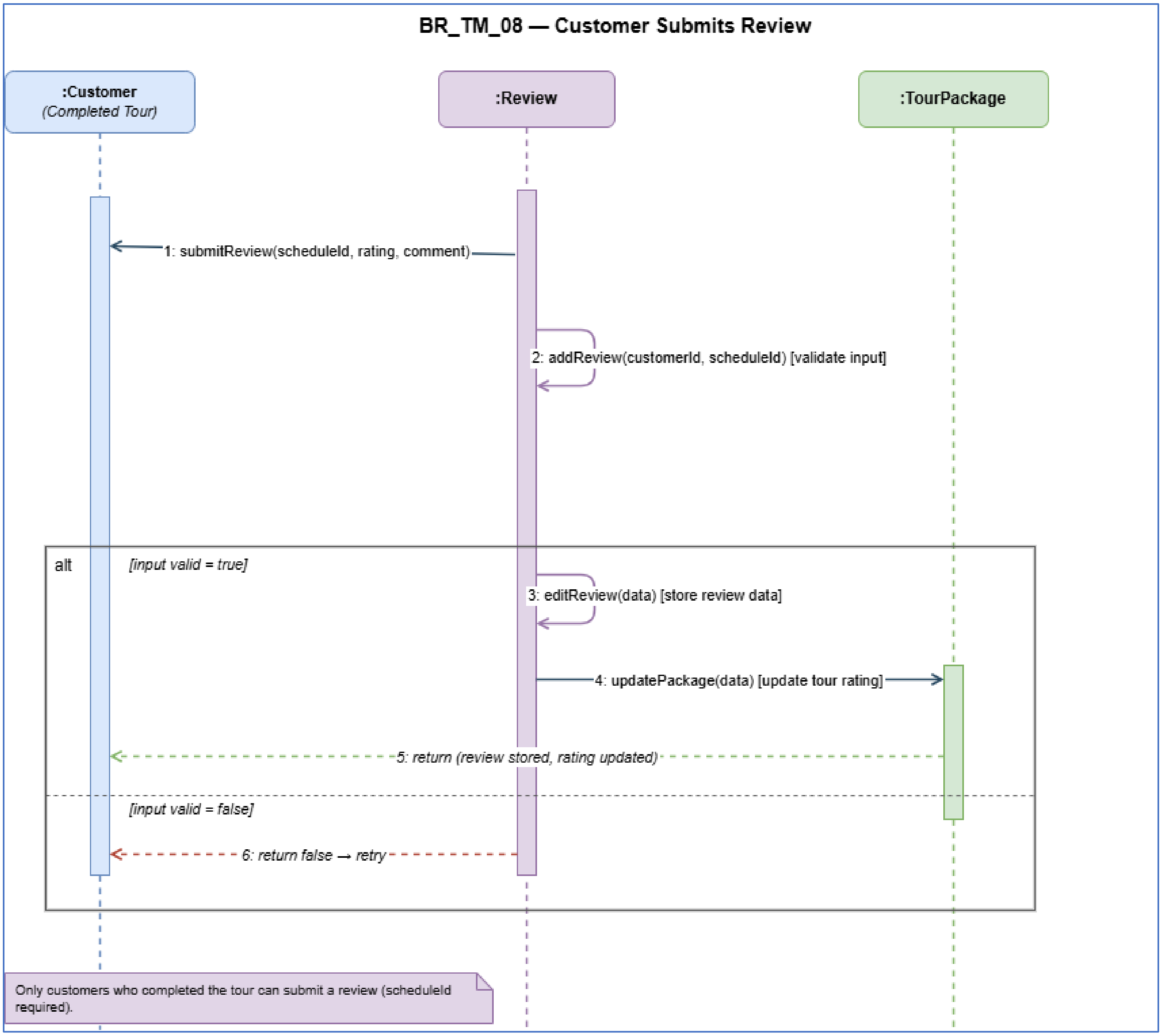
BR_TM_05 - Book Tour Package



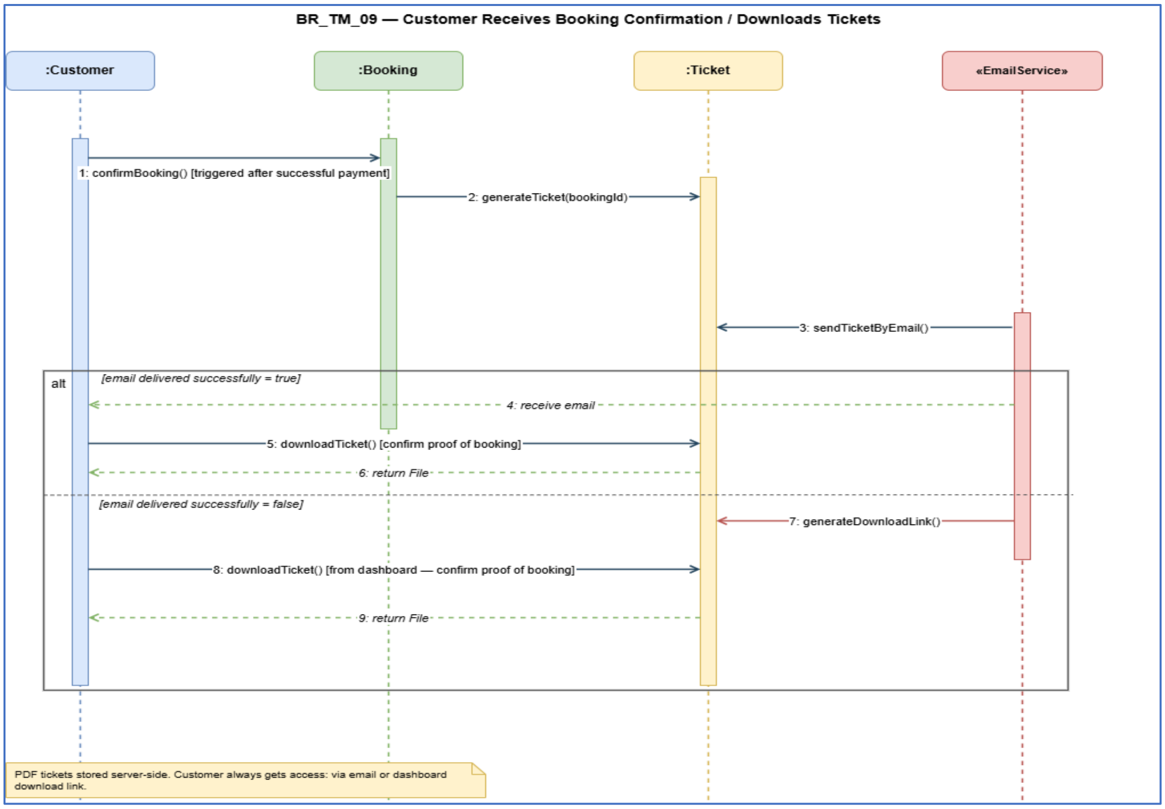
Alisa Dhima:

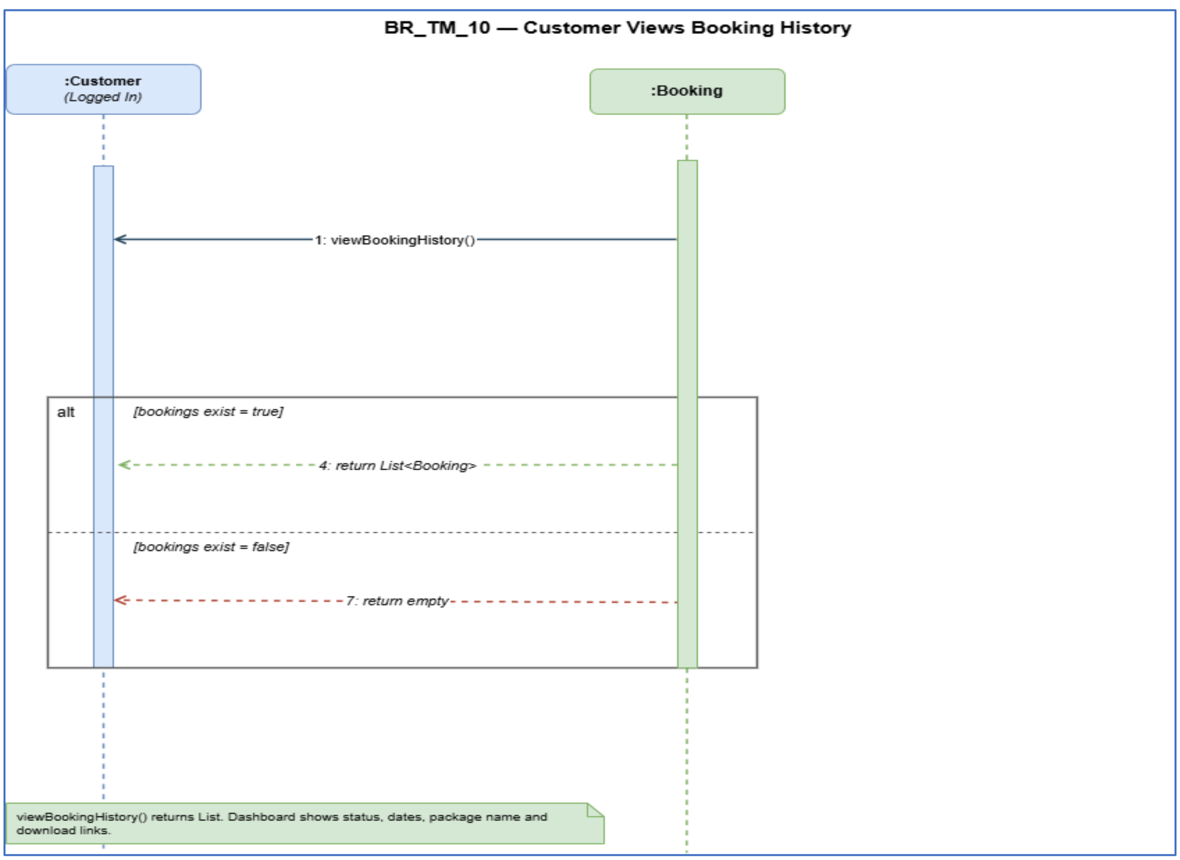

BR_TM_06 — Customer Enters Traveler Information
:Customer
:Booking
:Traveler
1: enterTravelerInfo()
2: addTraveler()
3: validateTraveler()
4: addTraveler(data) [securely save traveler data]
alt
[validateTraveler() = true]
5: return true
6: return (traveler info saved)
[validateTraveler() = false]
7: return false → retry
Customer retries: loop back to enterTravelerInfo()


BR_TM_07 — Customer Makes Payment for Booking
:Customer
:Payment
:Booking
1: makePayment(bookingId, paymentDetails)
2: processPayment(details) [PCI DSS Check]
3: validatePayment()
alt
[processPayment() = true]
4: confirmPayment()
5: updateBooking(data) [status = paid]
6: return true (booking confirmed)
[processPayment() = false]
7: return false → retry
Payment flow is PCI DSS compliant. On failure customer may retry or change method.



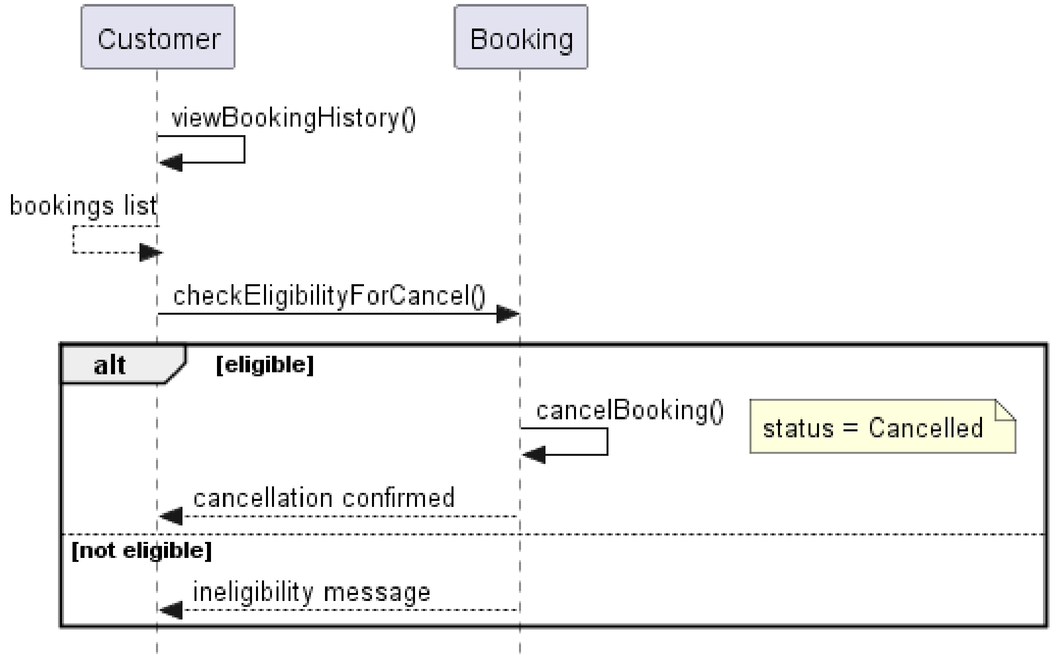
BR_TM_08 — Customer Submits Review
:Customer
(Completed Tour)
:Review
:TourPackage
1: submitReview(scheduleId, rating, comment)
2: addReview(customerId, scheduleId) [validate input]
alt
[input valid = true]
3: editReview(data) [store review data]
4: updatePackage(data) [update tour rating]
5: return (review stored, rating updated)
[input valid = false]
6: return false → retry
Only customers who completed the tour can submit a review (scheduleId required).


BR_TM_09 — Customer Receives Booking Confirmation / Downloads Tickets
:Customer
:Booking
:Ticket
«EmailService»
1: confirmBooking() [triggered after successful payment]
2: generateTicket(bookingId)
3: sendTicketByEmail()
alt
[email delivered successfully = true]
4: receive email
5: downloadTicket() [confirm proof of booking]
6: return File
[email delivered successfully = false]
7: generateDownloadLink()
8: downloadTicket() [from dashboard — confirm proof of booking]
9: return File
PDF tickets stored server-side. Customer always gets access: via email or dashboard download link.

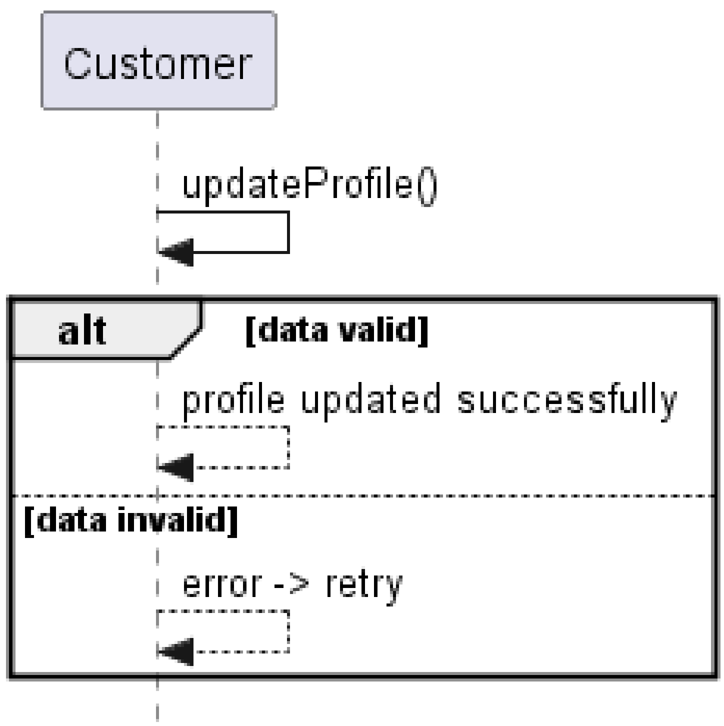

BR_TM_10 — Customer Views Booking History
:Customer
(Logged In)
:Booking
1: viewBookingHistory()
alt
[bookings exist = true]
4: return List<Booking>
[bookings exist = false]
7: return empty
viewBookingHistory() returns List. Dashboard shows status, dates, package name and download links.

Amanda Stafasani:

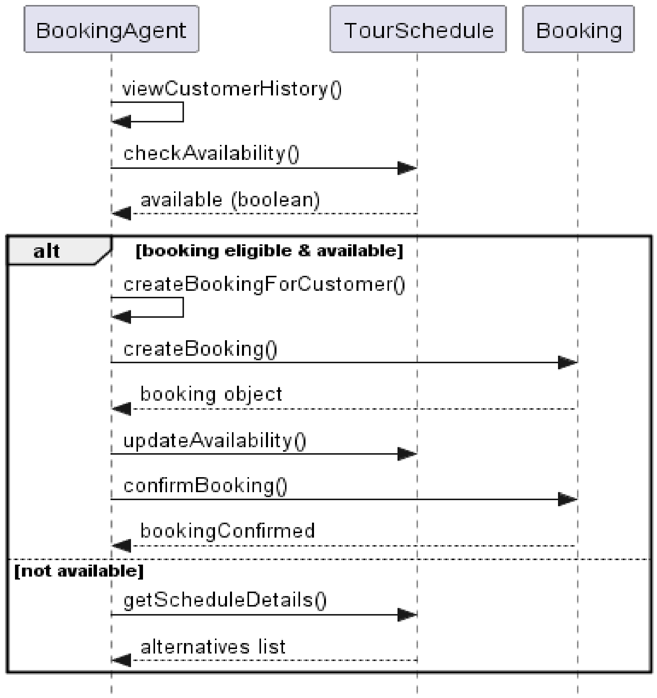
BR_TM_11 — Customer cancels a booking



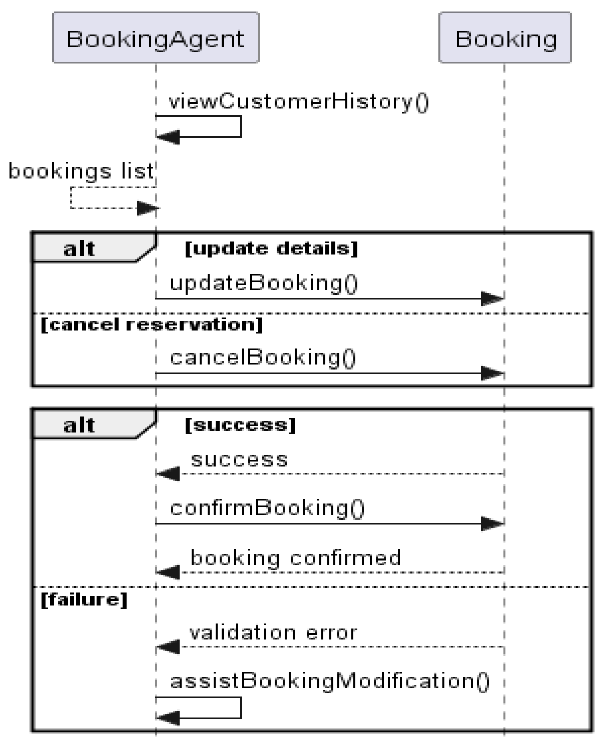
BR_TM_12 — Customer updates personal profile



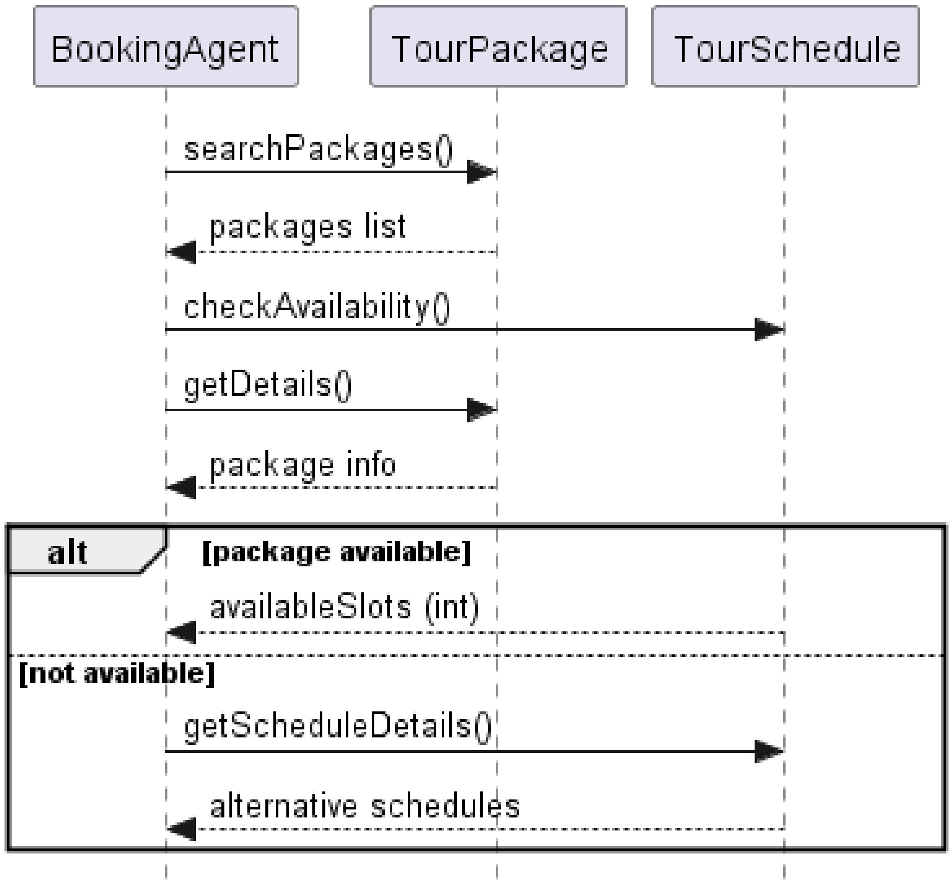
BR_TM_13 — Booking agent creates a booking for a customer



BR_TM_14 — Booking agent updates or cancels a reservation

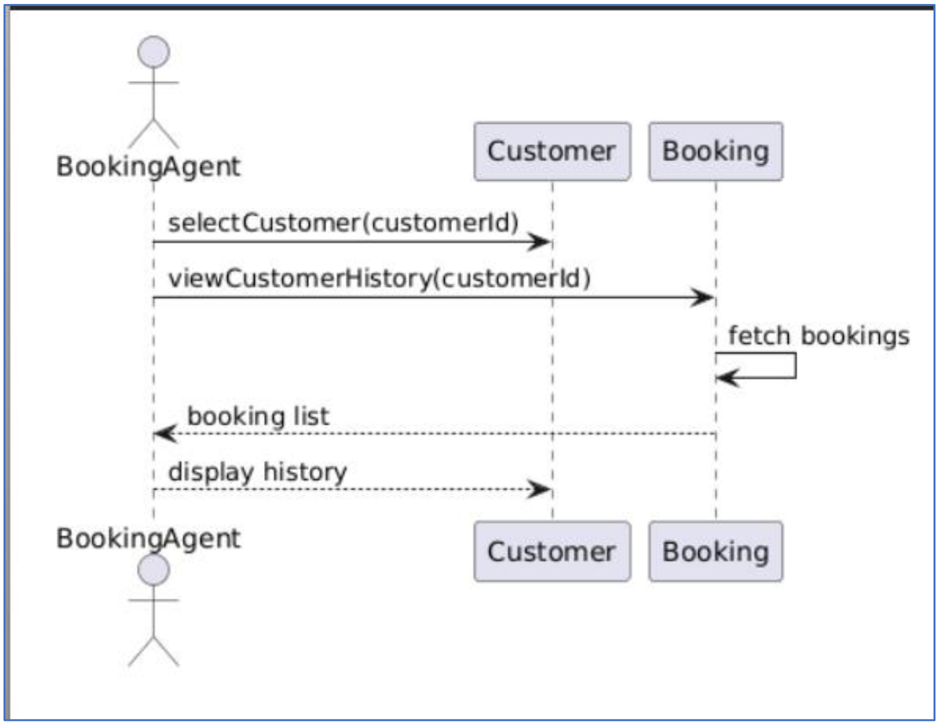


BR_TM_15 — Booking agent checks travel package availability

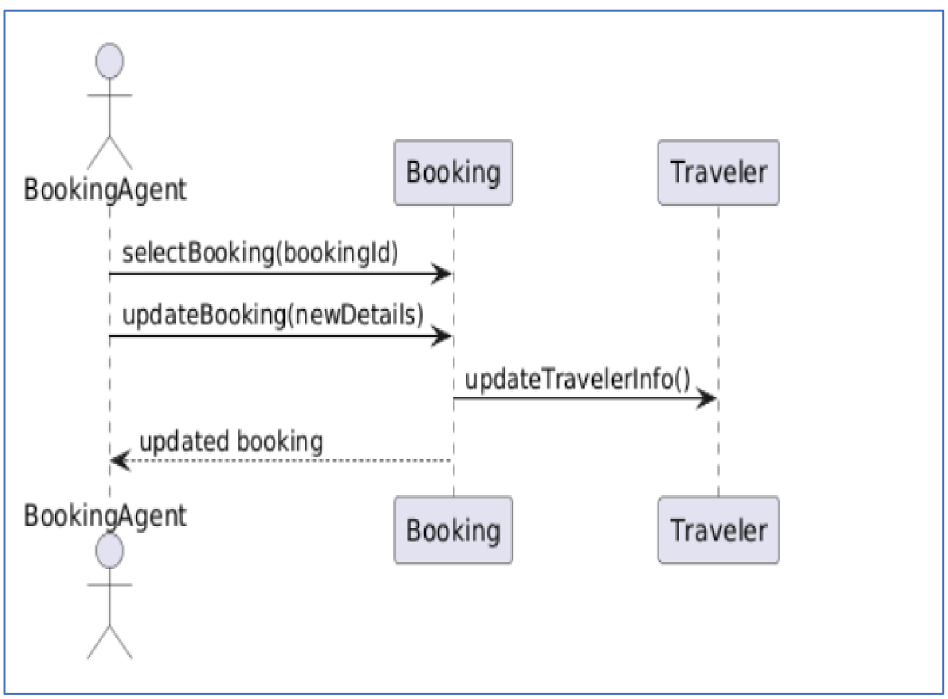


Lumturi Bedalli:

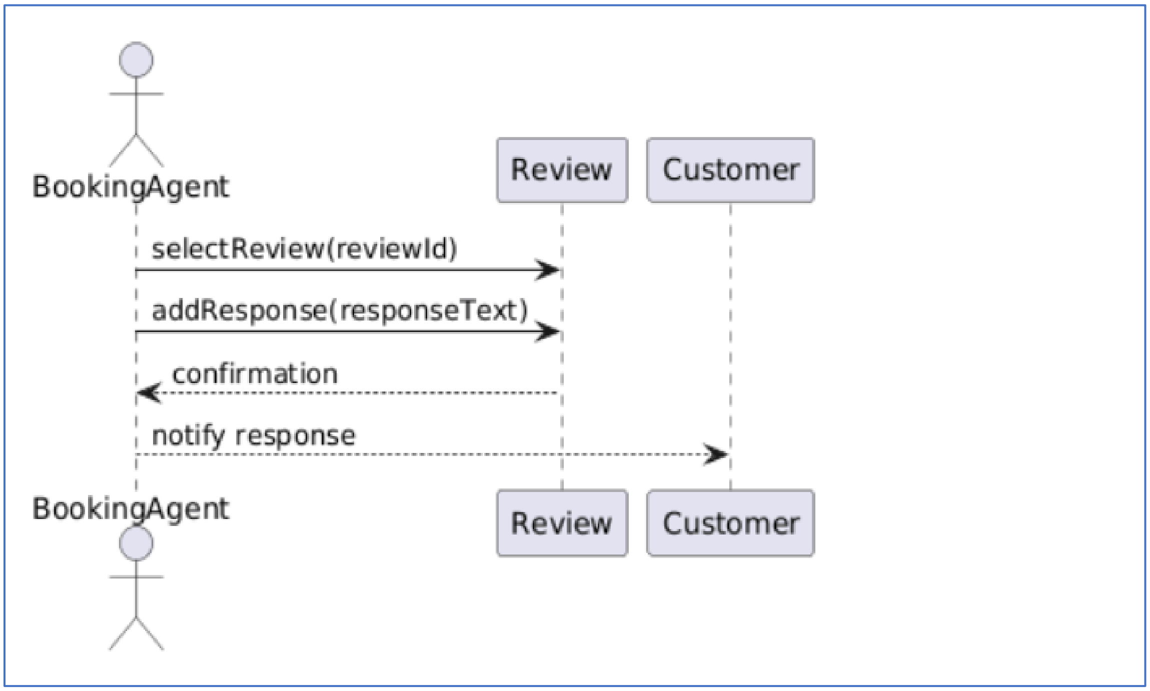
BR_TM_16 – Agent Views Customer Travel History



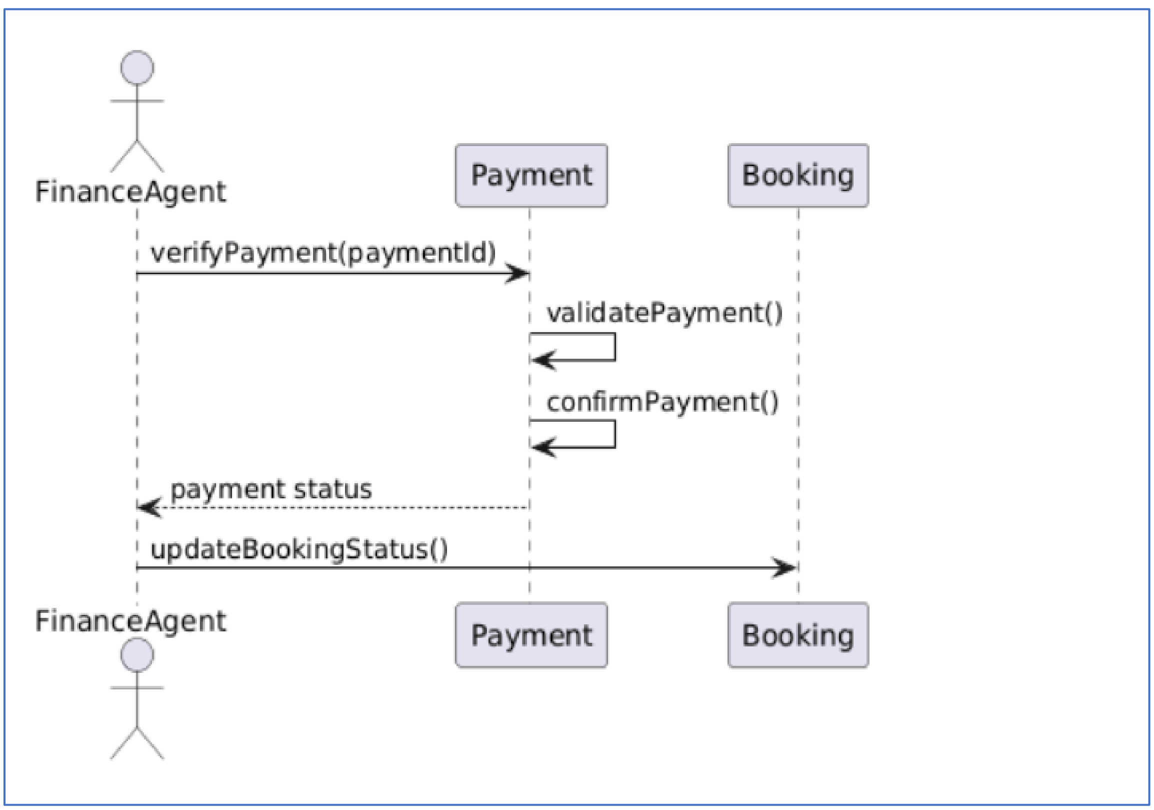
BR_TM_17 – Modify Existing Booking



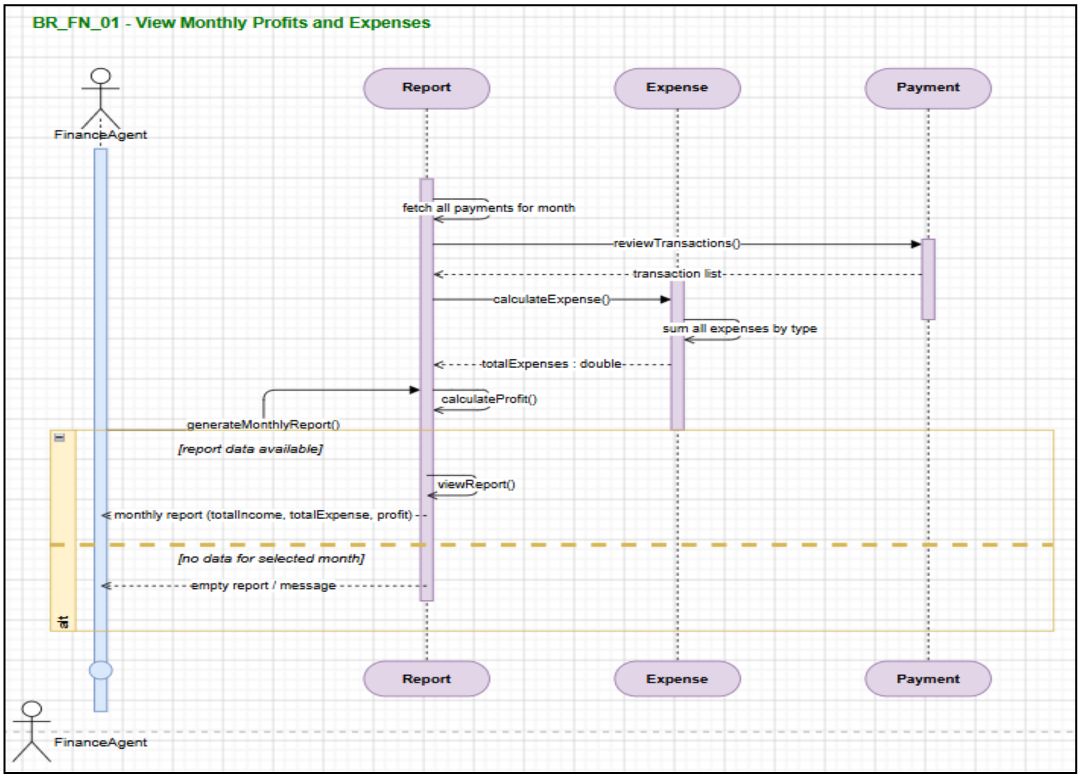
BR_TM_18 – Reply to Customer Reviews

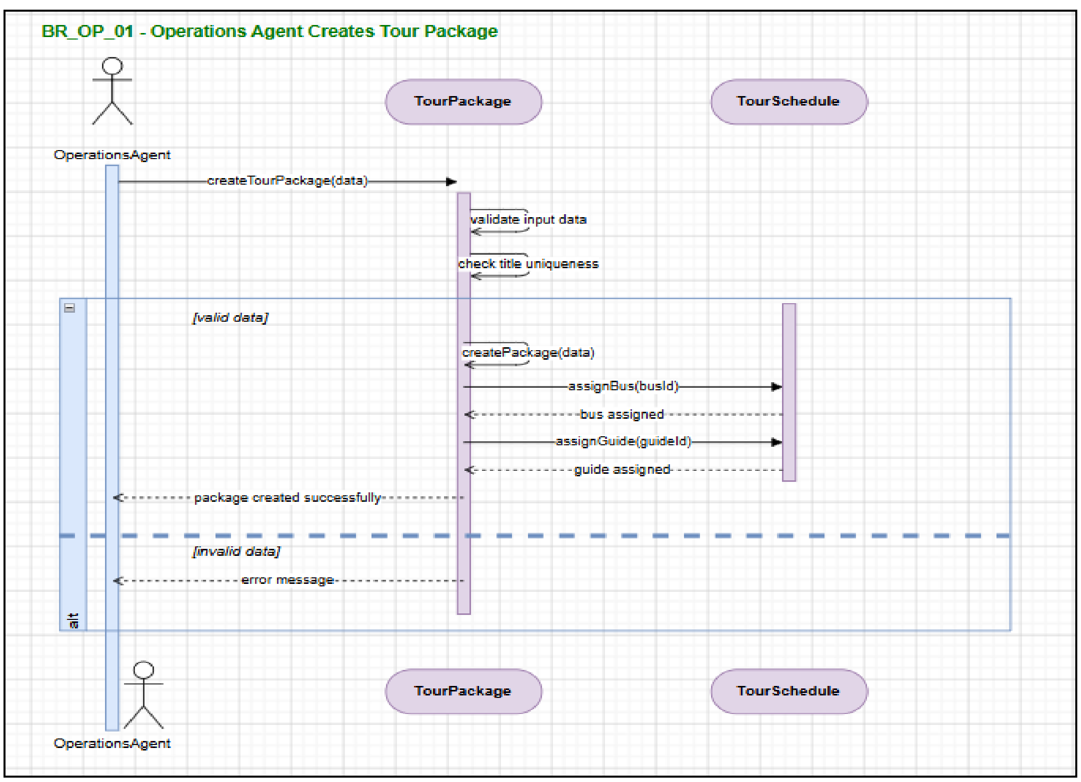


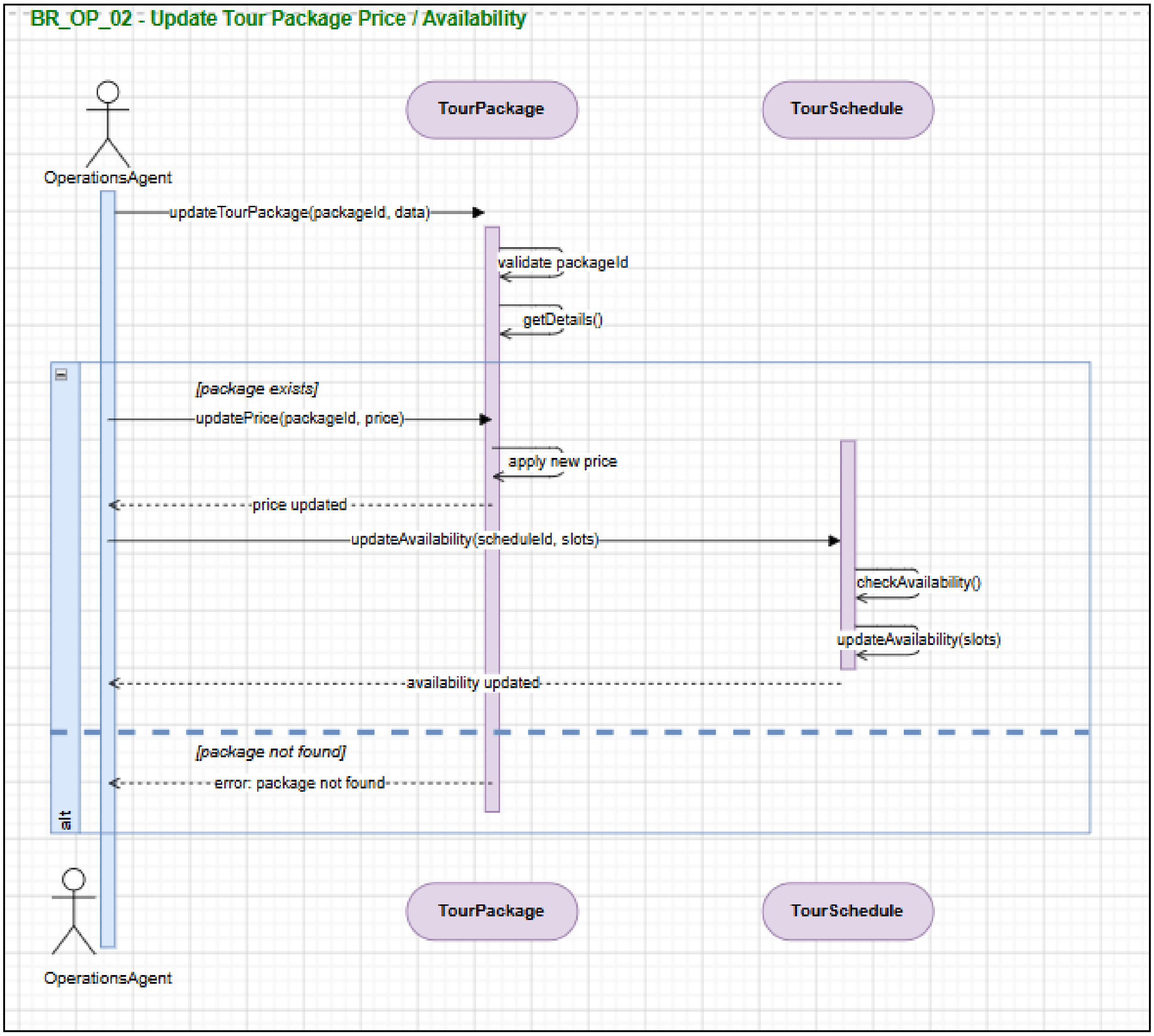
BR_TM_19 – Verify Customer Payments

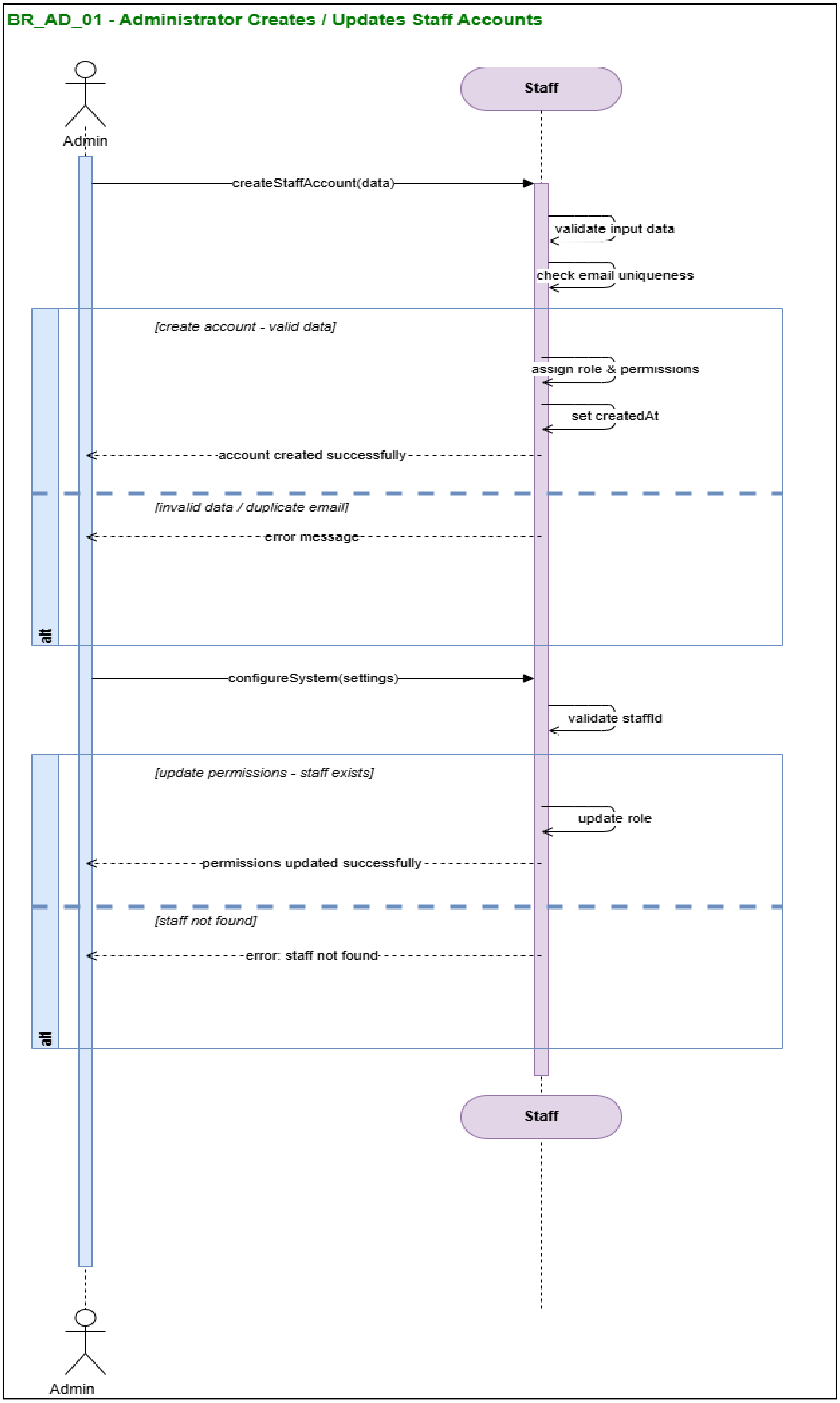


Henri Shehu:


BR_FN_01 - View Monthly Profits and Expenses
FinanceAgent
Report
Expense
Payment
fetch all payments for month
reviewTransactions()
transaction list
calculateExpense()
sum all expenses by type
totalExpenses : double
calculateProfit()
generateMonthlyReport()
[report data available]
viewReport()
monthly report (totalIncome, totalExpense, profit)
[no data for selected month]
empty report / message
alt

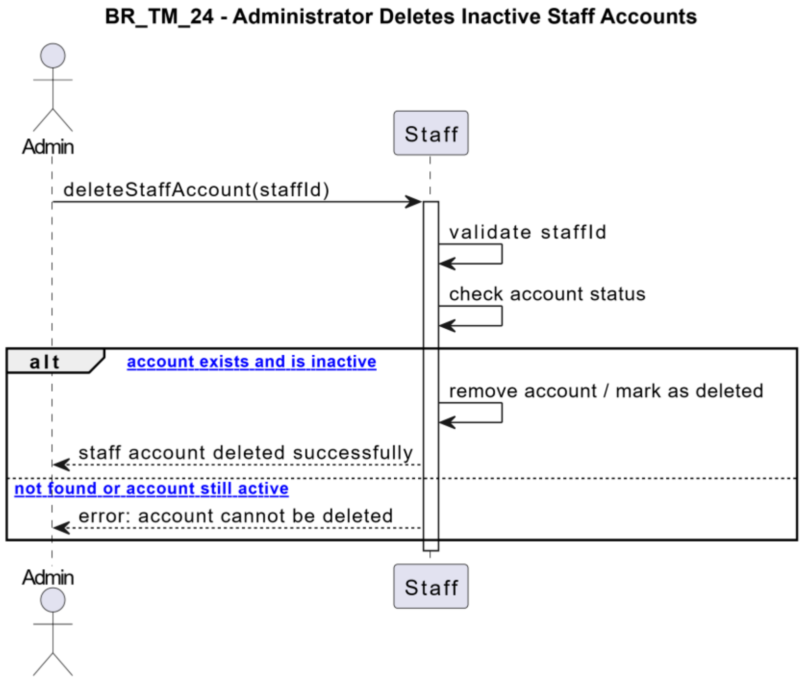

BR_OP_01 - Operations Agent Creates Tour Package
OperationsAgent
TourPackage
TourSchedule
createTourPackage(data)
validate input data
check title uniqueness
[valid data]
createPackage(data)
assignBus(busId)
bus assigned
assignGuide(guideId)
guide assigned
package created successfully
[invalid data]
error message
alt


BR_OP_02 - Update Tour Package Price / Availability
OperationsAgent
TourPackage
TourSchedule
updateTourPackage(packageId, data)
validate packageId
getDetails()
[package exists]
updatePrice(packageId, price)
apply new price
price updated
updateAvailability(scheduleId, slots)
checkAvailability()
updateAvailability(slots)
availability updated
[package not found]
error: package not found
alt
OperationsAgent
TourPackage
TourSchedule

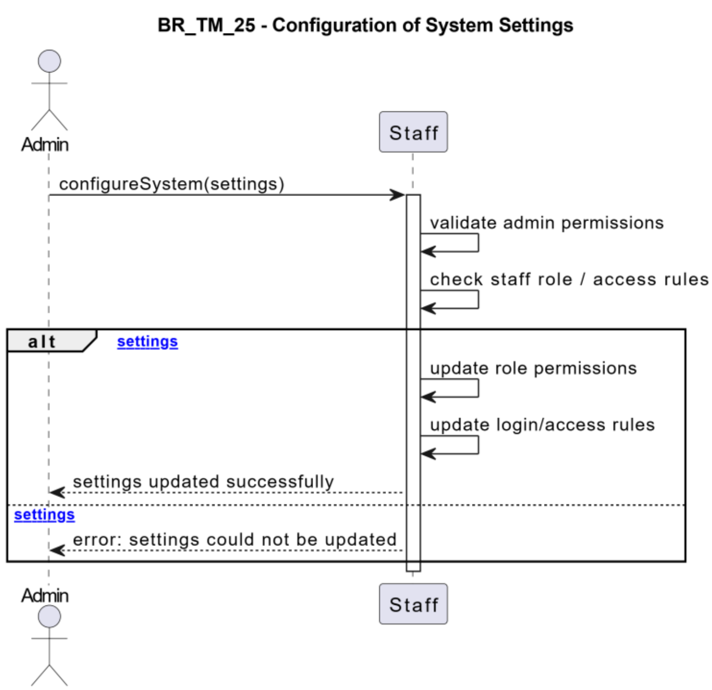

BR_AD_01 - Administrator Creates / Updates Staff Accounts
Admin
Staff
createStaffAccount(data)
validate input data
check email uniqueness
[create account - valid data]
assign role & permissions
set createdAt
account created successfully
[invalid data / duplicate email]
error message
alt
configureSystem(settings)
validate staffId
[update permissions - staff exists]
update role
permissions updated successfully
[staff not found]
error: staff not found
alt
Staff
Admin

Enian Xhixha:

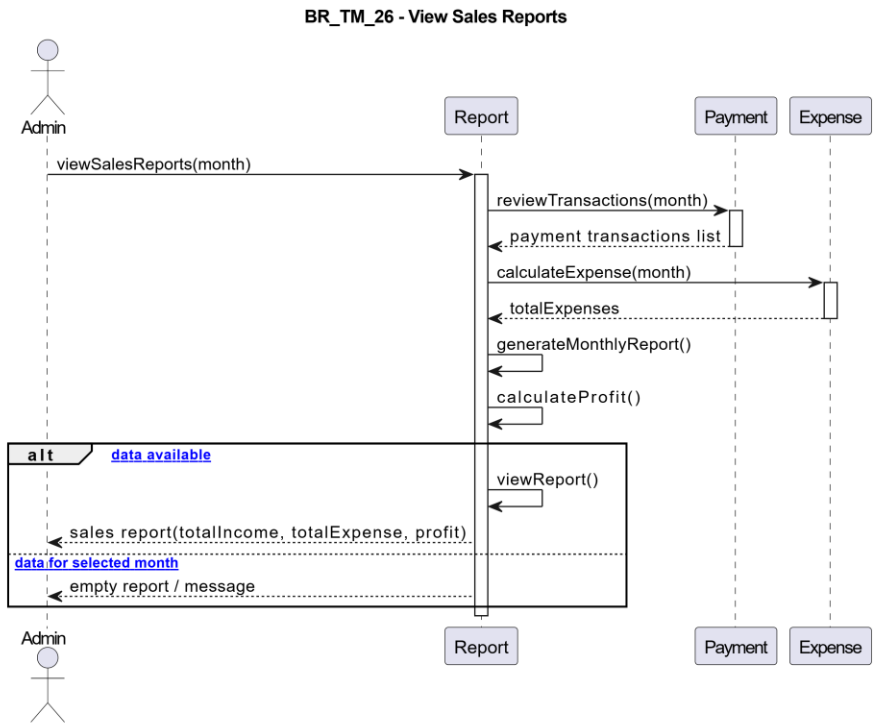
BR-TM-24 - Administrator deletes inactive staff accounts



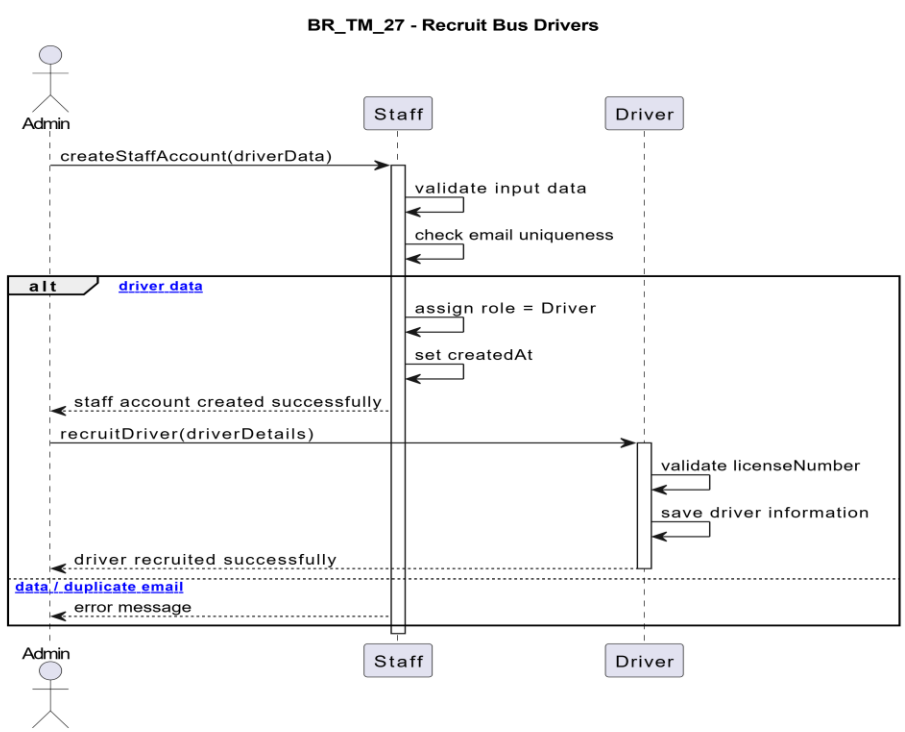
BR-TM-25 - Configuration of system settings



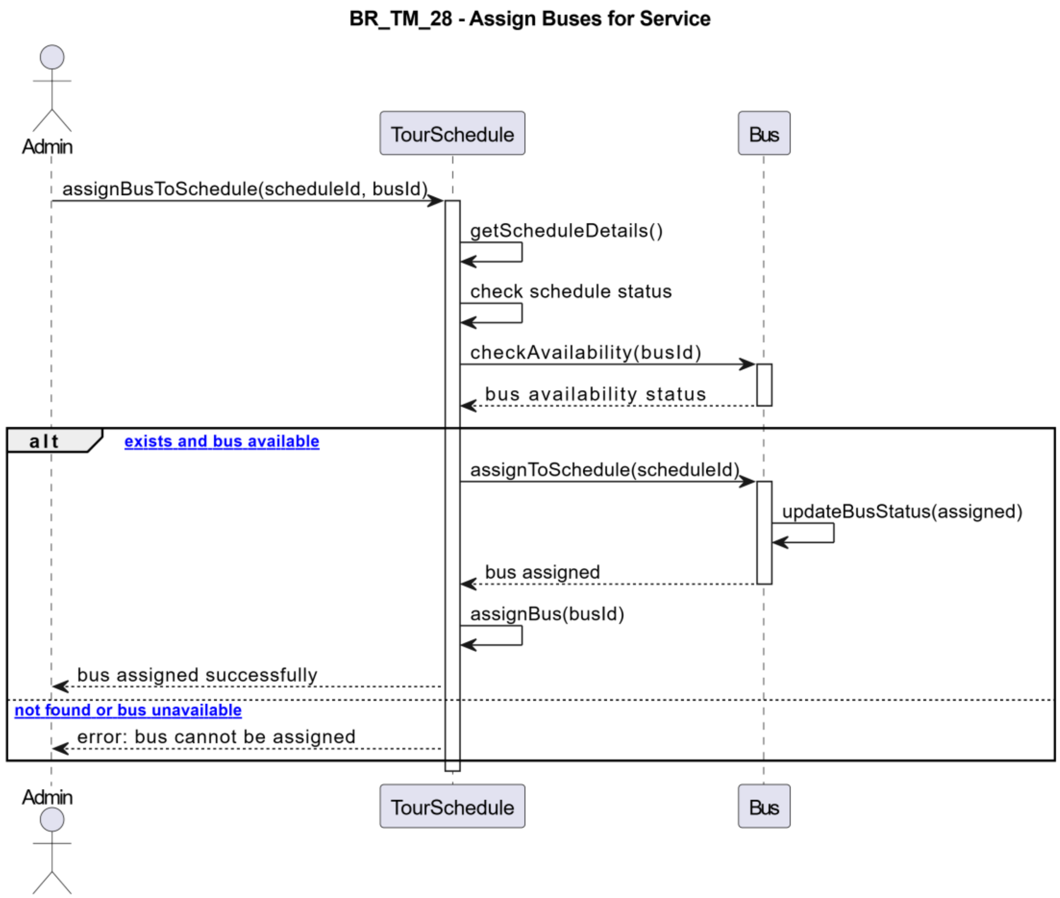
BR-TM-26: View sales reports



BR-TM-27: Recruit Bus Drivers

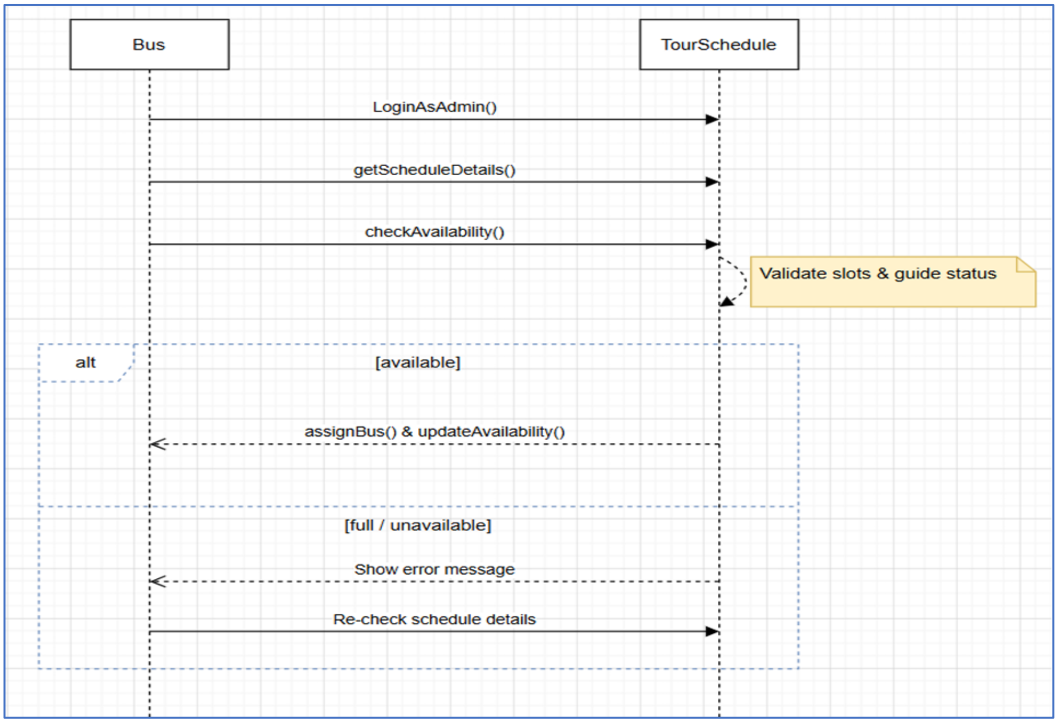


BR-TM-28: Assign Buses for Service

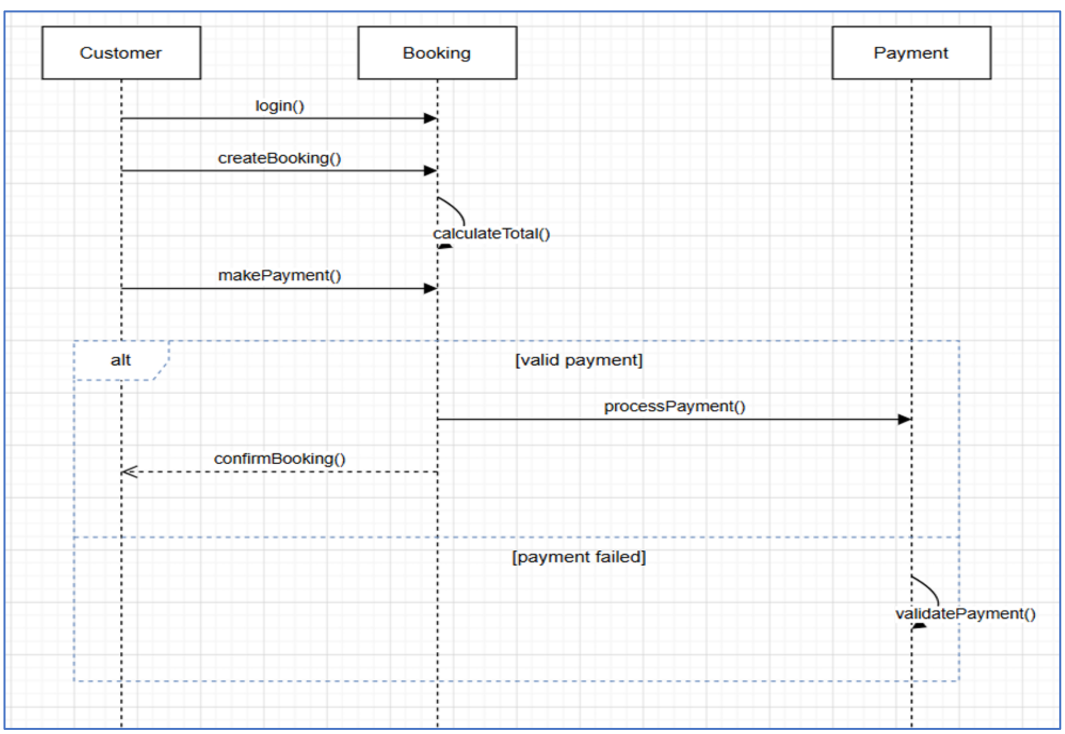


Era Selmanaj:

BR_TM_29 - Register number of buses



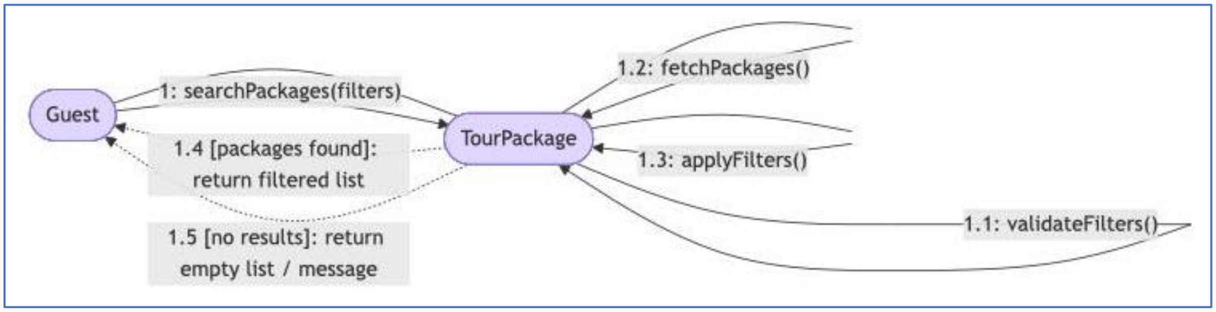
BR_TM_30 - Confirm Clients and Collect Payments



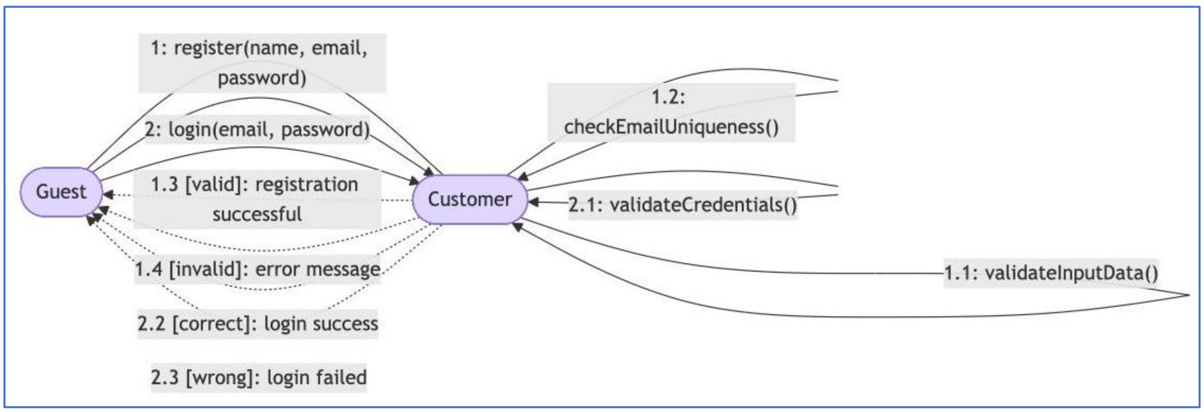
5.7 Collaboration Diagrams

Amar Kruja and Ermi Xhaferi

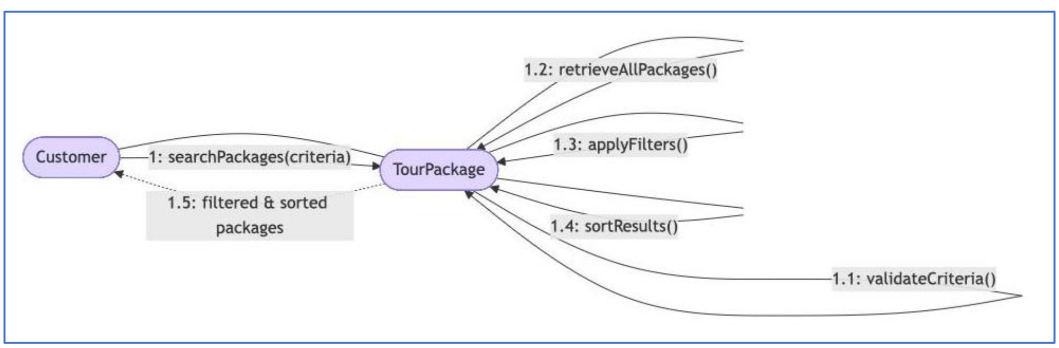
BR_TM_01 - View and Search Tour Packages



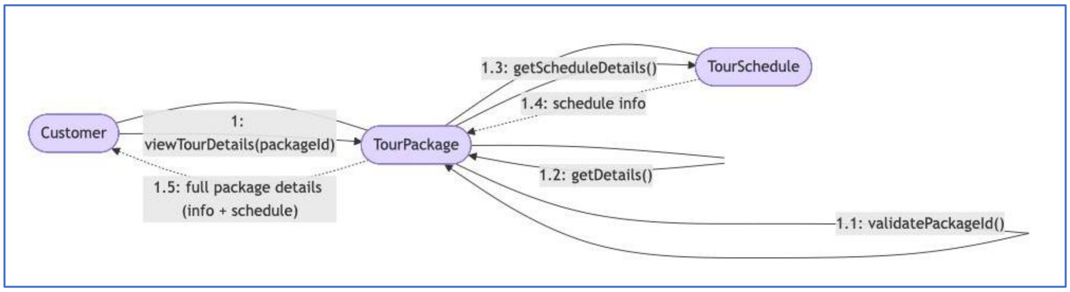
BR_TM_02 - Create Account and Login



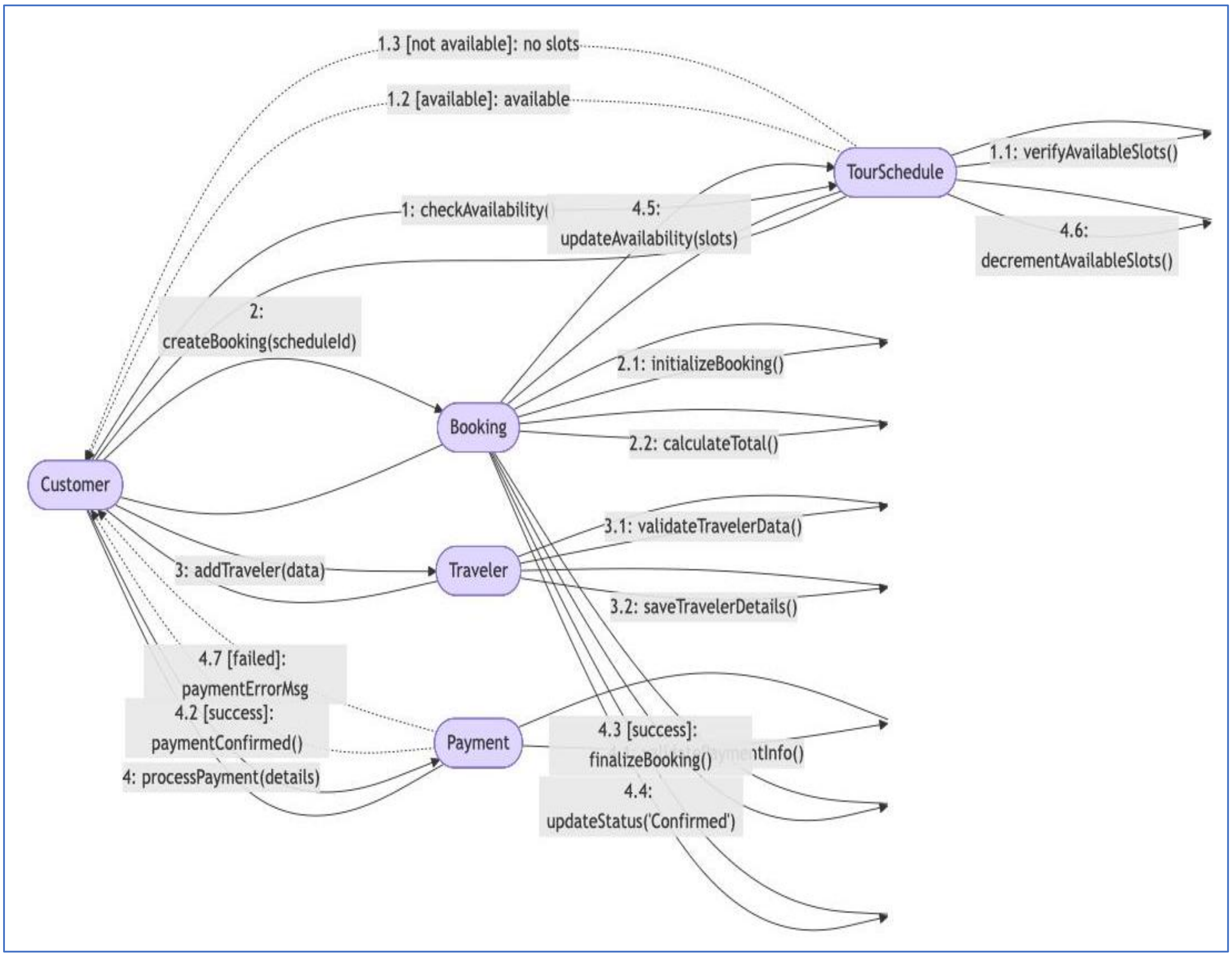
BR_TM_03 - Search and Filter Tour Packages



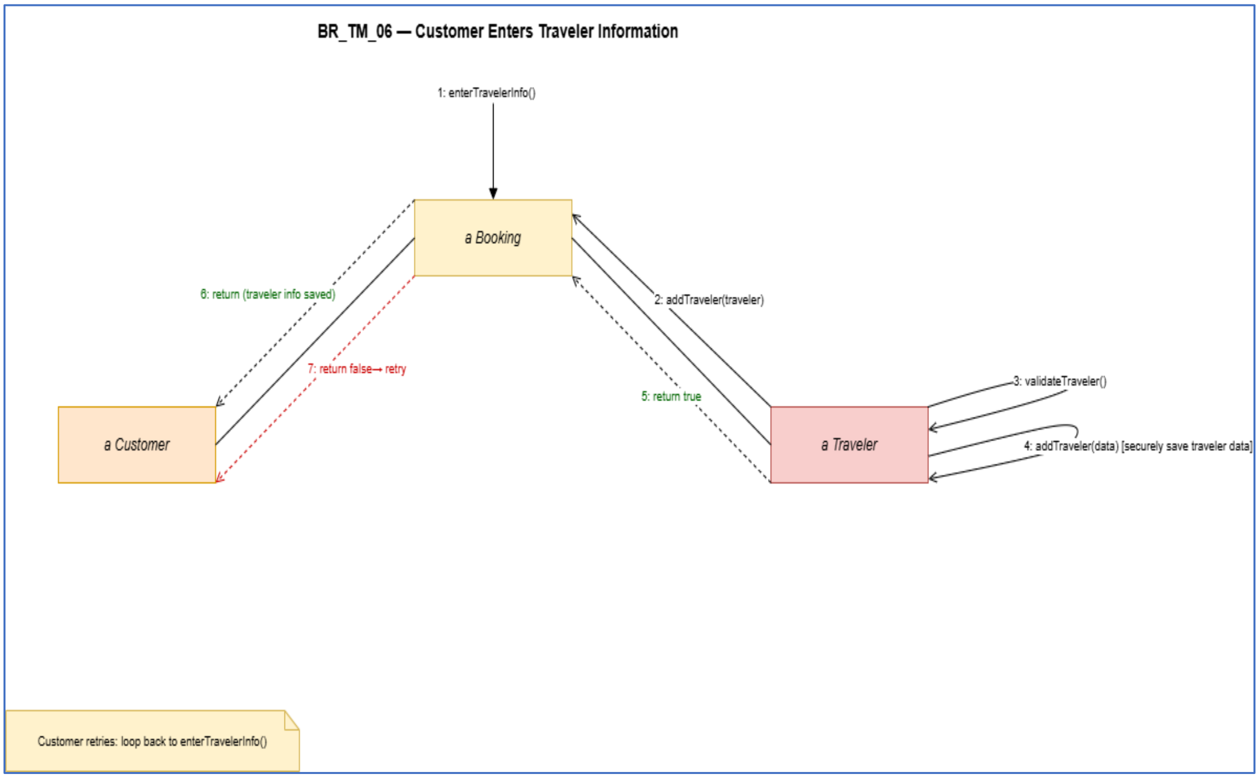
BR_TM_04 - View Tour Details

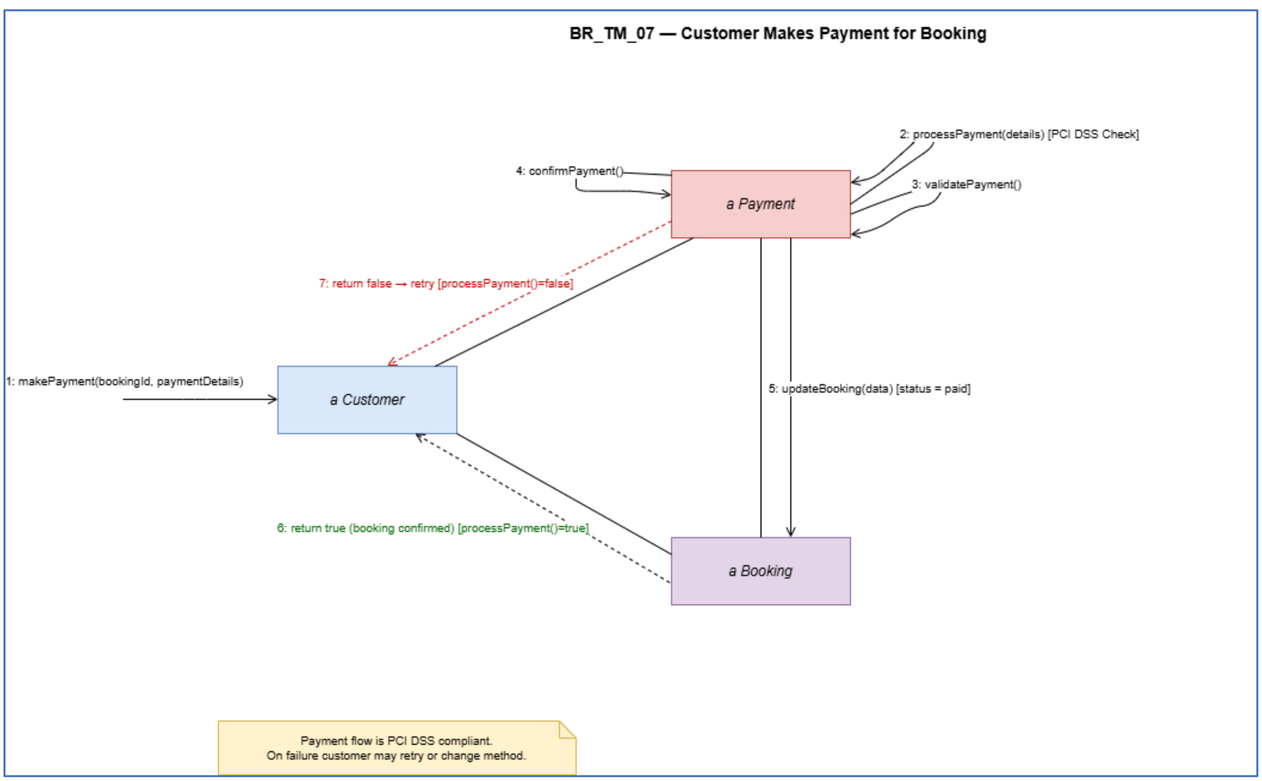


BR_TM_05 - Book Tour Package

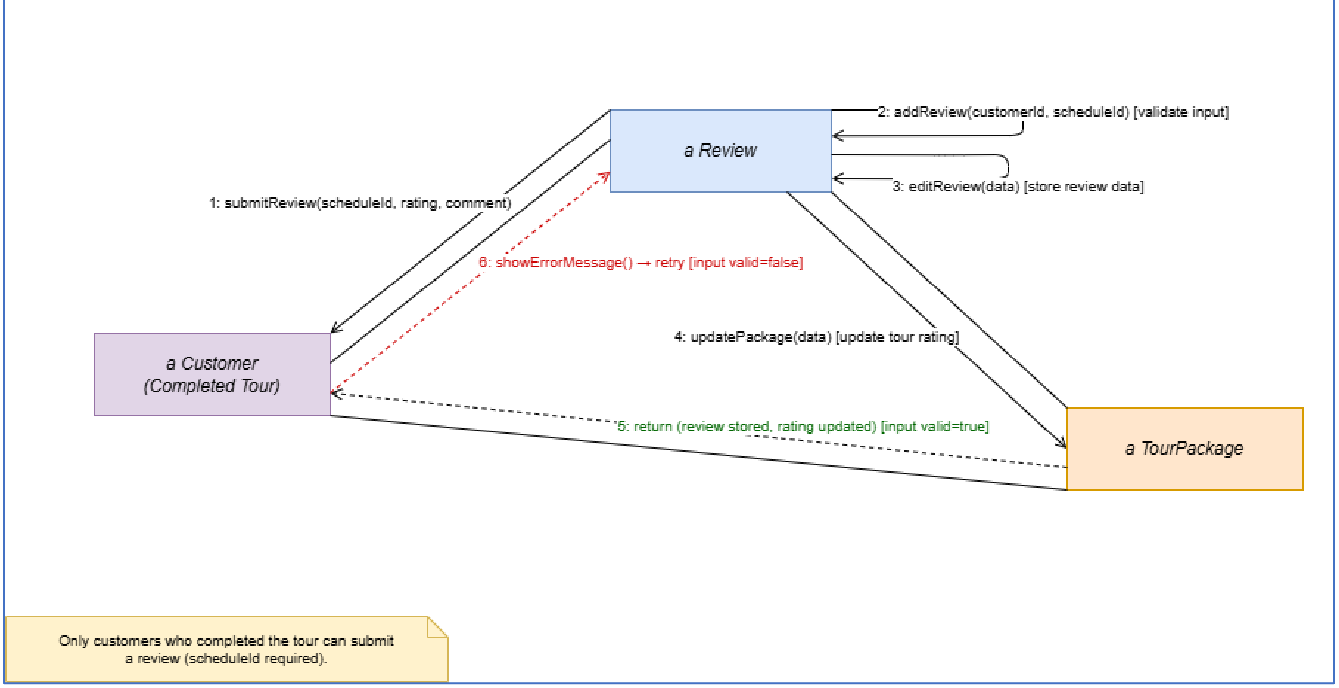


Alisa Dhima

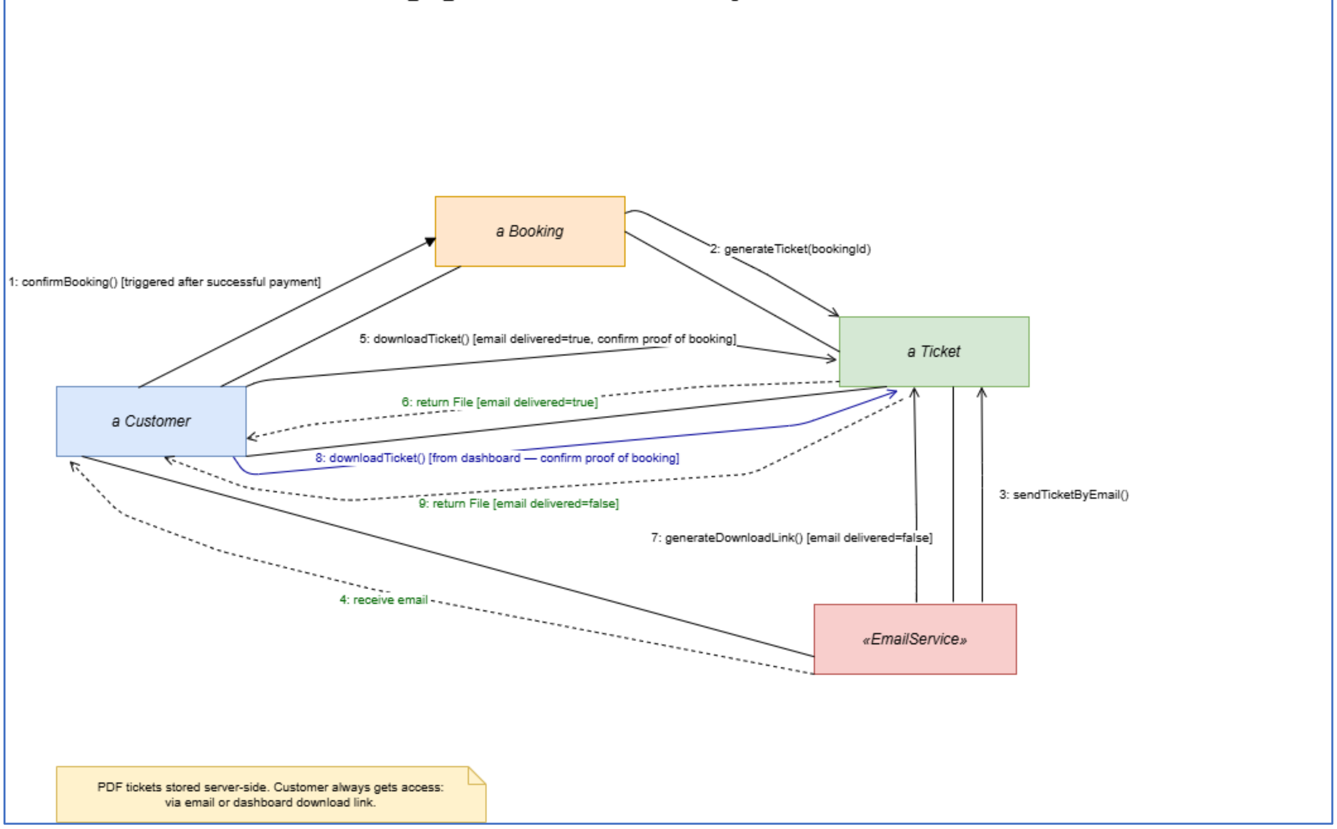

BR_TM_06 — Customer Enters Traveler Information
1: enterTravelerInfo()
a Booking
6: return (traveler info saved)
7: return false→ retry
2: addTraveler(traveler)
5: return true
a Customer
a Traveler
3: validateTraveler()
4: addTraveler(data) [securely save traveler data]
Customer retries: loop back to enterTravelerInfo()

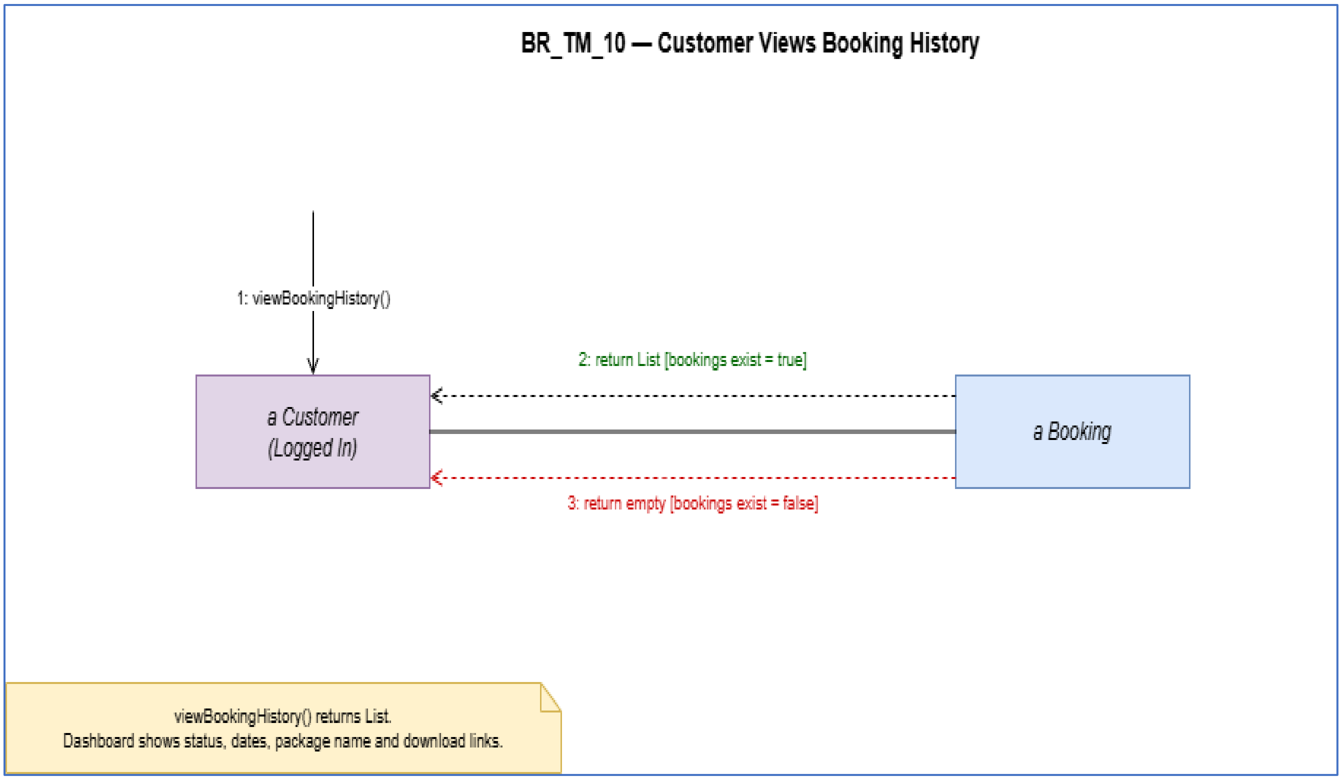

BR_TM_07 — Customer Makes Payment for Booking
2: processPayment(details) [PCI DSS Check]
4: confirmPayment()
a Payment
3: validatePayment()
7: return false → retry [processPayment()=false]
1: makePayment(bookingId, paymentDetails)
a Customer
5: updateBooking(data) [status = paid]
6: return true (booking confirmed) [processPayment()=true]
a Booking
Payment flow is PCI DSS compliant.
On failure customer may retry or change method.

BR_TM_08 — Customer Submits Review



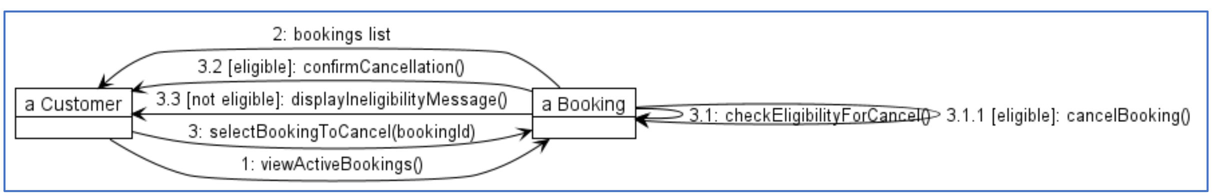
BR_TM_09 — Customer Receives Booking Confirmation / Downloads Tickets



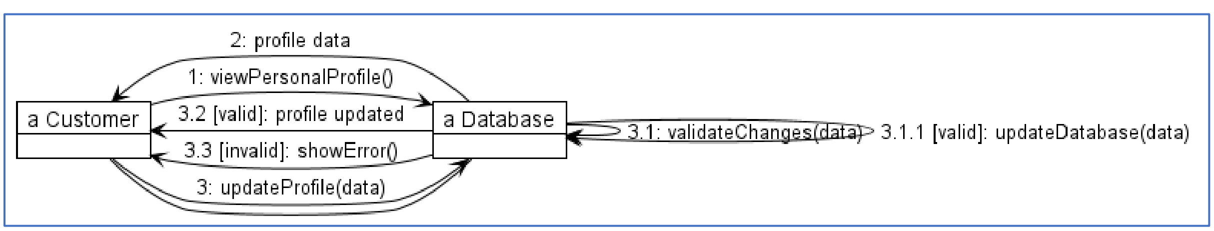


Amanda Stafasani

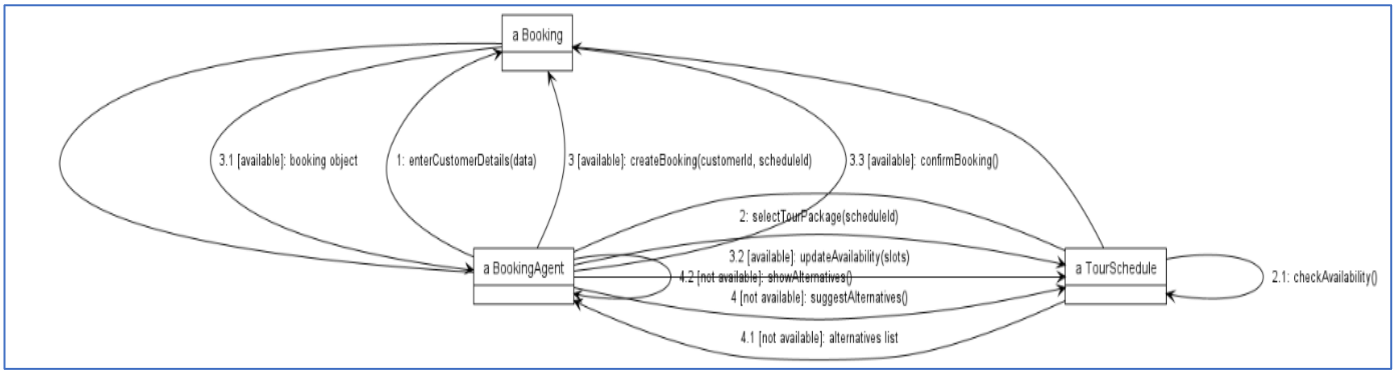
BR_TM_11 — Customer cancels a booking



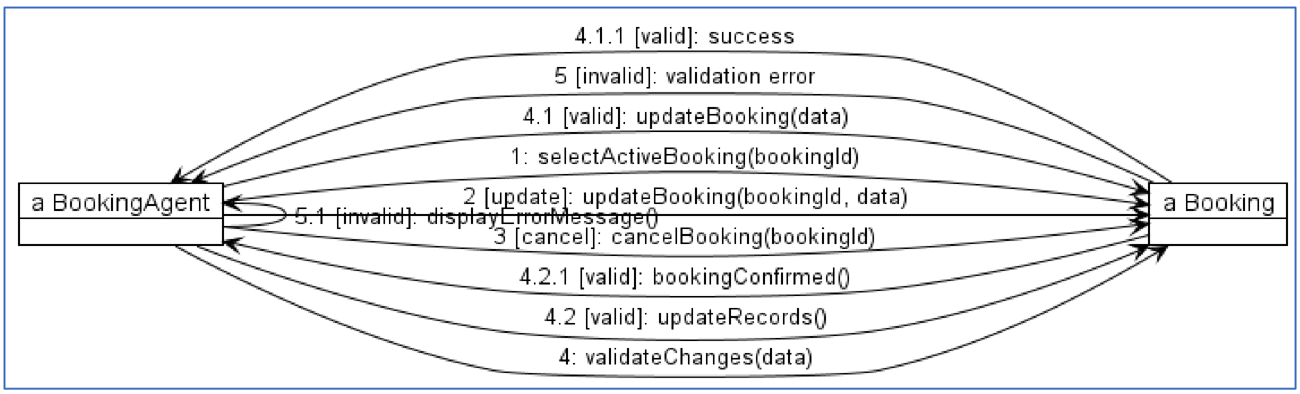
BR_TM_12 — Customer updates personal profile



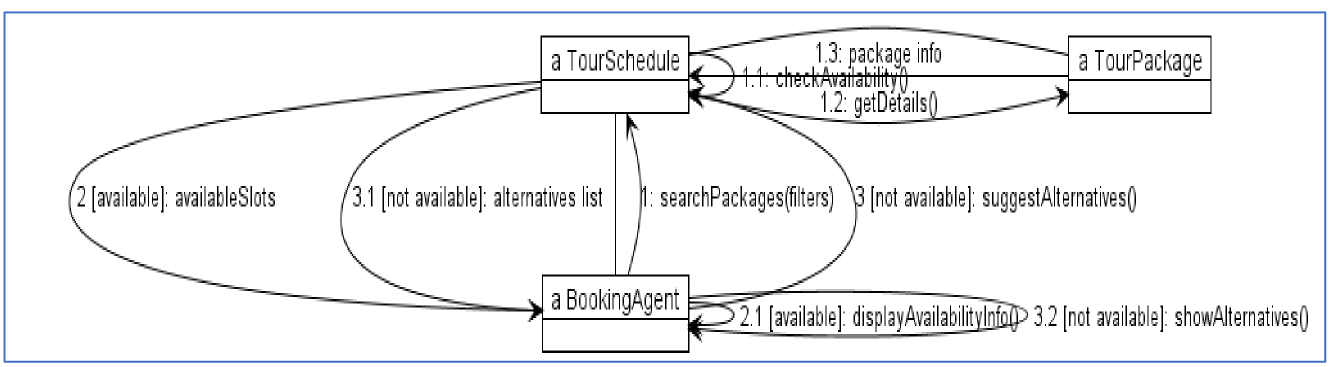
BR_TM_13 — Booking agent creates a booking for a customer



BR_TM_14 — Booking agent updates or cancels a reservation

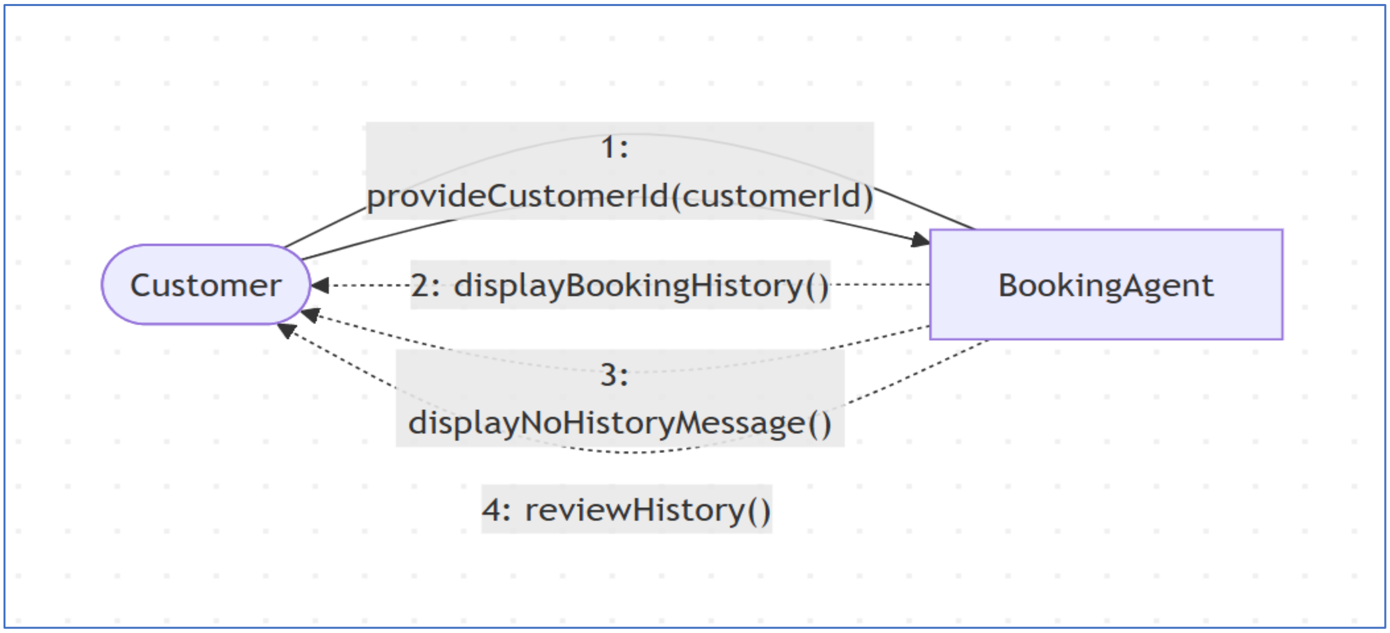


BR_TM_15 — Booking agent checks travel package availability

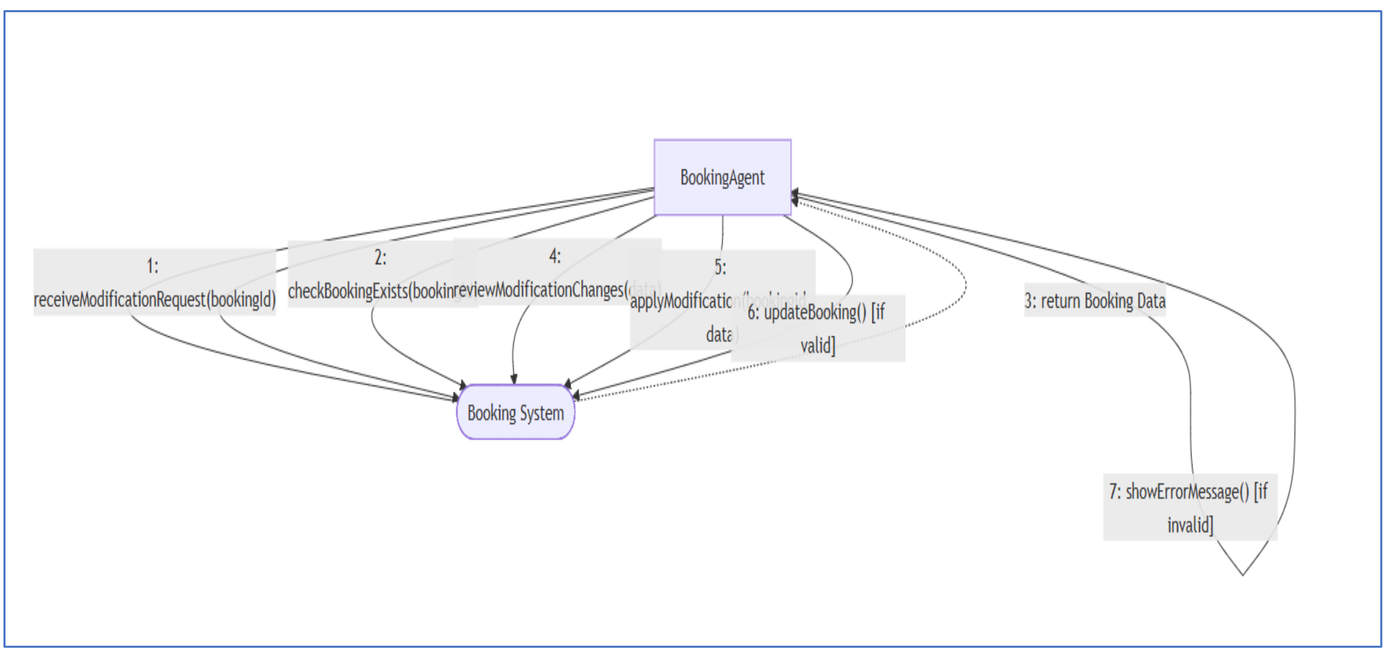


Lumturi Bedalli

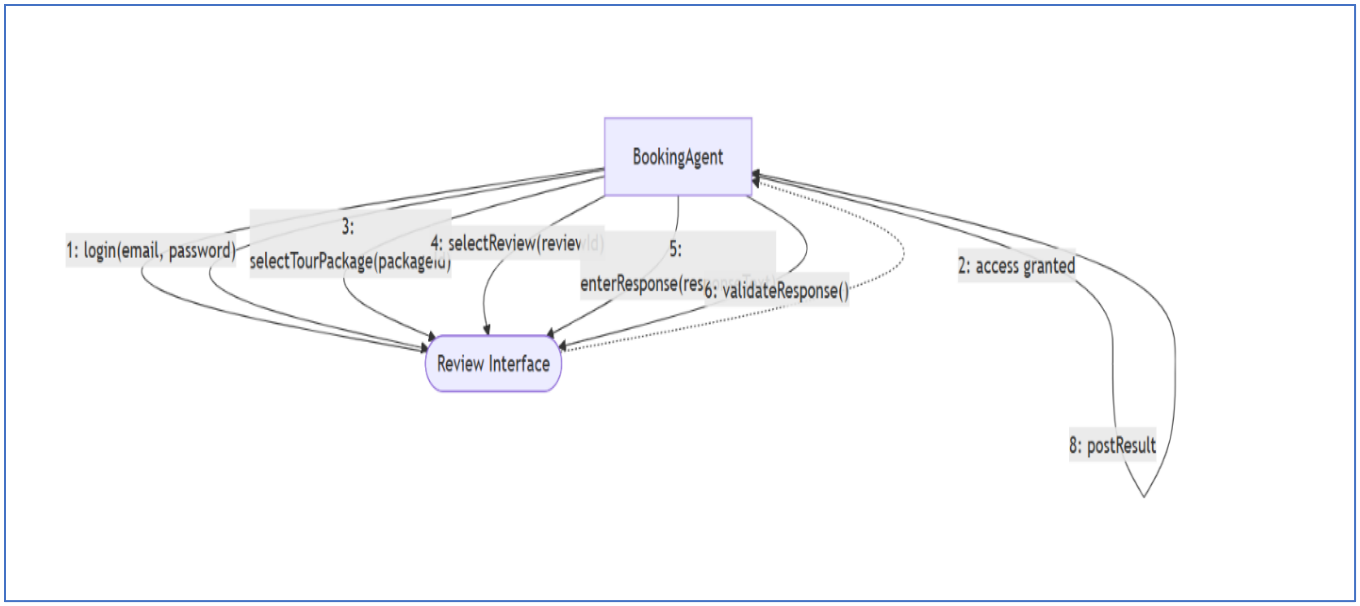
BR_TM_16 – Agent Views Customer Travel History



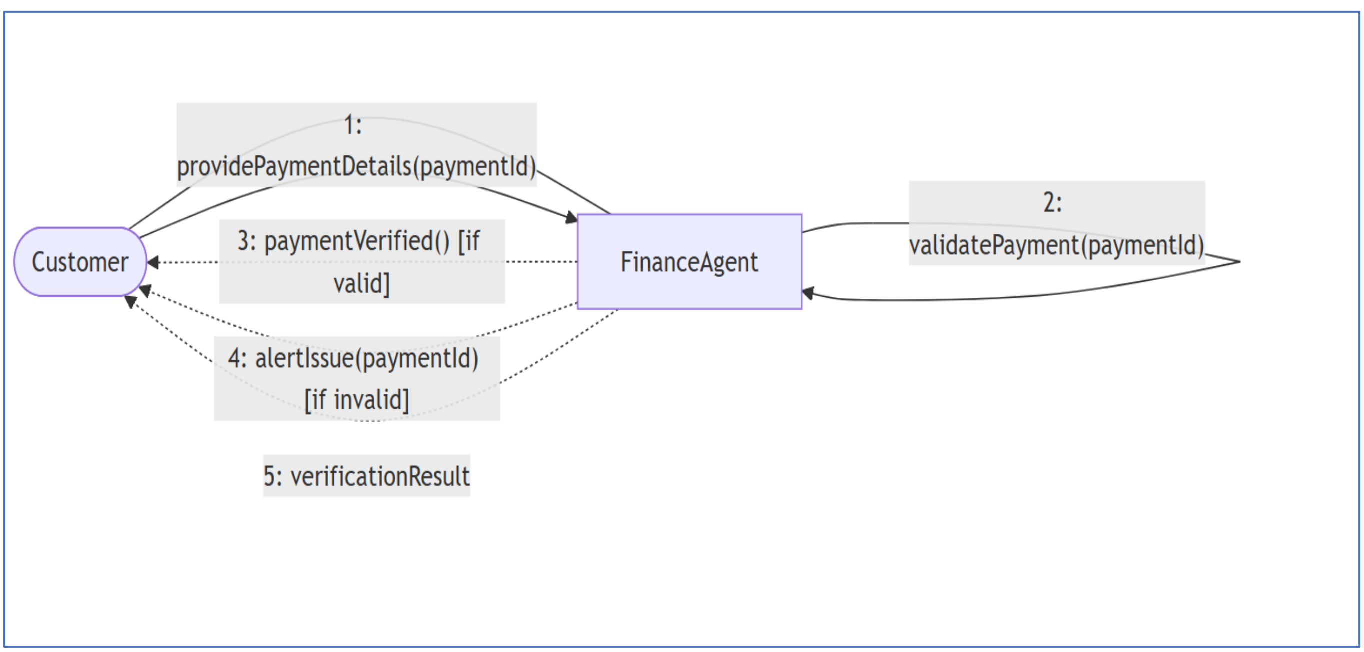
BR_TM_17 – Modify Existing Booking



BR_TM_18 – Reply to Customer Reviews

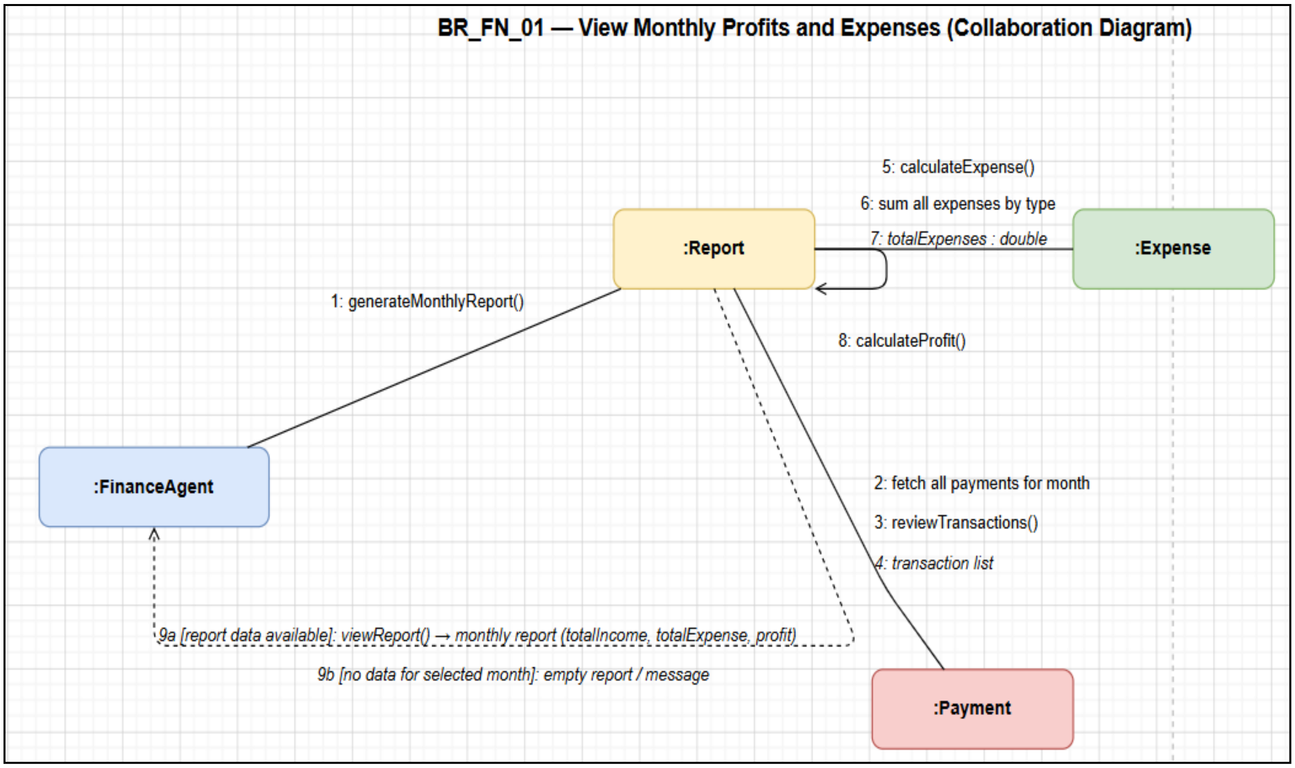


BR_TM_19 – Verify Customer Payments

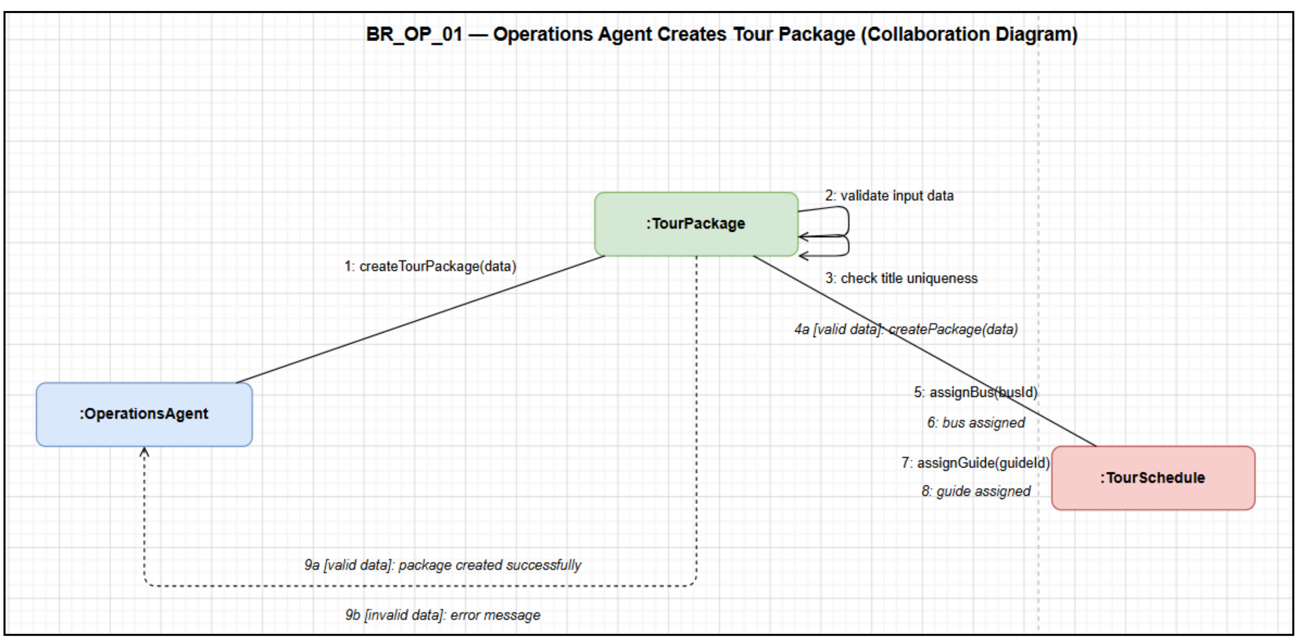


Henri Shehu

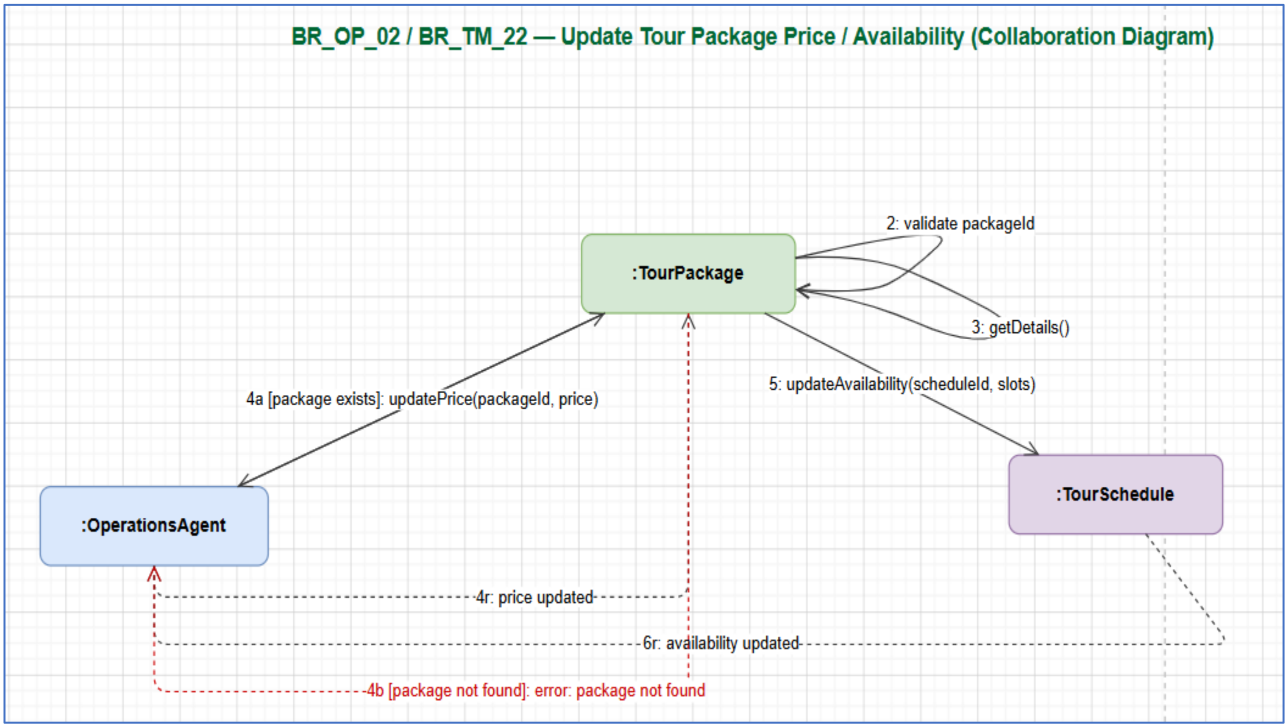
BR_TM_20



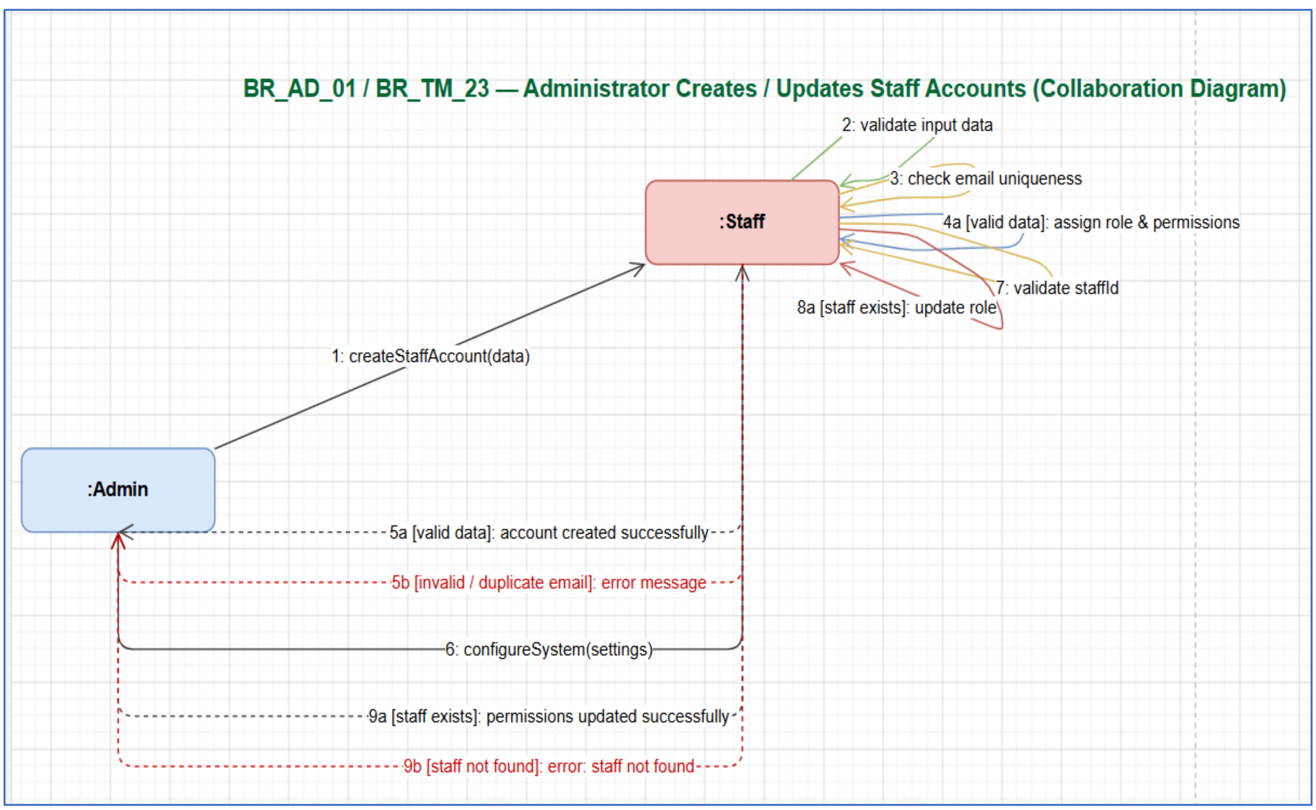
BR_TM_21



BR_TM_22

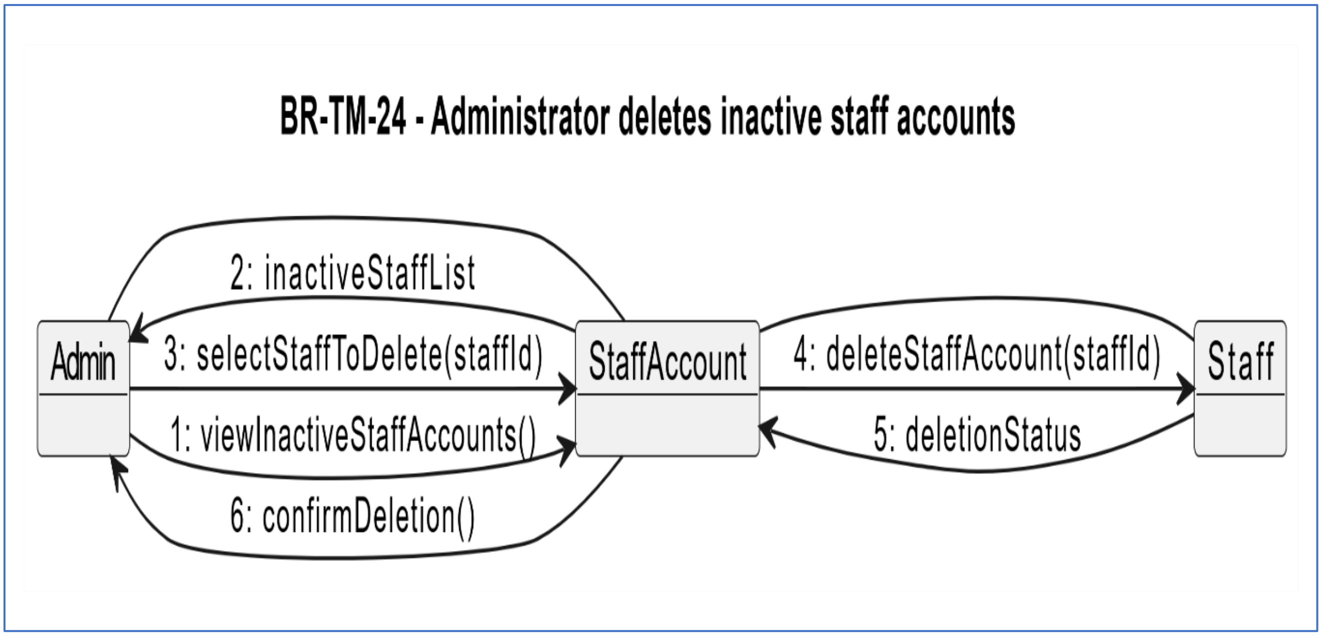


BR_TM_23

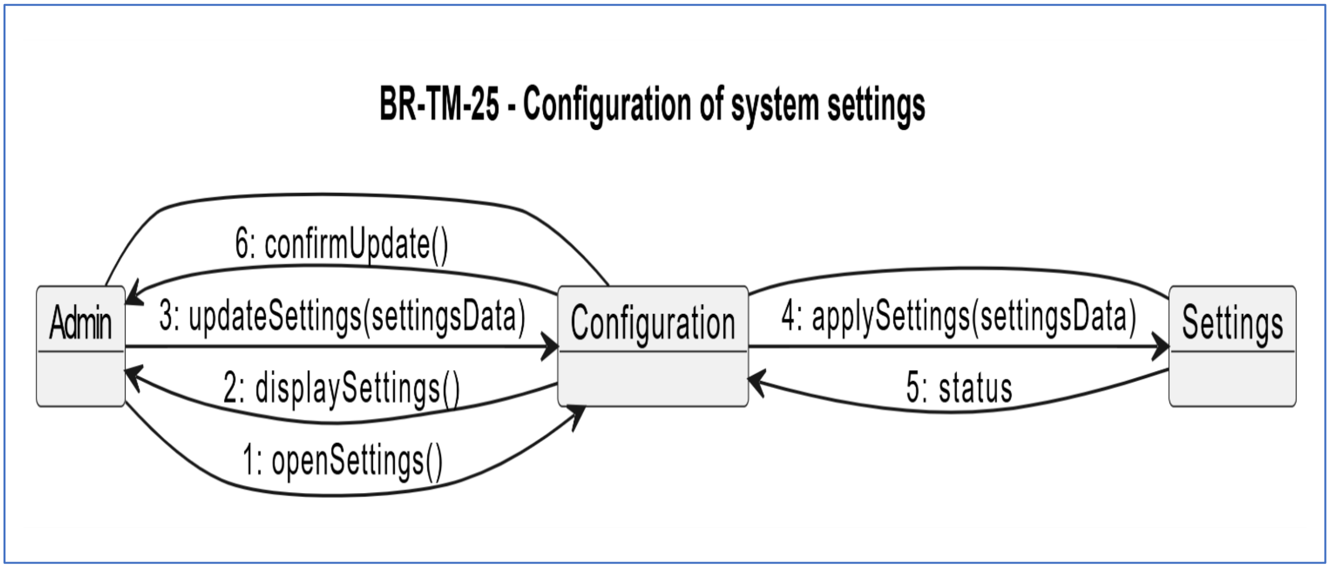


Enian Xhixha

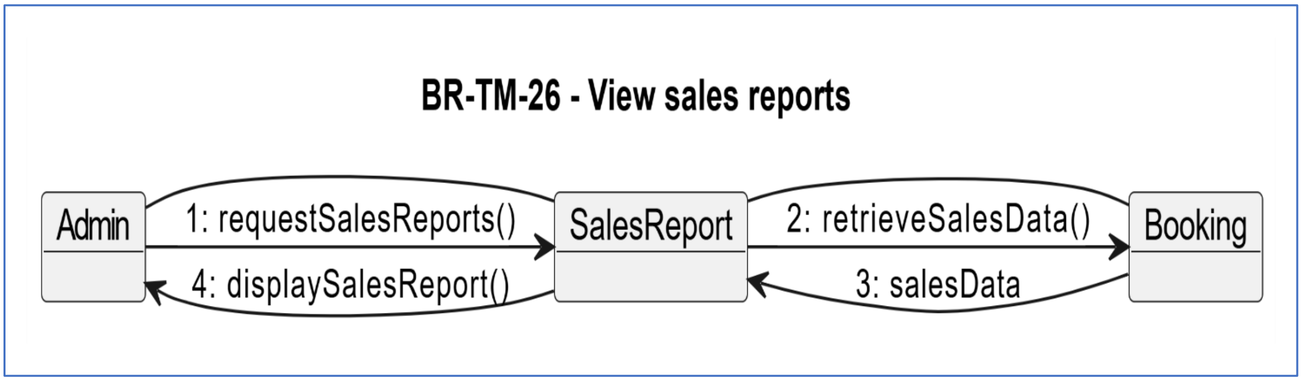
BR-TM-24



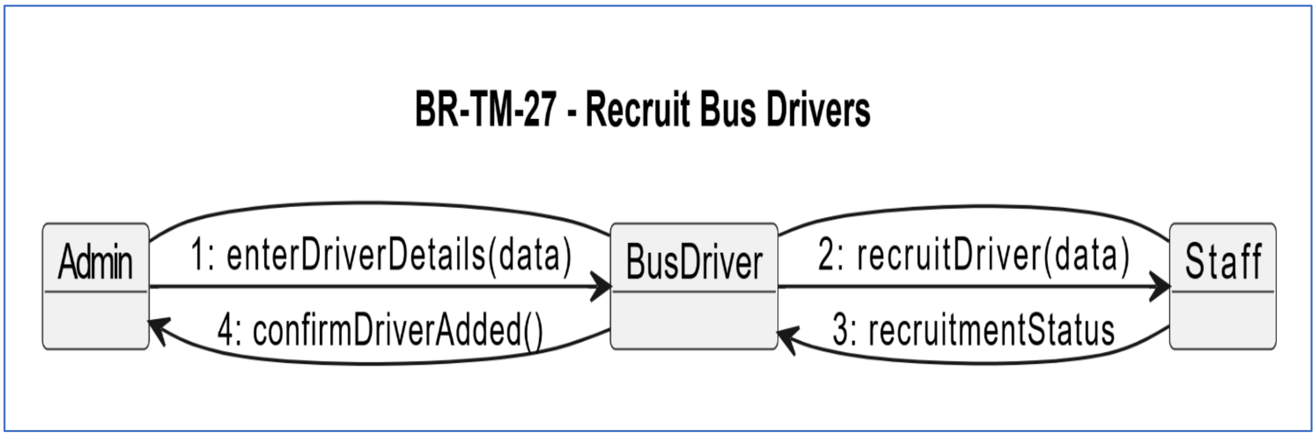
BR-TM-25



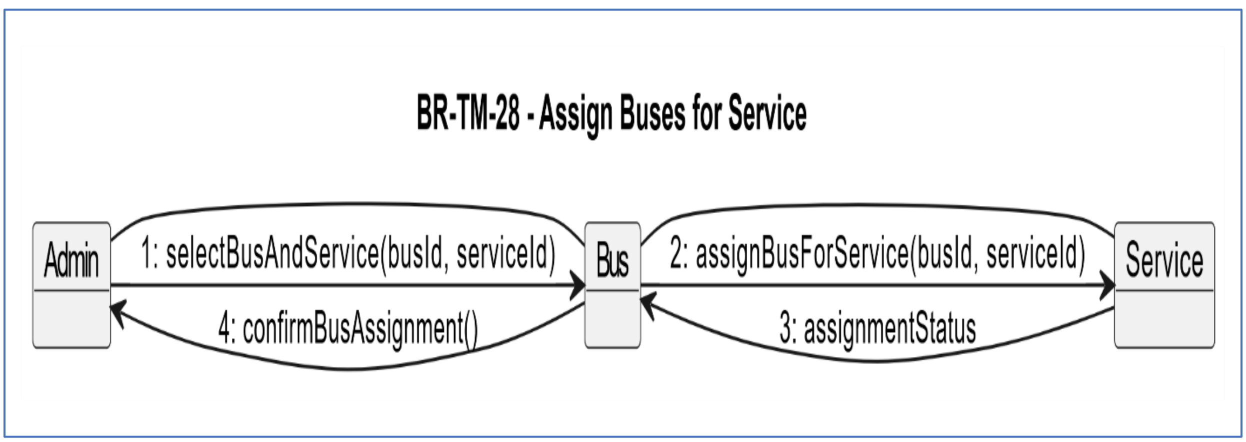
BR-TM-26



BR-TM-27

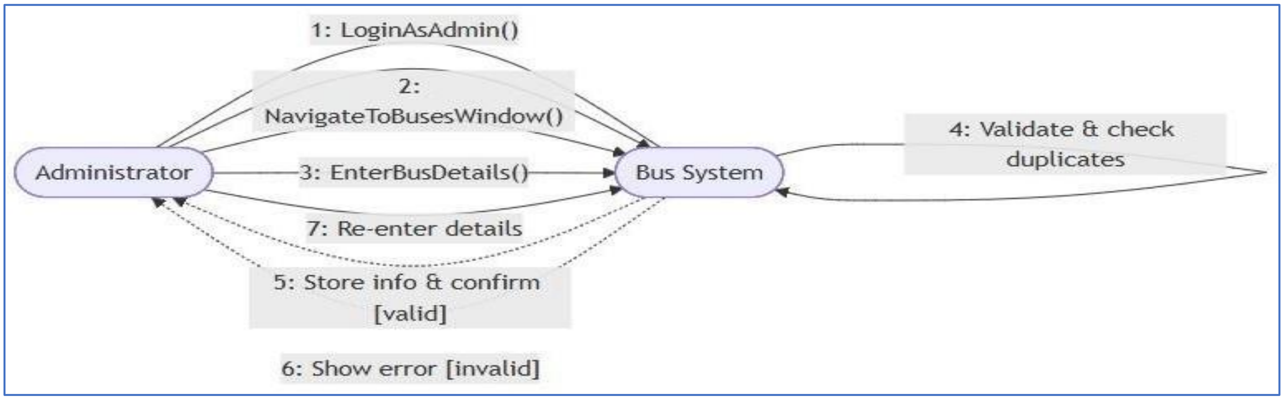


BR-TM-28

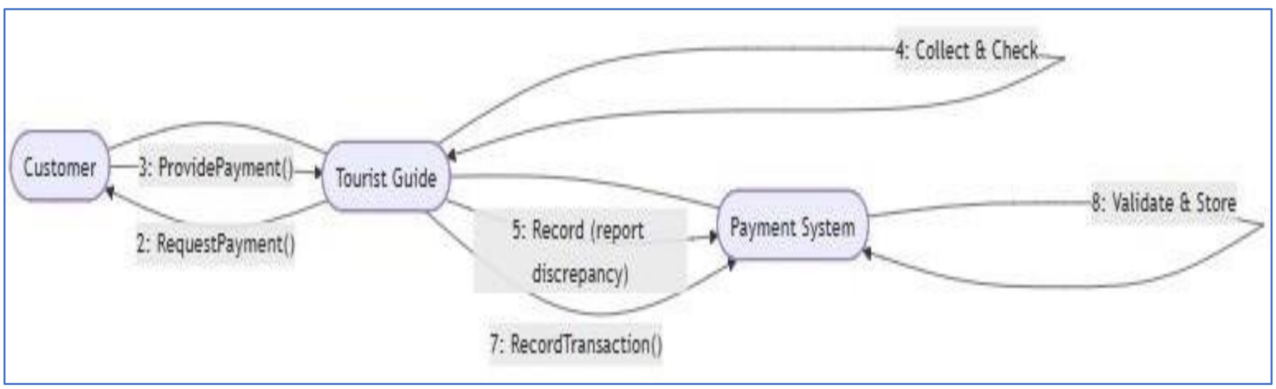


Era Selmanaj

BR_TM_29 – Register number of buses

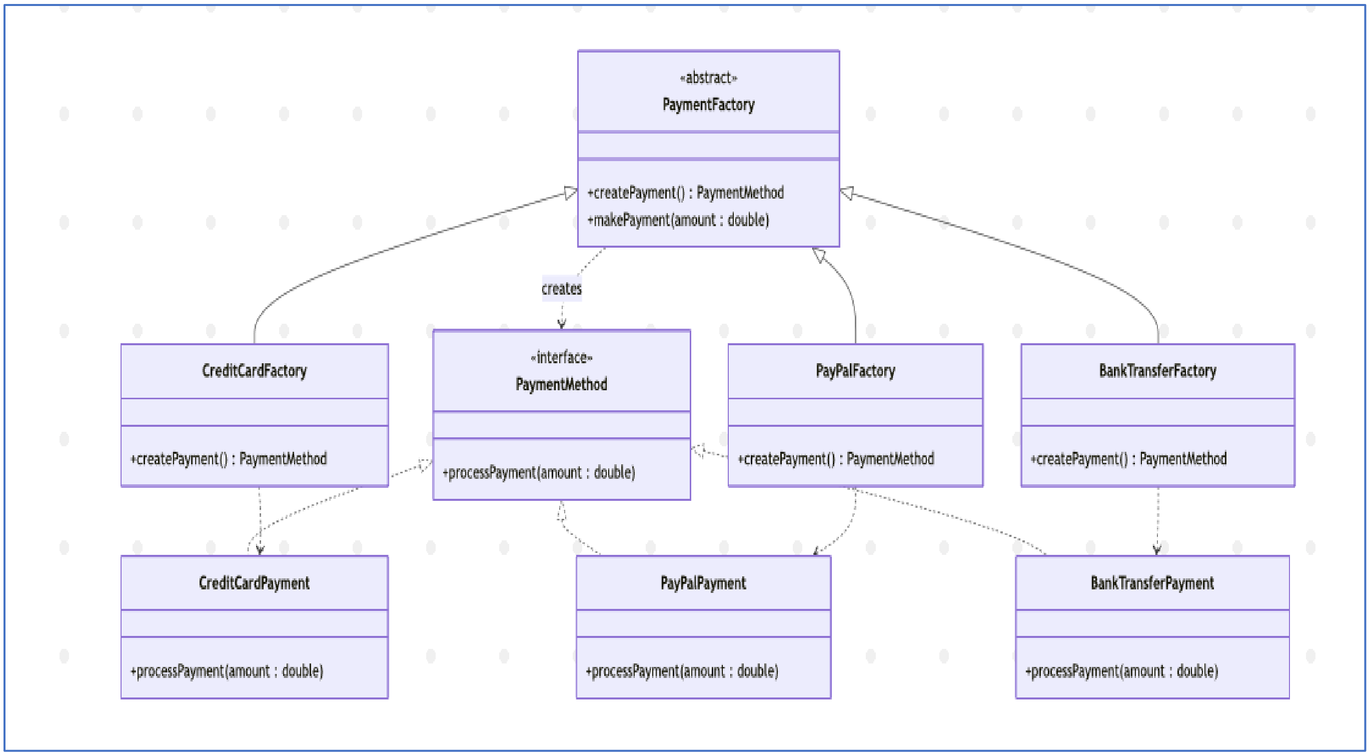


BR_TM_30 – Confirm Clients and Collect Payments



6. Design Patterns

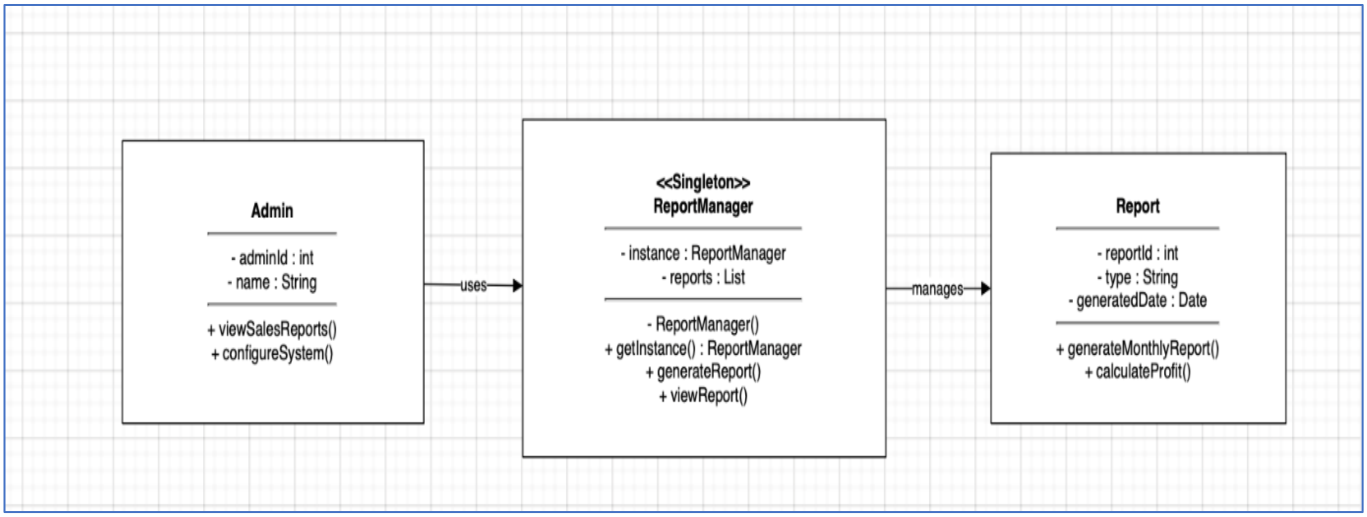
Henri Shehu & Alisa Dhima



Factory Pattern — Justification & Class Diagram

For our project, we picked the Factory Method pattern for handling payments in the Tourist Agency System. We chose this because the system supports multiple payment methods, including credit card, PayPal, and bank transfer, where each payment type requires different processing logic. Using this pattern helps us decouple the code, meaning the part of the system that processes payments does not need to know the exact details of how each payment method is created. As shown in our Class Diagram, we introduced an abstract PaymentFactory with a createPayment() factory method, while concrete factories such as CreditCardFactory, PayPalFactory, and BankTransferFactory are responsible for creating their corresponding PaymentMethod products. This allows the Customer or BookingAgent to simply work with the PaymentMethod interface without depending on specific payment classes. The pattern also makes the system much easier to scale because new payment methods can be added in the future by creating a new factory and product pair without modifying the existing code. This keeps the application maintainable, flexible, and aligned with standard object-oriented design principles.

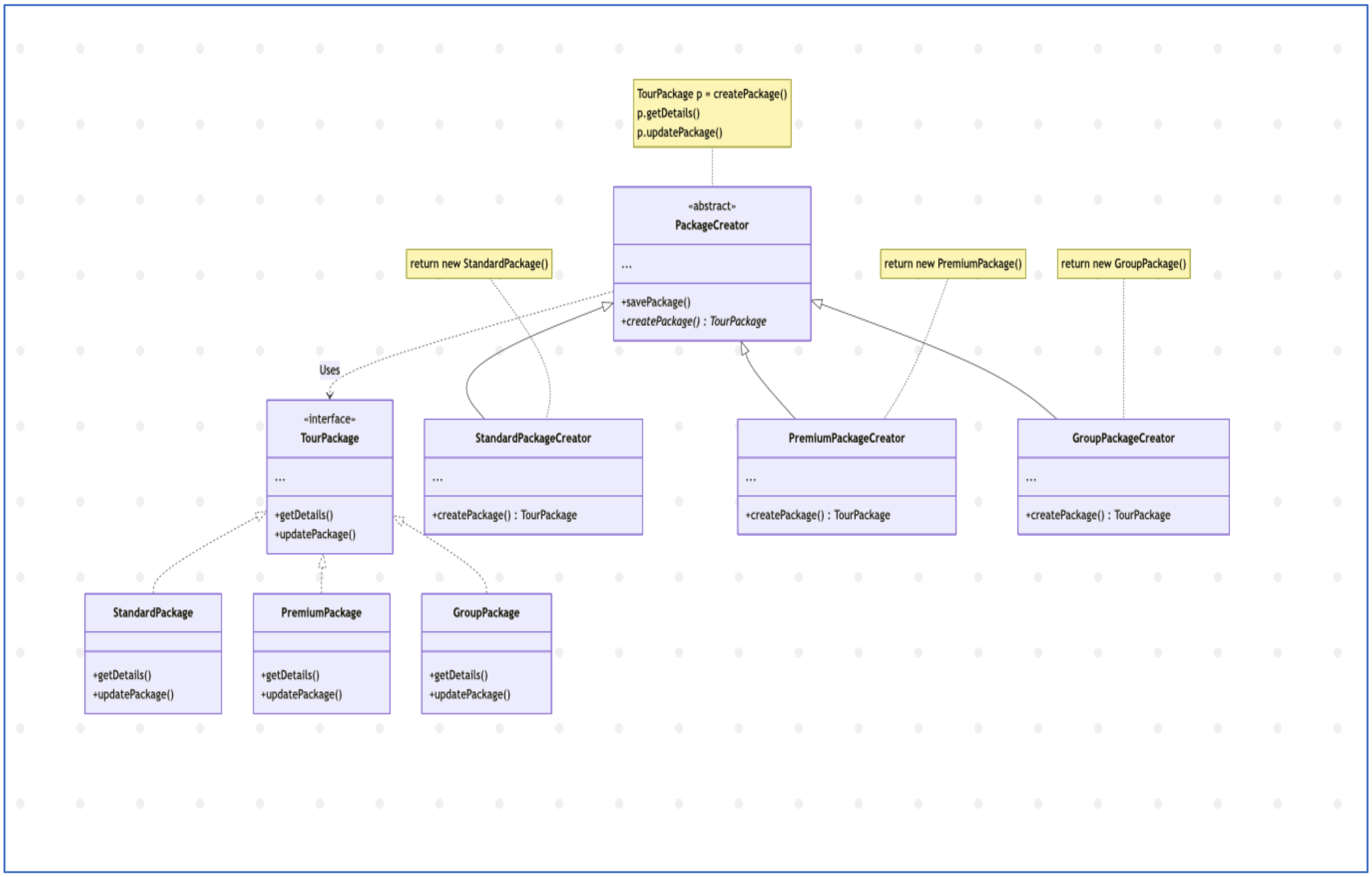
Amar Kruja & Lumturi Bedalli



Singleton Pattern — Justification & Class Diagram

For our project, we picked the Singleton pattern for managing reports in the tourist agency system. We chose this pattern because report generation is a system-wide operation that should be controlled from a single centralized point. If multiple instances of the ReportManager class were allowed, different parts of the system — such as the Admin viewing sales reports or the FinanceAgent generating financial reports — could produce duplicated or inconsistent report data. As shown in our Class Diagram, the ReportManager class is designed with a private constructor and a static getInstance() method, ensuring that only one instance of the class can exist throughout the application. This guarantees that all report-related operations are handled through the same shared object, maintaining consistency and reliability across the system. The Singleton pattern also improves resource management and keeps the report handling logic centralized, making the application easier to maintain and control.

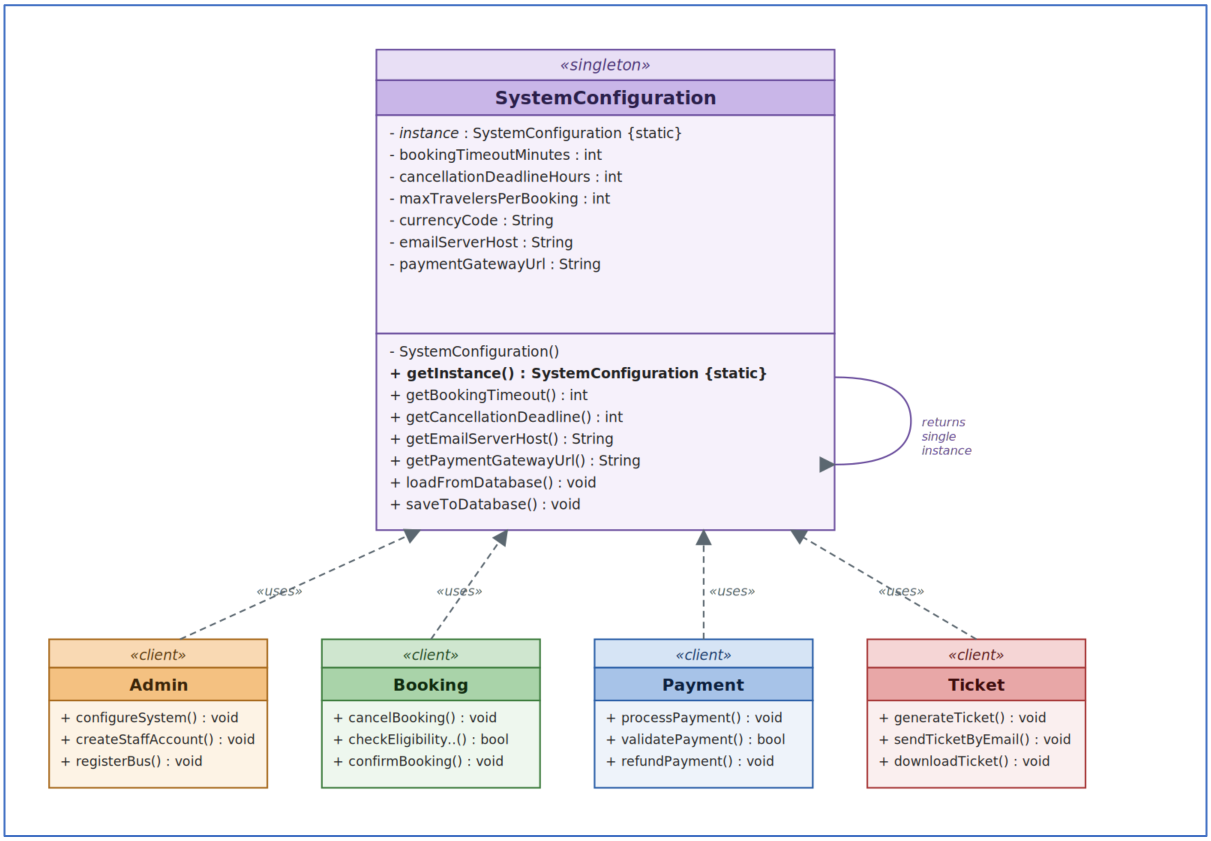
Ermi Xhaferi & Era Selmanaj



Factory Pattern — Justification & Class Diagram

For our project, we picked the Factory Method pattern for creating tour packages. We chose this because, in BR_TM_21, the Operations Agent has to manage three different types of tours (Standard, Premium, and Group). Using this pattern helps us decouple the code, meaning the part of the system that saves the tour doesn't need to know the specific details of how each package is built. As shown in our Class Diagram, this makes it way easier to scale. If we need to add more tour categories in the future, we don't have to change the main creation logic; we just add a new creator class. This keeps our code maintainable and follows the standard design principles we learned.

Amanda Stafasani & Enian Xhixha



Singleton Pattern — Justification & Class Diagram

How the diagram reads:

- SystemConfiguration is the **Singleton** — it holds a private static reference to its own single instance and exposes it through the getInstance() method
- The constructor is marked private (− SystemConfiguration()) so no client can use *new* to build another instance
- Admin, Booking, Payment, and Ticket are the **clients** — they all reach the same configuration object through SystemConfiguration.getInstance() instead of holding their own copies
- The static field *instance* and the static method getInstance() are shown in bold/italic with the {static} tag to emphasise that they belong to the class itself, not to any object

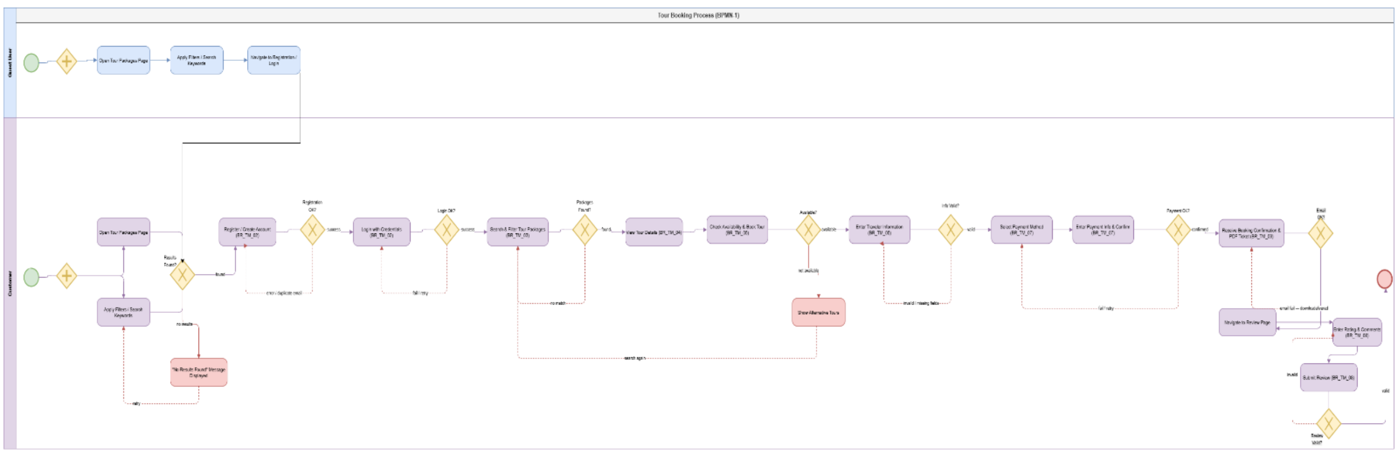
The dashed arrows with open arrowheads represent *dependency* («uses»): each client class depends on SystemConfiguration but does not own it. The looped arrow on the Singleton itself indicates that getInstance() returns a reference to the same single instance every time it is called.

The Singleton pattern is applied to the SystemConfiguration class. Activity diagram BR_TM_25 (Configuration of system settings) shows the Administrator accessing one settings module, updating configurations, and saving them, after which those settings govern behavior across the entire system. Without the Singleton pattern, different parts of the system could load their own copies of these settings, producing inconsistent rules — for example, one booking applying a different cancellation deadline than another, or two payment requests using different gateway URLs. By making SystemConfiguration a Singleton, the cancellation deadline used by Booking.checkEligibilityForCancel() in BR_TM_11, the payment gateway URL used by Payment.processPayment() in BR_TM_07, and the email server host used by Ticket.sendTicketByEmail() in BR_TM_09 all read from the same authoritative source. The pattern guarantees a single point of truth for system-wide configuration and a global, controlled point of access to it.

7. BPMN

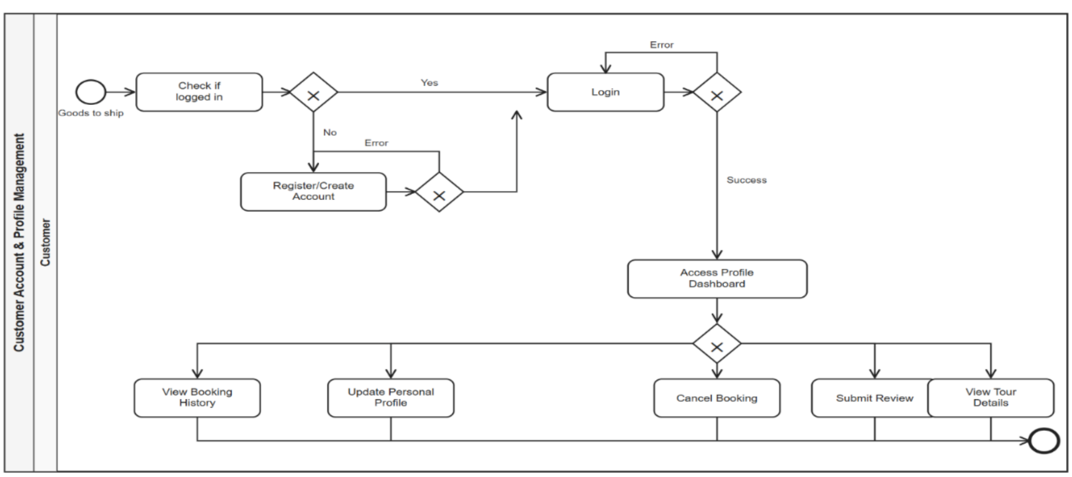
BPMN-1 → Tour Booking Process **Henri Shehu & Alisa Dhima**

Link for image: [Tourist-Agency-Information-System/BPMN.pdf at main · Ermi-Xh/Tourist-Agency-Information-System]



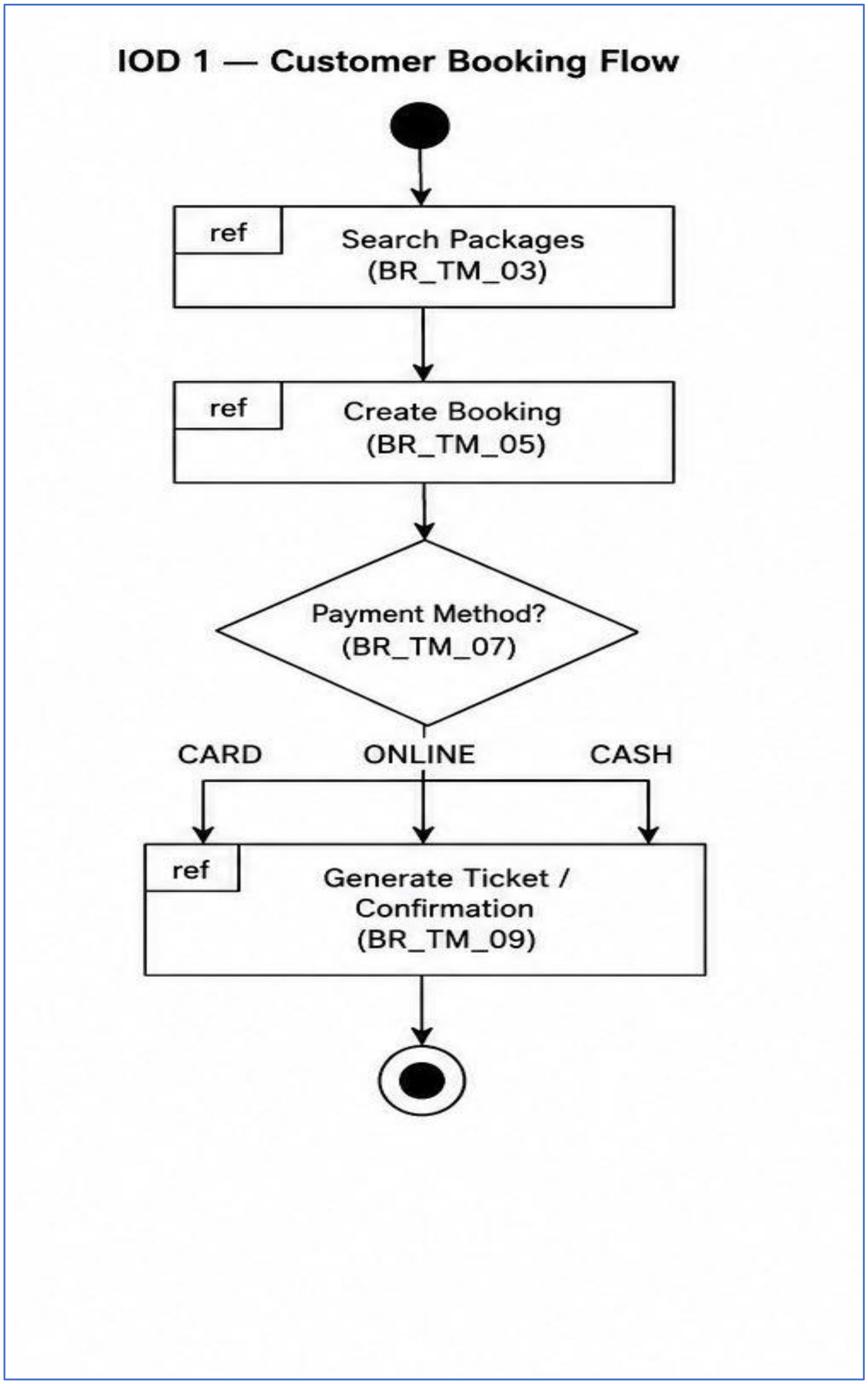
BPMN-2 → Customer Account & Profile Management **Amanda Stafastani & Era Selmanaj**

Link for image: [Tourist-Agency-Information-System/BPMN.pdf at main · Ermi-Xh/Tourist-Agency-Information-System]

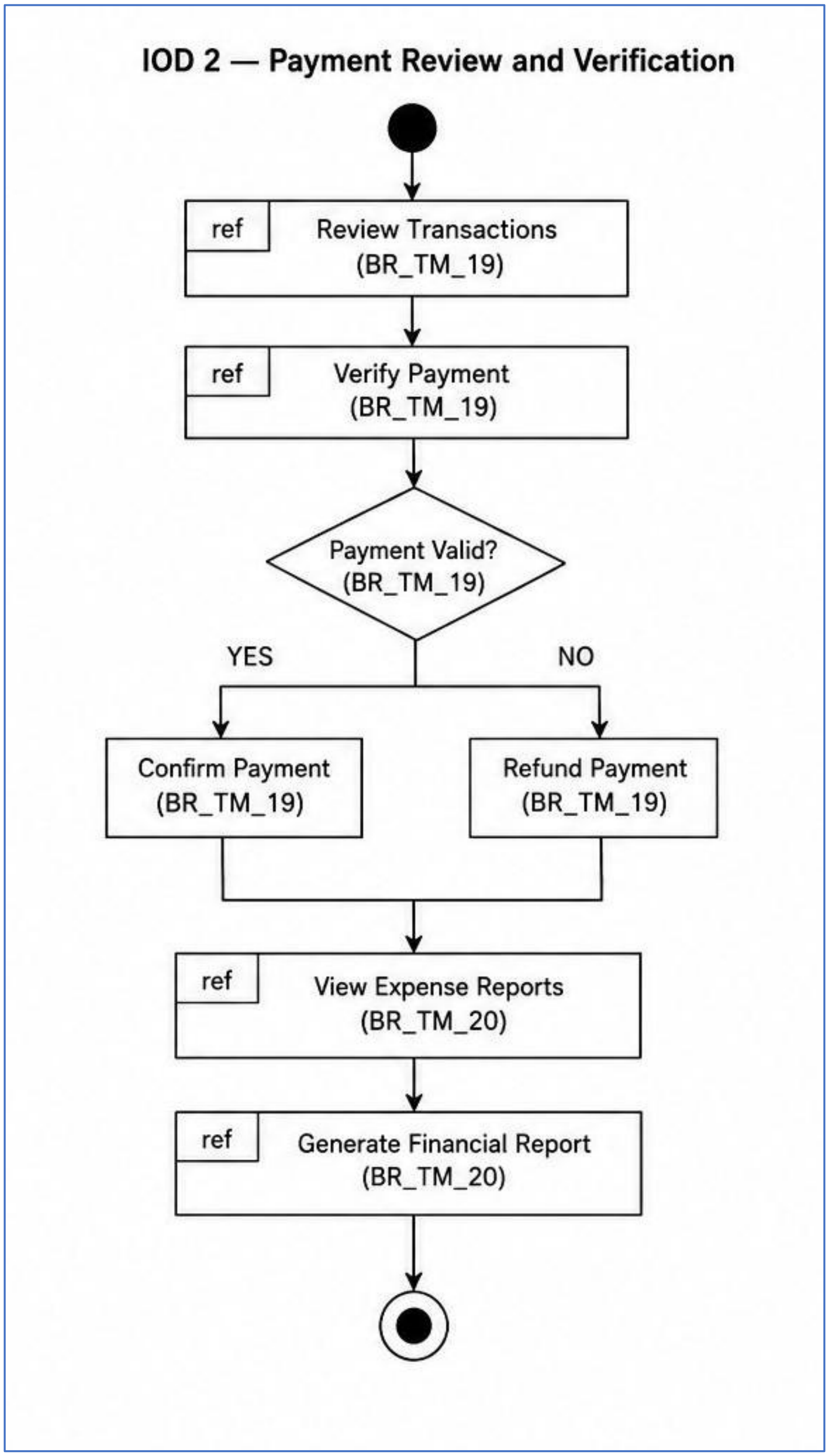


8. Interaction Overview Diagrams

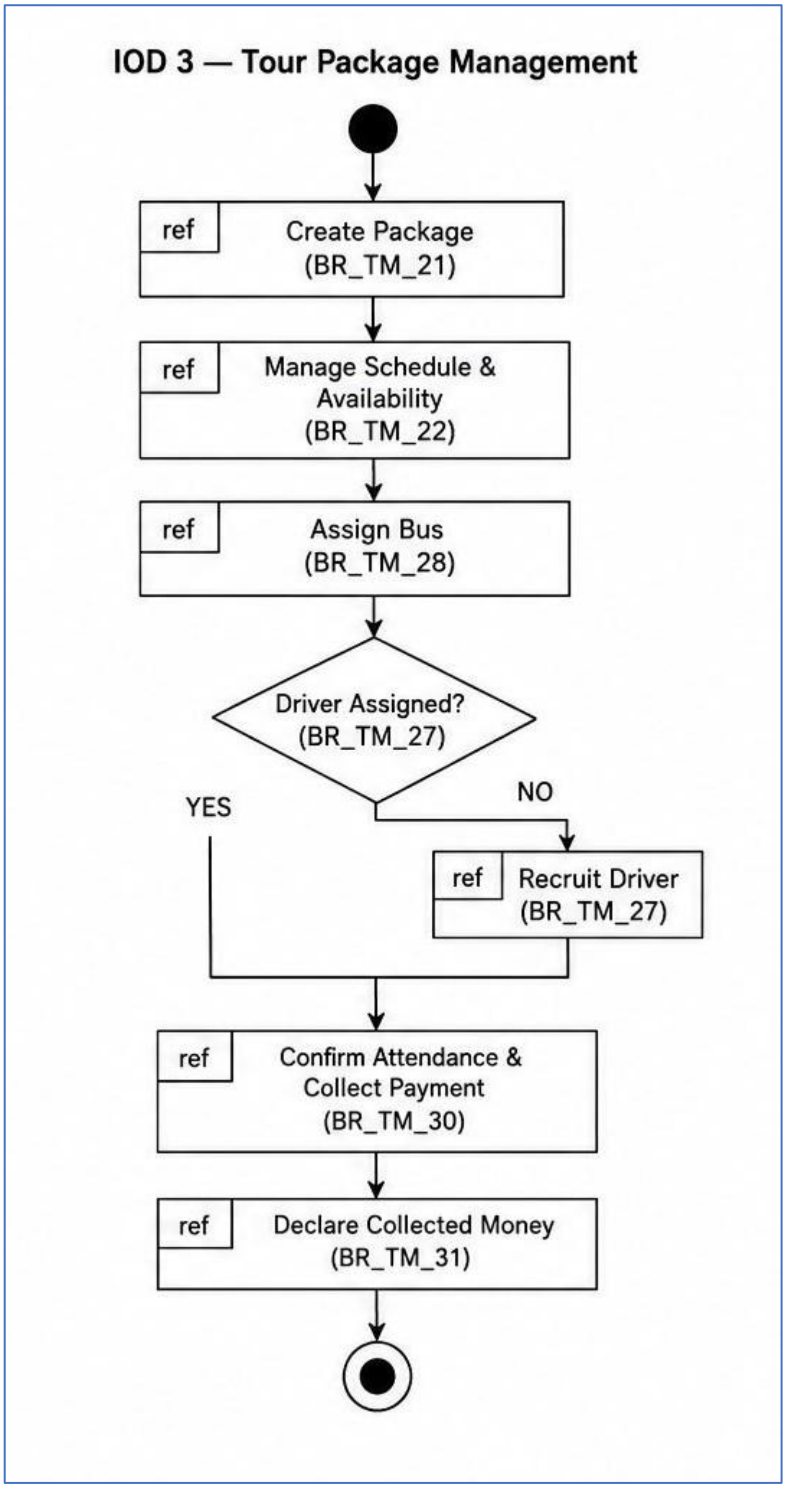
Ermi Xhaferi & Era Selmanaj & Amar Kruja:



Alisa Dhima & Henri Shehu:



Lumturi Bedalli & Amanda Stafasani & Enian Xhixha:



9. Appendix

Priority Level Definitions

The following priority levels are used throughout this document to classify requirements:

• Priority 1 — Must have. The system cannot function correctly or safely without this requirement being met.

• Priority 2 — Important. The requirement significantly improves the system and should be included in the initial release if time allows.

• Priority 3 — Nice to have. The requirement adds value but is not critical for the first version.

Requirement Naming Convention

Requirements follow the naming scheme: BR_TM_[NUMBER], where BR = Business Requirement and TM = Tourist Management. Use cases are named BR_TM_[NUMBER] and map directly to the corresponding activity and sequence diagrams throughout the document.